

A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security

by

Monowar Hossen Sourav (221002578)

Md. Sajeed Mia (221002251)

Md. Naim Ahammed (221002599)

*Thesis (CSE 400) submitted in partial fulfillment of the
requirements for the degree of*

Bachelor of Science in Computer Science and Engineering

Under the supervision of

Jannathul Moawa Hasi



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GREEN UNIVERSITY OF BANGLADESH**

SPRING 2026



© *Monowar Hossen Sourav, Md. Sajeeb Mia, and Md. Naim Ahammed*
All rights reserved

DECLARATION

Title A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security
Authors Monowar Hossen Sourav, Md. Sajeeb Mia, and Md. Naim Ahammed
Student IDs 221002578, 221002251, and 221002599
Supervisor Jannathul Moawa Hasi

We declare that this thesis entitled *A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security* is the result of our own work except as cited in the references. The thesis has not been accepted for any degree and is not concurrently submitted in candidature of any other degree.

Monowar Hossen Sourav
221002578

Department of Computer Science and Engineering
Green University of Bangladesh

Md. Sajeeb Mia
221002251

Department of Computer Science and Engineering
Green University of Bangladesh

Md. Naim Ahammed
221002599

Department of Computer Science and Engineering
Green University of Bangladesh

Date: May 2026

CERTIFICATE OF ENDORSEMENT

This is to certify that the thesis entitled *A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security* by **Monowar Hossen Sourav** (Student ID: 221002578), **Md. Sajeeb Mia** (Student ID: 221002251) and **Md. Naim Ahammed** (Student ID: 221002599) has been successfully defended and examined.

Based on the evaluation and discussion held on **May 2026**, we, the undersigned, hereby endorse the thesis for acceptance and approval.

External Member's Name
External Member, Board: 1

External Member's Position
External Member's Affiliation

Board Member's Name
Board Member, Board: 1

Board Member's Position
Department of Computer Science and Engineering
Green University of Bangladesh

Board Chair's Name
Chair, Board: 1

Board Chair's Position
Department of Computer Science and Engineering
Green University of Bangladesh

Date: May 2026

CERTIFICATE

This is to certify that the thesis entitled **A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security**, submitted by **Monowar Hossen Sourav** (Student ID: 221002578), **Md. Sajeeb Mia** (Student ID: 221002251) and **Md. Naim Ahammed** (Student ID: 221002599) are undergraduate students of the **Department of Computer Science and Engineering** has been examined. Upon recommendation by the examination committee, we hereby accord our approval of it as the presented work and submitted report fulfill the requirements for its acceptance in partial fulfillment for the degree of *Bachelor of Science in Computer Science and Engineering*.

Jannathul Moawa Hasi
Lecturer

Department of Computer Science and Engineering
Green University of Bangladesh

Name of the Chairperson
Chairperson

Department of Computer Science and Engineering
Green University of Bangladesh

Place: Narayanganj

Date: May 2026

ACKNOWLEDGMENT

We would like to express our deep and sincere gratitude to our research supervisor, *Jannathul Moawa Hasi*, for giving us the opportunity to conduct research and providing invaluable guidance throughout this work. Her dynamism, vision, sincerity and motivation have deeply inspired us. She has taught us the methodology to carry out the work and to present the works as clearly as possible. It was a great privilege and honor to work and study under her guidance.

We are greatly indebted to our honorable teachers of the Department of Computer Science and Engineering at the Green University of Bangladesh who taught us during the course of our study. Without any doubt, their teaching and guidance have completely transformed us to the persons that we are today.

We are extremely thankful to our parents for their unconditional love, endless prayers and caring, and immense sacrifices for educating and preparing us for our future. We would like to say thanks to our friends and relatives for their kind support and care.

Finally, we would like to thank all the people who have supported us to complete the project work directly or indirectly.

Monowar Hossen Sourav, Md. Sajeeb Mia and Md. Naim Ahammed

Green University of Bangladesh

Date: May 2026



Dedicated to Almighty
Allah —
*All praise to Him for His
endless mercy and guidance.
Dedicated to my beloved
parents for their love and
support, and to all who
inspired and encouraged me
along the way.*

– Monowar Hossen Sourav

Dedicated to Almighty
Allah —
*All praise to Him for His
endless mercy and guidance.
Dedicated to my beloved
parents for their love and
support.*

– Md. Sajeeb Mia

Dedicated to Almighty
Allah —
*All praise to Him for His
endless mercy and guidance.
Dedicated to my beloved
parents for their love and
support.*

– Md. Naim Ahammed

ABSTRACT

Electronic voting systems face core security challenges in simultaneously providing ballot privacy, vote integrity, voter anonymity, coercion resistance, and verifiability. Existing blockchain-based e-voting solutions address only subsets of these requirements, leaving critical gaps in coercion resistance and post-quantum security. This thesis presents a blockchain-based e-voting framework that integrates a multi-protocol cryptographic stack into a unified voting pipeline: Paillier homomorphic encryption with Helios-style CDS zero-knowledge proofs taken directly on the ciphertexts for vote validity, Pedersen commitments for Self-Masking Deniable Credential (SMDC) slot weights, linkable ring signatures for anonymous ballot submission, Samplable Anonymous Aggregation (SA²) for two-server vote privacy, Kyber-768 for post-quantum hybrid encryption, and Damgård–Jurik–Nielsen threshold decryption for distributed key management.

The novel SMDC mechanism issues $k = 5$ slot commitments per voter (1 real, 4 fake) and keys the real-slot index by a server-held HMAC secret that is disclosed to the legitimate voter only through a duress-bound reveal endpoint, gated on a registered behavioural signal. Together with a silent-discard short-circuit at cast time, these mechanisms provide coercion resistance against registration-time, post-registration, and cast-time coercers in practical blockchain voting contexts.

The system is implemented in approximately 20,000 lines of Go and deployed on Hyperledger Fabric v2.5. Performance evaluation confirms an $O(N \cdot m)$ total tally complexity ($O(N)$ per candidate) with a constant per-vote per-candidate cost of approximately $6.1 \mu\text{s}$, empirically validated from 100 to 100,000 voters. The complete cast pipeline reaches a per-vote primitive floor of 68.3 ms (with registration-time Pedersen and SMDC issuance amortised across all of a voter’s casts), and Hyperledger Caliper benchmarks of the deployed chaincode achieve approximately 200 TPS with 100% success at 10,000 transactions. Thirteen end-to-end election simulations spanning 100 to 100,000 voters and both blockchain modes confirm 100% tally integrity with a tight 81–111 ms per-vote latency band.

Compared with prior work, the proposed framework reduces the asymptotic tally cost from $O(N \cdot m^2)$ (ISE-Voting) to $O(N \cdot m)$ — a factor- m improvement — replaces BP-Vot’s $\geq 98\%$ differential-privacy approximation with 100% exact integer tallies, and adds post-quantum protection at only 5.35 ms per vote ($\approx 7.8\%$ of the per-vote primitive floor) via the Kyber-768 hybrid encryption layer. No surveyed prior framework combines this set of security properties with a Caliper-benchmarked Hyperledger Fabric deployment in a single architecture.

Keywords: blockchain, e-voting, homomorphic encryption, zero-knowledge proofs, ring

signatures, coercion resistance, post-quantum cryptography, Hyperledger Fabric, deniable credentials, anonymous aggregation

Contents

List of Acronyms	1
1 Introduction	2
1.1 Background	2
1.1.1 Electronic Voting Systems	2
1.1.1.1 Evolution of electronic voting	2
1.1.1.2 E-voting system architecture	3
1.1.2 Security Challenges in E-Voting	4
1.1.2.1 Traditional security approaches	4
1.1.2.2 Cryptographic approaches	4
1.1.3 Traditional E-Voting vs. Blockchain-Based E-Voting	5
1.1.3.1 Core differences	5
1.1.3.2 Advantages of blockchain-based approach	5
1.1.4 Blockchain Technology in E-Voting	5
1.1.5 Cryptographic Protocols for E-Voting	6
1.1.6 State of the Art in the Modern Context	7
1.1.7 Importance and Relevance of the Topic	7
1.2 Motivation	8
1.3 Scope and Limitations	9
1.3.1 In Scope	9
1.3.2 Out of Scope	10
1.4 Problem Statement	10
1.5 Research Objectives	11
1.6 Research Contributions	11
1.7 Thesis Outline	13
1.8 Conclusion	14
2 Related Works	15
2.1 Introduction	15
2.2 Review of Selected Works	15
2.2.1 Improved Secure and Efficient E-Voting on Blockchain	15
2.2.1.1 Methodology	15

2.2.1.2	Contributions	16
2.2.1.3	Limitations	16
2.2.2	Blockchain-Based E-Voting with Differential Privacy	16
2.2.2.1	Methodology	17
2.2.2.2	Contributions	17
2.2.2.3	Limitations	17
2.2.3	Biometric-Authenticated Blockchain Voting	18
2.2.3.1	Methodology	18
2.2.3.2	Contributions	18
2.2.3.3	Limitations	18
2.2.4	Timed-Release E-Voting with Paillier Homomorphic Encryption	19
2.2.4.1	Methodology	19
2.2.4.2	Contributions	19
2.2.4.3	Limitations	19
2.2.5	Scalable Coercion-Resistant Blockchain Voting	20
2.2.5.1	Methodology	20
2.2.5.2	Contributions	20
2.2.5.3	Limitations	20
2.2.6	Survey: Blockchain for Securing Electronic Voting Systems	21
2.2.6.1	Methodology	21
2.2.6.2	Contributions	21
2.2.6.3	Limitations	21
2.3	Background Technology	22
2.3.1	Paillier Public-Key Cryptosystem	22
2.3.2	Pedersen Commitments	22
2.3.3	Zero-Knowledge Σ -Protocols	22
2.3.4	Linkable Ring Signatures	23
2.3.5	Threshold Cryptography and Secret Sharing	23
2.3.6	Cryptographic Hash Functions and Message Authentication	23
2.3.7	Post-Quantum Key Encapsulation: ML-KEM (Kyber)	23
2.3.8	Authenticated Symmetric Encryption: AES-GCM	24
2.3.9	Hyperledger Fabric	24
2.3.10	Samplable Anonymous Aggregation	24
2.4	Comparison Among State-of-the-Art Works	24
2.5	Summary	26

3	System Design and Cryptographic Protocols	28
3.1	Introduction	28
3.2	Proposed Framework and Assumptions	29
3.3	Notation	32
3.4	Cryptographic Protocol Formulation	33
3.4.1	Paillier Homomorphic Encryption	33
3.4.1.1	Key Generation	33
3.4.1.2	Encryption	34
3.4.1.3	Decryption	34
3.4.1.4	Homomorphic Properties	34
3.4.2	Pedersen Commitments	35
3.4.2.1	Parameter Generation	35
3.4.2.2	Commitment	35
3.4.3	Zero-Knowledge Proofs (Σ -Protocol)	36
3.4.3.1	Binary Proof on Paillier Ciphertexts	36
3.4.3.2	Sum-to-One Proof on Paillier Ciphertexts	37
3.4.3.3	Relationship to the SMDC Slot Proofs	38
3.4.4	Linkable Ring Signatures	38
3.4.4.1	Key Generation	39
3.4.4.2	Signing	39
3.4.4.3	Verification and Linkability	40
3.4.5	SMDC: Self-Masking Deniable Credentials	40
3.4.5.1	Credential Generation	41
3.4.5.2	Coercion Resistance Property	41
3.4.5.3	Credential Verification	42
3.4.6	SA ² : Samplable Anonymous Aggregation	42
3.4.6.1	Per-candidate Vote Splitting	43
3.4.6.2	Per-candidate Reconstruction	43
3.4.7	Kyber768 Post-Quantum Hybrid Encryption	44
3.4.7.1	Hardness Assumption and Parameter Selection	44
3.4.7.2	KEM-DEM Hybrid Construction	44
3.4.7.3	Scope and Limitations of Post-Quantum Coverage	45
3.4.8	Threshold Paillier Decryption	45
3.4.8.1	Key Distribution	46
3.4.8.2	Partial Decryption with ZK Correctness Proof	46
3.4.8.3	Verification and Combination	46
3.4.9	Behavioural Duress Detection	46
3.5	Vote Casting Pipeline	48
3.6	Blockchain Integration	52
3.7	Engineering and Societal Considerations	53

3.7.1	Societal Impact	53
3.7.2	Sustainability	53
3.7.3	Accessibility	54
3.7.4	Ethical Considerations	54
3.7.5	Privacy and Data Protection	55
3.8	Security Analysis	55
3.8.1	Computational Security: Game-Based Reductions	55
3.8.1.1	Notational Conventions for Proofs	56
3.8.1.2	Hardness Assumptions	56
3.8.1.3	Theorem 1: Ballot Indistinguishability (DCRA)	57
3.8.1.4	Theorem 2: Vote Validity (Soundness of CDS OR-proofs in the ROM)	59
3.8.1.5	Theorem 3: Voter Anonymity (DDH)	62
3.8.1.6	Theorem 4: Coercion Resistance (Pedersen Perfect Hiding + JCJ)	63
3.8.1.7	Theorem 5: SA ² Aggregation Privacy (DCRA + A1)	65
3.8.1.8	Theorem 6: Post-Quantum Hybrid (Module-LWE + DCRA)	67
3.8.1.9	Composition Discussion: Toward UC Security	69
3.8.2	Duress Detection: Security Properties	71
3.9	Conclusion	71
4	Implementation, Results and Discussion	73
4.1	Introduction	73
4.2	Implementation Environment	73
4.3	Performance Metrics	74
4.4	Experimental Results	75
4.4.1	Cryptographic Micro-Benchmarks	75
4.4.1.1	Post-Quantum Layer Cost Analysis	76
4.4.2	End-to-End Pipeline Benchmark (Per-Primitive Decomposition)	77
4.4.3	Scalability Validation	78
4.4.3.1	Homomorphic Tally Scalability	78
4.4.3.2	Linear-in- m Total Scaling (Candidate-Count Validation)	80
4.4.3.3	Ring Signature Scalability	81
4.4.4	Blockchain Performance	82
4.4.5	End-to-End Election Simulation	84
4.5	Discussion	86
4.5.1	Pipeline Bottleneck Analysis	87
4.5.2	Scalability Significance	88
4.5.3	Blockchain Performance Analysis	89
4.5.4	Practical Deployment Considerations	89

4.6	Test Coverage Analysis	90
4.7	Tier 2, Symbolic Protocol Verification with ProVerif	91
4.8	Comparative Analysis	92
4.8.1	Comparison With Related Works	92
5	Conclusion and Future Work	95
5.1	Introduction	95
5.2	Summary of the Study	95
5.3	Limitations and Future Work	96
	References	100
A	ProVerif Formal Verification Artefacts	105
A.1	Reproducing the Verification	105
A.2	Ballot Privacy: <code>privacy.pv</code>	105
A.3	Individual Verifiability: <code>verifiability.pv</code>	107
A.4	Voter Eligibility: <code>eligibility.pv</code>	109
A.5	Summary	110

List of Figures

3.1	Proposed System Architecture Overview	30
3.2	SMDC Deniable Credential Slot Structure	42
3.3	Twenty-Step Vote Casting Pipeline	48
3.4	Voter Registration and Casting Activity	49
3.5	Vote Casting Sequence Diagram	50
4.1	Cryptographic Primitive Per-Operation Cost	76
4.2	Vote Pipeline Per-Vote Primitive Breakdown	78
4.3	Homomorphic Tally Scaling with Voter Count	80
4.4	Tally Time versus Candidate Count	81
4.5	Ring Signature Sign and Verify Scaling	82
4.6	Caliper saturation scenario: success rate collapses as request rate climbs . .	83
4.7	Caliper sustained-load scenario: state-bound degradation at fixed 200 TPS .	84

List of Tables

2.1	Comparison of State-of-the-Art Blockchain E-Voting Approaches	25
3.1	Mathematical Notation	32
4.1	Implementation Environment	74
4.2	Cryptographic Micro-Benchmark Results	75
4.3	Vote Casting Pipeline: Per-Vote Primitive Breakdown	77
4.4	Homomorphic Tally Scalability Results	79
4.5	Candidate-Independent Tally Scaling ($N = 1,000$)	80
4.6	Ring Signature Scalability	81
4.7	Hyperledger Caliper benchmark results — saturation vs sustained-load scenarios	82
4.8	End-to-End Election Simulation: Mock vs Real Hyperledger Fabric	85
4.9	Test Coverage by Package	90
4.10	ProVerif Formal Verification Results	92
4.11	Comparison of the Proposed Method with Existing Works	93

List of Acronyms

AES	Advanced Encryption Standard
CAI	Cast-As-Intended
CSP	Cloud Service Provider
DCRA	Decisional Composite Residuosity Assumption
DJN	Damgård-Jurik-Nielsen
DRE	Direct Recording Electronic
GCM	Galois/Counter Mode
gRPC	Google Remote Procedure Call
HE	Homomorphic Encryption
HMAC	Hash-based Message Authentication Code
IoT	Internet of Things
JCJ	Juels-Catalano-Jakobsson
LWE	Learning With Errors
ML-KEM	Module-Lattice Key Encapsulation Mechanism
MSP	Membership Service Provider
NIST	National Institute of Standards and Technology
NIZK	Non-Interactive Zero-Knowledge
PQ	Post-Quantum
RSA	Rivest-Shamir-Adleman
SA ²	Samplable Anonymous Aggregation
SHA	Secure Hash Algorithm
SMDC	Self-Masking Deniable Credentials
SSI	Self-Sovereign Identity
TLS	Transport Layer Security
TPS	Transactions Per Second
TRE	Timed-Release Encryption
ZK	Zero-Knowledge
ZKP	Zero-Knowledge Proof

1. Introduction

1.1 Background

Electronic voting (e-voting) has emerged as a practical approach to modernizing democratic processes by replacing traditional paper-based balloting with digital systems. As nations worldwide seek to improve voter accessibility, reduce administrative costs, and accelerate result tabulation, e-voting systems have gained considerable attention from both the research community and government institutions. However, adopting electronic voting introduces core security challenges that do not exist in conventional voting: ballot privacy, vote integrity, coercion resistance, and universal verifiability must all be simultaneously guaranteed in a system where digital data can be copied, intercepted, or manipulated.

The integration of blockchain technology [2] with advanced cryptographic protocols offers a promising foundation for addressing these challenges. By leveraging the immutability and transparency properties of distributed ledgers alongside homomorphic encryption, zero-knowledge proofs, and anonymous credential schemes, it becomes possible to construct voting systems that provide mathematical guarantees of security rather than relying solely on organizational trust.

1.1.1 Electronic Voting Systems

Electronic voting encompasses any voting method where the voter's intent is recorded through electronic means. These systems range from Direct Recording Electronic (DRE) machines used at physical polling stations to internet-based remote voting platforms that allow participation from any location.

1.1.1.1 Evolution of electronic voting

The evolution of electronic voting can be traced through several distinct generations. First-generation systems, deployed in the 1990s and early 2000s, were primarily DRE machines that digitized the ballot-marking process but stored votes in centralized databases controlled by election authorities. These systems improved tallying speed but introduced single points of failure and required voters to trust the machine vendor and election administrators completely.

Second-generation systems introduced cryptographic techniques such as mix-nets and blind signatures to provide some degree of voter anonymity. Notable examples include the Helios voting system, which pioneered web-based verifiable elections for organizational contexts. However, these systems relied on trusted third parties for key management and ballot processing, leaving them vulnerable to insider attacks and coercion.

Third-generation systems, emerging from 2018 onward, began integrating blockchain technology to provide immutable vote records and decentralized trust. Systems such as Voatz (used in US military overseas voting) and Agora (deployed as an observer in Sierra Leone's 2018 elections) represented early attempts at blockchain-based voting. However, these systems were criticized for weak cryptographic guarantees; a 2020 MIT security analysis of Voatz [3] revealed critical vulnerabilities including client-side attacks, server compromise possibilities, and vote manipulation risks.

Current e-voting research focuses on combining multiple cryptographic primitives into multi-layered security frameworks. The proposed framework represents this generation by integrating a multi-protocol cryptographic stack with a Caliper-benchmarked Hyperledger Fabric deployment to simultaneously address privacy, integrity, coercion resistance, and post-quantum security.

1.1.1.2 E-voting system architecture

A modern e-voting system architecture typically consists of four functional layers that interact to ensure secure vote processing:

1. **Authentication Layer:** Responsible for verifying voter identity and eligibility. This may involve biometric verification, digital certificates, or credential-based authentication. The authentication mechanism must confirm the voter's right to participate without revealing their identity to other system components.
2. **Computation Layer:** Handles the cryptographic operations that protect ballot privacy and ensure vote validity. This includes encryption, commitment generation, zero-knowledge proof construction, and digital signature creation. The computational demands of this layer directly impact the per-vote processing time and overall system throughput.
3. **Privacy Layer:** Ensures that individual votes cannot be linked to specific voters even by system administrators. Privacy-preserving techniques include mix networks, homomorphic encryption, secret sharing, and anonymous aggregation protocols. The strength of this layer determines whether the system can resist surveillance and coercion attacks.
4. **Storage Layer:** Provides persistent, tamper-evident storage of vote records. Blockchain technology serves this function by creating an append-only ledger that is distributed

across multiple nodes, making retrospective vote manipulation computationally infeasible.

1.1.2 Security Challenges in E-Voting

The security requirements of electronic voting are considerably more demanding than those of typical information systems. A voting system must simultaneously satisfy properties that are often in tension with one another; for example, votes must be both secret (unlinked to voters) and verifiable (provably counted correctly).

1.1.2.1 Traditional security approaches

Traditional approaches to e-voting security relied primarily on organizational controls and access restrictions. Votes were encrypted using standard symmetric or asymmetric algorithms (e.g., AES-256 [4], RSA [5]), and system integrity depended on the trustworthiness of election authorities who held the decryption keys. This model suffers from several critical weaknesses: a compromised authority can decrypt and read individual votes, the tallying process is opaque to outside observers, and there is no mechanism to prove that votes were counted correctly without revealing them.

Beyond the tallying problem, traditional systems provide no protection against voter coercion. If a coercer can observe a voter's actions or demand proof of how they voted, the voter has no way to cast a free vote while appearing to comply with the coercer's demands. This vulnerability is particularly acute in remote voting scenarios where the voter is not protected by the secrecy of a polling booth.

1.1.2.2 Cryptographic approaches

Modern cryptographic approaches address these limitations through mathematical guarantees rather than organizational trust. Homomorphic encryption enables vote tallying without decrypting individual ballots. Zero-knowledge proofs allow voters to prove their votes are well-formed without revealing their content. Ring signatures provide anonymity within a group of potential signers while enabling double-vote detection through key images.

However, achieving all security properties simultaneously through cryptographic means requires careful protocol composition. Individual protocols provide specific guarantees, but their combination must be analyzed to ensure that the security properties are preserved under composition. This challenge of secure multi-protocol integration is a central concern of the proposed system design.

1.1.3 Traditional E-Voting vs. Blockchain-Based E-Voting

The key distinction between traditional and blockchain-based e-voting lies in the trust model. Traditional systems require voters to trust a centralized authority, while blockchain-based systems distribute trust across a network of independent nodes.

1.1.3.1 Core differences

In traditional e-voting, a central election authority controls the encryption keys, manages the vote database, and performs the tally. Voters must trust that this authority will not manipulate votes, leak individual ballots, or selectively exclude certain votes from the count. The verification process, if any, is typically limited to auditing by authorized observers who must again be trusted.

Blockchain-based e-voting fundamentally alters this trust model. Votes are recorded on a distributed ledger that is maintained by multiple independent parties. The immutability property of the blockchain ensures that once a vote is recorded, it cannot be modified or deleted without detection. Smart contracts can enforce voting rules automatically, removing the need to trust election officials to follow procedures correctly. Cryptographic proofs attached to each vote enable universal verifiability: any observer can verify that all recorded votes are well-formed and correctly tallied.

1.1.3.2 Advantages of blockchain-based approach

Several advantages of blockchain-based e-voting are worth highlighting. First, the decentralized nature of the blockchain eliminates single points of failure: no single entity can compromise the entire election. Second, the immutable ledger provides an auditable trail that persists indefinitely, enabling post-election verification at any time. Third, smart contracts ensure consistent enforcement of election rules without human intervention. Fourth, blockchain transparency enables end-to-end verifiability, where voters can verify that their votes were recorded and counted correctly.

These properties are particularly relevant in contexts where institutional trust is low, such as elections in developing nations, organizational governance, or scenarios where election manipulation is a concern.

1.1.4 Blockchain Technology in E-Voting

Blockchain platforms used for e-voting fall into two broad categories: public blockchains (e.g., Ethereum [6]) and permissioned blockchains (e.g., Hyperledger Fabric [1]). Public blockchains offer maximum decentralization but suffer from scalability limitations, high transaction costs, and energy consumption concerns. Permissioned blockchains provide controlled access, higher throughput, and configurable privacy, making them more suitable

for election systems where the set of validating nodes is known and controlled by election authorities.

Our framework employs Hyperledger Fabric v2.5 [1], a permissioned blockchain framework maintained by the Linux Foundation. Fabric’s modular architecture supports pluggable consensus mechanisms, channel-based data isolation, and chaincode (smart contracts) written in general-purpose languages such as Go. The Fabric Gateway SDK enables application-level interaction with the blockchain over gRPC channels secured with TLS, providing authenticated communication between the voting application and the distributed ledger. Recent empirical studies [7] have demonstrated that Fabric achieves substantially higher throughput and lower latency than public blockchain alternatives for enterprise applications.

1.1.5 Cryptographic Protocols for E-Voting

The security of the proposed framework rests on a multi-protocol cryptographic stack in which each primitive provides a specific security guarantee:

- **Paillier Homomorphic Encryption** [8]: Enables encrypted vote tallying without decryption. Based on the Decisional Composite Residuosity Assumption (DCRA) [8], Paillier’s additive homomorphism allows the election result to be computed directly from encrypted ballots: $E(a) \times E(b) = E(a + b)$.
- **Pedersen Commitments**: Provide perfectly hiding and computationally binding commitments to vote values. Used as the foundation for zero-knowledge proofs that verify vote validity.
- **Zero-Knowledge Proofs (Σ -Protocol)**: Enable voters to prove that their encrypted votes contain valid values ($w \in \{0, 1\}$) and sum to exactly one ($\sum w = 1$) without revealing which candidate received the vote. The Strong Fiat-Shamir transformation [9] is used to make the proofs non-interactive, with the public parameters, statement, election identifier, and a per-proof nonce bound into the challenge to prevent the malleability attack identified by Bernhard, Pereira, and Warinschi against earlier non-interactive Fiat-Shamir constructions used in voting systems. In the deployed pipeline, both the per-slot binary proofs and the sum-one proof are verified at credential issuance and re-checked at the time of vote casting, so the soundness reduction stated in Chapter 3 as Theorem 2 (Vote Validity) is enforced at every entry point that consumes a credential.
- **Linkable Ring Signatures**: Provide voter anonymity within a ring of 100 members while enabling double-vote detection through deterministic key images. Based on the Liu-Wei-Wong construction [10].

- **SMDC (Self-Masking Deniable Credentials):** A novel coercion-resistance mechanism in which each voter receives $k = 5$ credential slots (1 real + 4 fake). The slot commitments are perfectly hiding under Pedersen, and the real-slot index itself is derived from a server-held HMAC secret k_{srv} that an external observer cannot recompute from public inputs. Under coercion, voters can present any fake slot to produce valid-looking but silently discarded votes. The index is disclosed to the legitimate voter only through a duress-gated reveal endpoint.
- **SA² (Samplable Anonymous Aggregation):** A two-server privacy model adapted from Apple’s federated learning research [11]. Votes are split into masked shares distributed between two non-colluding servers. As long as one server remains honest, individual votes cannot be reconstructed.
- **Kyber-768 Post-Quantum Encryption:** NIST Level 3 post-quantum key encapsulation based on Module-LWE [12], providing protection against future quantum computer attacks through hybrid encryption with Paillier.
- **DJN Threshold Paillier Decryption [13]:** Distributes the Paillier private key across n trustees via Shamir’s (t, n) -secret sharing [14], so that no single entity can unilaterally decrypt election results; any t trustees jointly reconstruct the tally through Lagrange interpolation.

1.1.6 State of the Art in the Modern Context

Recent blockchain e-voting research has produced systems that each strengthen one or two security properties but none that simultaneously achieve all of: homomorphic tallying for exact results, formal zero-knowledge proofs for vote validity, linkable ring signatures for anonymity with double-vote detection, deniable credentials for coercion resistance, two-server anonymous aggregation, post-quantum security, and a Caliper-benchmarked Hyperledger Fabric deployment with empirical performance data. Recurring shortcomings across the surveyed work include super-linear tally complexity, reliance on trusted counting authorities or differential-privacy approximations, weak symmetric-only privacy, and the absence of a coercion-resistance mechanism that survives a coercer who can demand proof of how the voter cast a ballot. A detailed analysis of representative works, their reported performance, and the precise gaps they leave open is presented in Chapter 2. The proposed framework directly addresses these gaps and is positioned in detail against the cited systems in Section 4.8 of Chapter 4.

1.1.7 Importance and Relevance of the Topic

Secure electronic voting matters beyond technical innovation; it touches on democratic principles directly. In many developing nations, voter intimidation and coercion remain

serious obstacles to free elections. International election observers frequently report incidents of vote buying, family voting, and employer pressure on voting choices. A voting system that provides mathematical guarantees of coercion resistance, not just procedural safeguards, would mark a genuine step forward for democratic integrity.

On a related note, the emergence of quantum computing poses a long-term threat to current cryptographic systems [15]. Elections may produce records that must remain confidential for decades. A vote encrypted today with RSA, ECC, or any scheme whose security reduces to integer factorisation or discrete logarithms could potentially be decrypted by a future quantum computer running Shor’s algorithm [15], retroactively compromising voter privacy. This *harvest-now-decrypt-later* threat model has motivated the U.S. National Institute of Standards and Technology to standardise lattice-based replacements, including ML-KEM (FIPS 203 [12]) for key encapsulation and ML-DSA (FIPS 204 [16]) for digital signatures, both finalised in 2024. Post-quantum cryptographic protection is therefore not merely a theoretical concern but a practical and standards-aligned necessity for long-term ballot secrecy.

From a scalability perspective, nations with large populations require voting systems whose per-candidate tally cost grows only linearly with the number of voters N and whose total tally cost grows no worse than linearly with the number of candidates m . The proposed framework achieves $O(N)$ per candidate and $O(N \cdot m)$ total tally complexity, confirmed through empirical benchmarks — a factor- m improvement over ISE-Voting’s $O(N \cdot m^2)$. Feasibility for national-scale deployments is further evidenced by approximately 200 TPS Caliper-verified chaincode throughput and 100% tally integrity across thirteen end-to-end election simulations spanning 100 to 100,000 voters.

The intersection of blockchain technology, advanced cryptography, and post-quantum security in the context of electronic voting represents an active and important area of research with direct societal impact.

1.2 Motivation

The development of the proposed framework is motivated by several gaps observed in the blockchain-based e-voting literature surveyed in Chapter 2. Substantial progress has been made on each individual security property, but the surveyed work leaves room for a more tightly integrated composition.

First, the works surveyed in Chapter 2 typically address only a subset of the required security properties. Schemes built around homomorphic tallying often omit a deployable coercion-resistance mechanism, while the seminal coercion-resistance constructions of JCJ [17] and its instantiation in Civitas [18] introduced the deniable-credential paradigm and were realised as research prototypes. Building on that line of work, the present thesis integrates a JCJ-style deniable credential mechanism (SMDC) into a Hyperledger Fabric permissioned-blockchain pipeline that also incorporates post-quantum hybrid encryption,

two-server anonymous aggregation, and linkable ring signatures — a composition that, to the best of our knowledge and within the works surveyed in Chapter 2, has not been previously reported.

Second, several of the surveyed tallying mechanisms exhibit super-linear scaling in the number of candidates. ISE-Voting’s $O(N \times m^2)$ complexity, for example, requires approximately 10^8 operations for a one-million-voter election with ten candidates, motivating a linear-in-voters and linear-in-candidates tally for national-scale deployments.

Third, many of the systems reviewed in Chapter 2 report theoretical designs or local-only simulations; an end-to-end Hyperledger Fabric integration with empirical throughput and latency measurements on a permissioned platform is reported in only a subset of these works. Bridging this design–deployment gap with reproducible Hyperledger Caliper benchmarks is one of the engineering objectives of this thesis.

Fourth, although standalone post-quantum voting proposals exist in the broader literature, the integration of post-quantum key encapsulation into the same pipeline as coercion-resistant credentials, ring-signature anonymity, and two-server anonymous aggregation is not reported among the works surveyed in Chapter 2. The proposed framework adds a Kyber-768 hybrid encryption layer to address the harvest-now-decrypt-later threat described in Section 1.1.7.

These observations motivate the design and implementation of the proposed framework as an integrated e-voting system that composes a multi-protocol cryptographic stack on real permissioned-blockchain infrastructure, with the cast endpoint wired end-to-end through the Fabric Gateway SDK so that successfully cast votes are published to the deployed chaincode during normal operation.

1.3 Scope and Limitations

This thesis focuses on the cryptographic protocol design, implementation, and performance evaluation of the proposed system. The scope and boundaries of this research are defined as follows:

1.3.1 In Scope

- Design and implementation of a multi-protocol cryptographic stack (Paillier homomorphic encryption [8], Pedersen commitments [19], zero-knowledge Σ -proofs with Strong Fiat-Shamir [9], linkable ring signatures [10], the proposed SMDC deniable credentials, two-server SA^2 aggregation [11], Kyber-768 (CRYSTALS-Kyber round-3 via CIRCL [12]), and Damgård-Jurik-Nielsen threshold Paillier decryption [13]) in Go 1.24.
- Integration with Hyperledger Fabric v2.5 [1] as the underlying permissioned-blockchain infrastructure.

- Empirical performance evaluation including crypto micro-benchmarks, pipeline analysis, scalability testing (100 to 100,000 voters), and Caliper blockchain benchmarks (10,000 to 100,000 transactions).
- Security analysis with formal theorem statements, proof sketches for six security properties, and ProVerif formal verification of three core properties.
- Comparative analysis with five state-of-the-art blockchain e-voting systems.

1.3.2 Out of Scope

The following items fall outside the boundary of the present work: user interface (UI/UX) design and implementation for the voter-facing application, voter registration infrastructure and national identity integration, formal verification using Tamarin or other automated tools beyond the ProVerif symbolic model, multi-constituency deployment with geographic distribution, mobile client implementation and accessibility testing, and real-world election deployment and usability studies with actual voters. These out-of-scope items are discussed as future work directions in Chapter 5.

1.4 Problem Statement

Despite the advances in blockchain-based e-voting, significant challenges remain in building a system that provides comprehensive security guarantees while maintaining practical performance. The core problems addressed by this thesis are as follows:

- How can multiple cryptographic protocols (homomorphic encryption, zero-knowledge proofs, ring signatures, deniable credentials, anonymous aggregation, and post-quantum key encapsulation) be securely composed into a unified voting pipeline that preserves the individual security properties of each protocol under composition?
- How can coercion resistance be practically implemented in a blockchain e-voting context, such that a coerced voter can present plausible fake credentials to a coercer without the coercer being able to distinguish real from fake votes, while ensuring that only real votes are counted in the final tally?
- How can the vote tallying process achieve $O(N)$ computational complexity *per candidate* (linear in the voter count N), with $O(N \cdot m)$ total cost across m candidates, while preserving the exact correctness of results without introducing noise or approximation?
- How can a deployed Hyperledger Fabric blockchain be integrated with the cryptographic voting pipeline to provide immutable vote records and chaincode-level verification, and what throughput and latency characteristics does this integration achieve under realistic load conditions?

1.5 Research Objectives

The primary goal of this research is to design, implement, and empirically evaluate a blockchain-based e-voting system that provides comprehensive cryptographic security guarantees while demonstrating practical performance on a deployed Hyperledger Fabric network. The specific objectives are:

- To design and implement a multi-protocol cryptographic voting pipeline that provides ballot privacy (Paillier homomorphic encryption), vote validity (Pedersen commitments with zero-knowledge Σ -proofs), voter anonymity (linkable ring signatures), coercion resistance (SMDC deniable credentials), privacy-preserving aggregation (SA² two-server model), post-quantum security (Kyber-768 hybrid encryption), and distributed key management (Damgård-Jurik-Nielsen threshold Paillier decryption).
- To develop a novel coercion resistance mechanism (SMDC) based on deniable credentials with $k = 5$ slots, integrated with behavioural duress detection, that provides computationally indistinguishable real and fake credentials in a deployed blockchain voting context.
- To achieve $O(N)$ per-candidate tally complexity (and $O(N \cdot m)$ total) through Paillier’s additive homomorphism, enabling linear-in-voters vote counting that grows only linearly, not quadratically, with the number of candidates, and to validate this complexity through empirical benchmarks from 100 to 100,000 voters.
- To integrate the voting system with a deployed Hyperledger Fabric v2.5 network and empirically measure throughput and latency using Hyperledger Caliper benchmarks across transaction volumes from 10,000 to 100,000.

1.6 Research Contributions

This thesis makes the following contributions to the field of blockchain-based electronic voting. To avoid overstating novelty, the contributions are explicitly grouped into (a) genuinely novel cryptographic constructions, (b) system-integration and engineering contributions, and (c) empirical results.

1. **SMDC Coercion Resistance with Duress-Bound Index Disclosure (Novel Construction).** The thesis introduces Self-Masking Deniable Credentials (SMDC), a practical coercion-resistance mechanism in which each voter receives $k = 5$ credential slots (1 real and 4 fake). The real-slot index is derived via $\text{HMAC-SHA256}(k_{\text{srv}}, \text{"smdc-real-index:"} \parallel \text{voter_id})$ where k_{srv} is a server-held secret kept confidential to the election authority. The index is therefore re-derivable only by the authority and is *never* returned in the credential-issuance response. To learn the real index, a voter must submit the duress

signal registered at issue time to a dedicated reveal endpoint that returns the index only on a constant-time signal match; a registration-time or post-registration coercer who knows only the public pair (vid, eid) therefore obtains nothing exploitable, and a voter under duress can deceive the coercer by presenting any wrong signal. SMDC delivers a complete implementation of deniable credentials integrated with a blockchain e-voting pipeline, addressing the implementation gap left open by the theoretical credential-management schemes of JCJ [17] and Civitas [18] for the works surveyed in Chapter 2.

2. **Behavioural Duress Detection (Novel Construction).** A behavioural coercion-detection mechanism is presented, using HMAC-SHA256 hashing of a voter-chosen behavioural signal (blink count, head tilt, long press, time delay, voice command, etc.) under a server-held key K_d , with constant-time equality verification against a per-voter stored hash h_{stored} that is recorded at registration time. The same signal gates the SMDC reveal-slot-index endpoint (above) and the cast-time coercion short-circuit (below), so a single registered secret simultaneously protects (i) the voter’s ability to retrieve the real index without leaking it to a coercer at registration and (ii) the voter’s ability to silently discard a coerced vote at cast time. The deterministic per-voter equality check, the registration-time signal capture, and the dual-purpose gating are original to this work.
3. **Coercion Short-Circuit Semantics (Novel Construction).** The vote-casting pipeline is extended with silent-substitution semantics gated on the behavioural-duress equality check: when the presented signal fails to match h_{stored} , the behavioural weight is zeroed but the full ring-signature and SA^2 share-split steps still execute (preserving timing indistinguishability), and the per-candidate ciphertexts are scalar-multiplied by zero so that the homomorphic tally counts $E(0)$. Crucially, the key image is *not* marked used in this path, so the voter retains the ability to cast a genuine vote once the coercer is gone. The receipt returned to a coerced voter is observationally identical to a genuine-cast receipt. This short-circuit triggers on the behavioural signal only and is orthogonal to the SMDC slot choice (a genuine vote on a fake SMDC slot still proceeds to chain as a legitimate decoy).
4. **Multi-Protocol Pipeline Integration (Engineering Contribution).** This work presents a unified pipeline that composes the multi-protocol cryptographic stack — Paillier homomorphic encryption [8], Pedersen commitments [19], zero-knowledge Σ -proofs (Strong Fiat-Shamir) [9], linkable ring signatures [10], SA^2 anonymous aggregation [11], Kyber-768 post-quantum hybrid encryption [12], and DJN threshold Paillier decryption [13] — together with the novel SMDC and duress mechanisms above into a single voting pipeline. None of the works surveyed in Chapter 2 reports the same combination of primitives in a single integrated pipeline. The underlying primitives

are existing constructions; the contribution lies in their composition, the resolution of inter-protocol parameter compatibility, and the deployment engineering.

5. **SA² Adaptation for Voting (Integration Choice, not Novel).** Apple’s Samplable Anonymous Aggregation primitive [11], originally designed for federated data analysis, is integrated with Paillier additive homomorphism so that the encrypted tally is reconstructed via mask cancellation across two non-colluding aggregators. The SA² construction itself is not a contribution of this thesis; the contribution is its placement in the voting pipeline.
6. **Hyperledger Fabric Integration (Engineering Contribution).** The vote-casting pipeline is wired end-to-end through the Fabric Gateway SDK (fabric-gateway v1.10.1): every non-coerced vote produced by the API server’s cast endpoint is published to the deployed chaincode via `SubmitTransaction`, and the resulting chain transaction identifier is returned in the voter’s receipt. Independent Hyperledger Caliper benchmarks at the chaincode level report approximately 200 TPS throughput with a 100% success rate at 10 000 transactions; an end-to-end Caliper run that exercises the integrated cast endpoint with a live Fabric peer in the loop is identified as future work.
7. **$O(N)$ -per-Candidate Tally with Empirical Validation (Result).** Through Paillier’s additive homomorphism, the tally achieves $O(N)$ complexity per candidate and $O(N \cdot m)$ total complexity (with N denoting the voter count, distinct from the Paillier modulus n), confirmed empirically: the per-vote per-candidate cost is approximately $6.1 \mu\text{s}$ across four orders of magnitude (100 to 100,000 voters), and the total tally time scales linearly with m (Section 4.4.3.2). This is a factor- m improvement over ISE-Voting’s $O(N \cdot m^2)$, and the 100% exact tally contrasts with BP-Vot’s $\geq 98\%$ differential-privacy approximation.

1.7 Thesis Outline

The remainder of this thesis is organized as follows:

- **Chapter 2, Related Works**, discusses the existing research on blockchain-based e-voting systems with a focus on security properties, cryptographic techniques, and coercion resistance mechanisms. It reviews six key works in detail, analyzing their methodologies, contributions, and limitations. The chapter concludes with a comprehensive comparison table that positions our framework relative to the state of the art.
- **Chapter 3, System Design and Cryptographic Protocols**, presents the complete architecture of the proposed system. This chapter includes the detailed mathematical

formulation of the multi-protocol cryptographic stack, the twenty-step vote casting pipeline, the SMDC credential generation mechanism, the SA² aggregation protocol, and the blockchain integration architecture. Engineering and societal considerations are also addressed.

- **Chapter 4, Implementation and Performance Evaluation**, describes the implementation details including the technology stack (Go 1.24, Gin v1.11, Hyperledger Fabric v2.5.15), test coverage analysis, and comprehensive performance benchmarks. Results include crypto micro-benchmarks, end-to-end pipeline measurements, scalability validation from 100 to 100,000 voters, and Hyperledger Caliper blockchain throughput benchmarks from 10,000 to 100,000 transactions. Comparative analysis with existing systems is presented.
- **Chapter 5, Conclusion and Future Work**, summarizes the key findings of this research, discusses the system's limitations, and outlines directions for future work including formal verification, lattice-based ring signatures, and production deployment considerations.

1.8 Conclusion

This chapter has provided the background, motivation, and problem statement for this research. The fundamental security challenges in electronic voting (ballot privacy, vote integrity, coercion resistance, and post-quantum security) were identified, along with the limitations of existing blockchain-based solutions. The research objectives were defined to address these gaps through a multi-protocol cryptographic stack deployed on a Caliper-benchmarked Hyperledger Fabric network. The next chapter reviews the existing literature in detail, analyzing the methodology, contributions, and limitations of state-of-the-art blockchain e-voting systems.

2. ❖ Related Works

2.1 Introduction

Blockchain-based electronic voting has attracted growing research interest in recent years, driven by the demand for transparent, secure, and verifiable election systems. As democratic processes face increasing scrutiny regarding integrity and trust, researchers have explored various combinations of blockchain technology, cryptographic protocols, and privacy-preserving mechanisms to meet the core security requirements of voting.

This chapter reviews six key works that represent the current state of the art in blockchain-based e-voting. The reviewed papers span multiple security dimensions including privacy preservation through homomorphic encryption and differential privacy, voter authentication through biometric verification, coercion resistance through formal game-based proofs, post-quantum security considerations, and broad architectural surveys. Each work is analyzed in terms of its methodology, contributions, and limitations. The chapter concludes with a comparative analysis that positions our framework relative to existing work and identifies the research gaps that this thesis addresses.

2.2 Review of Selected Works

2.2.1 Improved Secure and Efficient E-Voting on Blockchain

The paper by Zhang et al. [20] investigates the problem of achieving voter anonymity, distributed vote counting, and public verifiability in blockchain-based e-voting systems designed for IoT-assisted environments. The authors propose ISE-Voting, a scheme that combines identity-based ring signatures with secret sharing to eliminate the need for a single trusted counting authority.

2.2.1.1 Methodology

The authors formulate the e-voting problem as a multi-party computation task on the Ethereum blockchain. Their approach employs identity-based ring signatures to provide voter anonymity, where each voter signs their ballot using a ring of eligible voter identities. To address the centralized tallying problem, the authors introduce a Cloud Service Provider (CSP) that assists in distributed vote counting through a ballot cutting algorithm. The

voting protocol operates in three phases: registration (where voters obtain identity-based keys from a trusted authority), voting (where ballots are signed and submitted to the blockchain), and counting (where the CSP and blockchain nodes collaboratively tally the results). The secret sharing mechanism distributes the counting responsibility across multiple parties, preventing any single entity from learning individual votes during the tally.

2.2.1.2 Contributions

The main contributions of this work are threefold. First, the authors design an identity-based ring signature scheme that provides 128-bit security for voter anonymity in a claimed post-quantum setting through symmetric key foundations. Second, they propose a distributed counting mechanism using a CSP and ballot cutting algorithm that reduces the computational pressure on blockchain nodes. Third, the authors provide formal security analysis demonstrating correctness, anonymity, unforgeability, and verifiability of the voting protocol. Experimental evaluation shows that ISE-Voting can perform distributed counting within practical timeframes for medium-scale elections.

2.2.1.3 Limitations

Despite its contributions, ISE-Voting exhibits several notable limitations. The tally complexity of the ballot cutting algorithm is $O(n \times m^2)$, where n is the number of voters and m is the number of candidates. For a national-scale election with millions of voters and dozens of candidates, this quadratic dependency on candidate count renders the approach impractical. In addition, the system relies on a CSP as a semi-trusted third party for counting, which partially undermines the decentralization benefits of blockchain. The scheme provides no coercion resistance mechanism: a coerced voter cannot produce a plausible fake vote. Also, while the authors claim post-quantum security through symmetric key foundations, the identity-based signature scheme itself relies on assumptions that may not withstand quantum attacks. The system also lacks homomorphic encryption, meaning individual ballots must be decrypted for counting, creating a privacy vulnerability during the tally phase.

2.2.2 Blockchain-Based E-Voting with Differential Privacy

The paper by Baniata and Caluna [21] addresses the trade-off between voter privacy and election transparency in blockchain-based systems. The authors propose BP-Vot, which integrates smart contracts, differential privacy, and self-sovereign identities on the Hyperledger Besu platform.

2.2.2.1 Methodology

The authors design a permissioned blockchain architecture on Hyperledger Besu where smart contracts govern the election lifecycle including voter registration, ballot submission, and result announcement. The key innovation is a (k, ϵ) -differential privacy mechanism that protects individual voter preferences while allowing statistical approximation of election results. The privacy mechanism works by “pivoting” a randomly selected candidate and redistributing retrievable votes to other candidates, creating plausible deniability for individual choices. Voter authentication is handled through Web3-based self-sovereign identity (SSI), where voters use verifiable credentials stored in their own digital wallets rather than relying on centralized identity providers. The system is deployed on a cloud-based permissioned blockchain with configurable privacy parameters.

2.2.2.2 Contributions

BP-Vot makes several notable contributions. First, it introduces a novel application of differential privacy to blockchain-based voting, achieving voter privacy without requiring homomorphic encryption or mix-nets. Second, the SSI-based authentication provides a decentralized identity management solution that avoids single points of failure. Third, the system demonstrates practical performance with a 24% improvement in latency compared to state-of-the-art solutions (approximately 1 s/TX versus 1.24 s/TX). The authors provide extensive experimental results testing with 10,000 to 50,000 votes and 2 to 8 candidates, demonstrating at least 98% accuracy in vote approximation across all scenarios. Fourth, the differential privacy mechanism is formally evaluated and proven robust against reconstruction attacks.

2.2.2.3 Limitations

The core limitation of BP-Vot lies in its differential privacy approach. By design, differential privacy adds noise to the election results, meaning the published tally is an approximation rather than an exact count. While the authors report $\geq 98\%$ accuracy, this means that in close elections with narrow margins, the system could potentially report incorrect winners. This trade-off between privacy and accuracy is unacceptable in high-stakes political elections where every vote must be counted exactly. Beyond this, BP-Vot provides no coercion resistance mechanism, no zero-knowledge proofs for vote validity verification, and no post-quantum security considerations. The system also lacks anonymous signatures, meaning voter identity is protected only through the differential privacy layer rather than through cryptographic anonymity guarantees.

2.2.3 Biometric-Authenticated Blockchain Voting

Faruk et al. [22] present a blockchain-based voting system that combines biometric authentication with Hyperledger Fabric to address voter impersonation and fraud in online elections. The system was tested with 100 real participants.

2.2.3.1 Methodology

The authors design an internet-based voting system built on the Hyperledger Fabric blockchain framework. Voter authentication employs dual biometric modalities (fingerprint scanning and facial recognition) to prevent impersonation and duplicate voting. The registration process requires voters to submit biometric samples, which are stored as templates for subsequent verification during the voting phase. The blockchain serves as a decentralized and immutable ledger for storing voting records, ensuring transparency and auditability. Votes are encrypted using standard AES-256 encryption before submission to the blockchain. The system architecture follows a client-server model where the biometric verification occurs on the server side before the vote is recorded on the distributed ledger.

2.2.3.2 Contributions

The primary contribution of this work is the practical demonstration of biometric-authenticated blockchain voting with real participants. In testing with 100 participants, the system achieved an 87% successful registration rate using biometrics and approximately 88% of participants successfully cast votes using voter ID paired with either fingerprint or facial recognition. The dual-factor biometric approach provides stronger identity verification than single-factor methods. The use of Hyperledger Fabric ensures enterprise-grade blockchain infrastructure with permissioned access control. The authors also provide a comprehensive analysis of user acceptance and practical deployment challenges.

2.2.3.3 Limitations

Despite its practical contributions, the system's cryptographic foundation is notably weak. The use of basic AES-256 encryption provides confidentiality but does not support homomorphic tallying, meaning votes must be decrypted by a trusted authority for counting. There are no zero-knowledge proofs to verify vote validity without revealing content, no ring signatures for anonymous ballot submission, no coercion resistance mechanism, and no post-quantum security considerations. The 87% registration success rate suggests significant usability challenges with biometric enrollment. Furthermore, the system lacks formal security proofs and relies primarily on the blockchain's immutability for integrity guarantees. The small test scale (100 participants) provides limited evidence for scalability.

claims. The privacy model depends entirely on AES encryption and Fabric’s channel isolation, offering no mathematical privacy guarantees comparable to homomorphic encryption or differential privacy.

2.2.4 Timed-Release E-Voting with Paillier Homomorphic Encryption

Yuan et al. [23] propose a blockchain-based e-voting scheme that combines Paillier homomorphic encryption with timed-release encryption to control the decryption timeline and eliminate the need for a trusted counting authority.

2.2.4.1 Methodology

The authors integrate two cryptographic primitives: Paillier’s additively homomorphic encryption for privacy-preserving vote tallying, and timed-release encryption (TRE) for automated key release. The TRE mechanism controls when the private key becomes available for decryption through an independent microservice, eliminating human intervention in the tallying process. Ballot legitimacy is ensured through partial knowledge proofs that verify each vote is well-formed without revealing its content. The system operates on a blockchain where encrypted votes are submitted as transactions. The homomorphic property of Paillier encryption allows the tally to be computed by multiplying ciphertexts ($E(v_1) \times E(v_2) = E(v_1 + v_2)$), after which the result is decrypted only when the timed-release condition is met. The scheme aims to prevent premature result disclosure that could influence ongoing voting.

2.2.4.2 Contributions

This work makes two primary contributions. First, the integration of Paillier homomorphic encryption with timed-release encryption provides both ballot privacy and temporal fairness: no party can learn intermediate results before the election ends. Second, the partial knowledge proofs ensure that only legitimate ballots are included in the tally without requiring a trusted authority to inspect individual votes. The authors demonstrate semantic security of the scheme and provide feasibility analysis for practical implementation. The use of an automated microservice for key release reduces the trust assumptions compared to systems that rely on election officials to perform decryption.

2.2.4.3 Limitations

While the scheme provides strong privacy through Paillier encryption, it lacks several critical security properties. There are no ring signatures or comparable mechanisms for voter anonymity beyond encryption. The system does not provide coercion resistance: a voter cannot produce a fake credential or plausible alternative vote under duress. The timed-release mechanism introduces a dependency on an external microservice, which, if

compromised, could enable premature decryption. The system provides no post-quantum security, leaving it vulnerable to future quantum attacks on the Paillier cryptosystem. Furthermore, the scheme does not implement threshold decryption, meaning the decryption key must be held by a single entity (or the TRE service), creating a potential single point of failure for ballot secrecy.

2.2.5 Scalable Coercion-Resistant Blockchain Voting

Yin et al. [24] present a scalable coercion-resistant voting scheme specifically designed for blockchain decision-making, published in IEEE Transactions on Dependable and Secure Computing (TDSC). This work addresses the inherent tension between coercion resistance and scalability in blockchain-based voting systems.

2.2.5.1 Methodology

The authors design a voting protocol that achieves coercion resistance by ensuring the randomness used by the voting client cannot serve as a witness or proof of the actual vote to a coercer. The scheme supports private weighted votes and 1-layer liquid democracy, where voters can delegate their voting power to trusted experts. The protocol achieves $O(n)$ overall complexity (where n is the number of voters) and reduces the ballot size to $\Theta(1)$, independent of the number of candidates, a significant improvement over prior work which required $\Theta(m)$ ballot size. The authors provide formal game-based security proofs demonstrating ballot privacy, verifiability, and coercion resistance under standard cryptographic assumptions.

2.2.5.2 Contributions

The main contribution is the first scalable coercion-resistant blockchain voting scheme with $O(n)$ complexity and constant-size ballots. The prototype evaluation demonstrates that the tally procedure is more than $6\times$ faster than previous state-of-the-art schemes such as VoteAgain for elections with over 50,000 voters. The formal security proofs under game-based definitions provide rigorous guarantees. The support for weighted voting and liquid democracy extends the applicability beyond simple majority elections to more complex governance scenarios.

2.2.5.3 Limitations

Despite achieving scalable coercion resistance, the scheme has several limitations. It does not implement post-quantum cryptographic protection, leaving ballot privacy vulnerable to future quantum adversaries. The system lacks homomorphic encryption for privacy-preserving tallying, biometric voter authentication, and anonymous aggregation mechanisms. There is no integration with a production blockchain platform (such as

Hyperledger Fabric) with deployed chaincode and real network benchmarks. The protocol implements only two cryptographic primitives, compared to the multi-protocol composition proposed in our framework. While the game-based formal proofs are stronger than symbolic-model verification, the system does not provide practical deployment validation or comprehensive test coverage.

2.2.6 Survey: Blockchain for Securing Electronic Voting Systems

Ohize, Alfa, and Ibrahim [25] present a comprehensive survey of blockchain-based electronic voting systems, analyzing architectures, trends, solutions, and challenges across the field. Published in *Cluster Computing* (Springer), this survey provides a systematic overview of the research landscape.

2.2.6.1 Methodology

The authors conduct a systematic literature review of blockchain-based e-voting systems, categorizing existing works along multiple dimensions: blockchain platform (public vs. permissioned), consensus mechanism, cryptographic techniques employed, security properties achieved, and deployment maturity. The survey analyzes both theoretical proposals and implemented systems, identifying common architectural patterns and recurring security trade-offs. The authors evaluate each system against a set of core e-voting requirements including ballot secrecy, voter eligibility verification, universal verifiability, coercion resistance, and receipt-freeness.

2.2.6.2 Contributions

The survey provides a structured taxonomy of blockchain e-voting approaches and identifies several important findings. First, while blockchain provides strong integrity guarantees, most systems still rely on additional cryptographic layers for privacy; blockchain transparency alone is insufficient for ballot secrecy. Second, the survey identifies that coercion resistance and receipt-freeness remain the most challenging properties to achieve in blockchain-based systems, with very few implementations addressing these requirements. Third, scalability remains a persistent concern, with most systems tested only at small scales (hundreds to low thousands of voters). Fourth, the survey notes that post-quantum security is largely ignored in existing implementations, despite being critical for long-term ballot confidentiality.

2.2.6.3 Limitations

As a survey paper, this work does not propose new solutions but rather identifies gaps and challenges. The authors note that the field lacks standardized evaluation frameworks, making direct comparisons between systems difficult. The survey also observes that

most reviewed systems remain at the prototype or proof-of-concept stage, with very few undergoing real-world deployment or large-scale testing. The identified gaps, particularly in coercion resistance, post-quantum security, and scalable cryptographic tallying, directly motivate the design of the proposed framework.

2.3 Background Technology

This section briefly reviews the pre-published cryptographic primitives, protocols, and platforms upon which the proposed framework is constructed. Each subsection identifies the foundational reference, summarises the underlying hardness assumption or mathematical structure, and notes the role the primitive plays in the overall voting pipeline. Detailed protocol-level constructions and parameter choices are deferred to Chapter 3.

2.3.1 Paillier Public-Key Cryptosystem

The Paillier cryptosystem [8] is a probabilistic public-key encryption scheme whose security reduces to the Decisional Composite Residuosity Assumption (DCRA) over the group $\mathbb{Z}_{n^2}^*$, where $n = pq$ is a product of two large primes. Its defining property is *additive homomorphism*: the product of two ciphertexts decrypts to the sum of the underlying plaintexts. This property enables an authority to compute encrypted vote totals without ever observing individual ballots. The scheme has been adopted by a number of e-voting designs because it permits exact tallying, in contrast to approximate techniques such as differential privacy.

2.3.2 Pedersen Commitments

A Pedersen commitment [19] binds a committer to a secret value m via $C = g^m h^r \bmod p$, where g and h are generators of a prime-order subgroup with $\log_g h$ unknown and r is fresh randomness uniformly sampled from $\mathbb{Z}_q$. The construction is *perfectly hiding* (the distribution of C over uniform r is independent of m , so even an unbounded adversary learns nothing about m from C) and *computationally binding* under the discrete logarithm assumption (a collision $g^m h^r = g^{m'} h^{r'}$ with $(m, r) \neq (m', r')$ would yield $\log_g h$). Pedersen commitments serve as the cryptographic substrate for the zero-knowledge proofs used to verify ballot well-formedness and for the deniable credentials that underpin coercion resistance in this work.

2.3.3 Zero-Knowledge Σ -Protocols

A Σ -protocol is a three-move public-coin proof system (commit-challenge-response) for languages defined by a discrete-logarithm relation. The Schnorr proof of knowledge [26] is the canonical instance and provides the building block for the binary and sum proofs employed here. The Fiat-Shamir transformation [27] renders such proofs non-interactive

by replacing the verifier’s challenge with the output of a cryptographic hash function. Bernhard, Pereira, and Warinschi [9] demonstrated that naïve applications of Fiat-Shamir in e-voting systems are vulnerable to malleability attacks; their *Strong Fiat-Shamir* variant, which incorporates the public parameters and statement into the hash input, is therefore used throughout the proposed framework.

2.3.4 Linkable Ring Signatures

Ring signatures, introduced by Rivest, Shamir, and Tauman [28], allow a signer to produce a signature on behalf of an ad-hoc group of public keys without revealing which group member produced it. *Linkable* ring signatures, formalised by Liu, Wei, and Wong [10], additionally publish a deterministic key image $I = H(pk)^{sk}$ that allows two signatures by the same signer to be detected without breaking anonymity. This dual property, anonymity within the ring together with verifiable non-duplication, is precisely what e-voting requires for ballot secrecy and double-vote prevention.

2.3.5 Threshold Cryptography and Secret Sharing

Shamir’s (t, n) -secret sharing scheme [14] distributes a secret among n parties such that any t shares reconstruct it via Lagrange interpolation while any $t - 1$ shares reveal nothing. Damgård, Jurik, and Nielsen [13] extended Paillier’s scheme to a threshold variant in which the decryption key is split across talliers, each contributing a partial decryption together with a non-interactive zero-knowledge proof of correctness. The proposed framework adopts this DJN construction so that no single trustee can unilaterally decrypt election results.

2.3.6 Cryptographic Hash Functions and Message Authentication

The security of the Fiat-Shamir transform, the ring signature key image, and the SMDC index derivation rely on cryptographic hash functions. SHA-256, standardised in NIST FIPS 180-4 [29], and the Keccak-based SHA-3 family in FIPS 202 [30] are used as random oracles in this work. For keyed message authentication, HMAC [31, 32] provides existential unforgeability under the assumption that the underlying compression function is a pseudorandom function. HMAC-SHA256 is used both for SMDC slot index derivation and for the constant-time duress score computation introduced in Chapter 3.

2.3.7 Post-Quantum Key Encapsulation: ML-KEM (Kyber)

Shor’s polynomial-time quantum algorithms for integer factorisation and discrete logarithms [15] undermine the long-term confidentiality of ciphertexts produced by Paillier, RSA, and elliptic-curve schemes. ML-KEM, standardised by NIST as FIPS 203 [12] and

based on the CRYSTALS-Kyber proposal, is a key-encapsulation mechanism whose security reduces to the Module Learning-With-Errors (Module-LWE) problem over algebraic lattices. The Kyber-768 parameter set targets NIST security category 3 (equivalent to AES-192) and is the variant adopted in this work to provide a quantum-resistant binding for vote ciphertexts in transit and at rest.

2.3.8 Authenticated Symmetric Encryption: AES-GCM

For non-homomorphic payloads (audit metadata, transport-level confidentiality), the proposed framework uses AES in Galois/Counter Mode (AES-GCM) as specified in NIST SP 800-38D [33]. AES-GCM provides IND-CCA2 confidentiality and INT-CTXT integrity in a single pass, with hardware-accelerated performance on commodity processors.

2.3.9 Hyperledger Fabric

Hyperledger Fabric [1] is a permissioned distributed ledger platform with a modular architecture that separates transaction execution, ordering, and validation. Smart contracts (*chaincode*) execute in isolated containers; ordering is performed by a pluggable consensus service (Raft in the present deployment); and validation is performed independently by each peer. Recent empirical studies [7] have measured Fabric's throughput on the order of hundreds to thousands of transactions per second, which is several orders of magnitude beyond the throughput attainable on public proof-of-work blockchains. The combination of permissioned membership, configurable endorsement, and high throughput makes Fabric the natural deployment target for an election-grade voting system.

2.3.10 Samplable Anonymous Aggregation

Samplable Anonymous Aggregation (SA²) [11] is a two-server cryptographic primitive originally proposed by Apple for privacy-preserving federated data analysis, building on the secure aggregation construction of Bonawitz et al. [34]. Each client splits its contribution into two additive shares blinded by a fresh mask; one share is delivered to a *leader* server and the complementary share to a *helper* server. Provided the two servers do not collude, neither can reconstruct any individual contribution while their combined output yields the exact aggregate. The proposed framework adapts this primitive from federated learning to encrypted voting, a novel use described in Chapter 3.

2.4 Comparison Among State-of-the-Art Works

Table 2.1 presents a comparative analysis of the reviewed e-voting systems across key security properties and implementation characteristics. This comparison highlights the gaps in existing solutions that The proposed framework addresses through its multi-protocol cryptographic stack.

Table 2.1: Comparison of State-of-the-Art Blockchain E-Voting Approaches

Feature	ISE [20]	BP-Vot [21]	Faruk [22]	Yuan [23]	Yin [24]	Ours
Homomorphic Enc.	✗	✗	✗	✓	✗	✓
Zero-Knowledge	✗	✗	✗	✓ [†]	✗	✓
Ring Signatures	✓	✗	✗	✗	✗	✓
Coercion Resist.	✗	✗	✗	✗	✓	✓
Post-Quantum	✓ [†]	✗	✗	✗	✗	✓
Biometric Auth.	✗	✗	✓	✗	✗	✓ [†]
Anonymous Agg.	✗	✗	✗	✗	✗	✓
Threshold Dec.	✗	✗	✗	✗	✗	✓
Formal Proofs	✗	✗	✗	✗	✓	✓
Exact Tally	✓	✗	✓	✓	✓	✓
Blockchain Deploy	✗	✓	✓	✗	✗	✓
Complexity	$O(nm^2)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Protocol Count	2	2	1	2	2	7
Implementation	✓	✓	✓	✓ [†]	✓ [†]	✓

✓ = full support, ✗ = no support, ✓[†] = partial support, $O(\cdot)$ = tally complexity.

The “Ours” Biometric Authentication entry is marked partial (✓[†]) rather than full (✓). The proposed framework includes a biometric pipeline (internal/biometric) that performs SHA3 hash-equality comparison of fingerprint byte input against a stored per-voter hash, with constant-time verification to prevent timing side channels and a liveness pre-check. However, this construction is a hash-equality stub rather than a full biometric matcher: it does not perform feature extraction, fuzzy minutiae matching, or a distance-metric comparison, and the False Acceptance Rate (FAR), False Rejection Rate (FRR), and enrollment success rate have not been empirically characterised through a user trial. Faruk et al. [22] provide a 100-participant trial reporting an 87% biometric registration success rate, which the present work does not match. The hash-equality stub is suitable for the cryptographic-pipeline integration scope of this thesis but should not be interpreted as a deployment-ready biometric matcher; characterising FAR, FRR, and enrollment success rates with a user study is identified as future work in Chapter 5.

From Table 2.1, several observations can be made. First, no existing system provides all the security properties simultaneously. ISE-Voting offers ring signatures but lacks homomorphic encryption and coercion resistance. BP-Vot provides differential privacy but sacrifices exact tallying. Faruk et al. offer biometric authentication with a 100-participant user trial but with minimal cryptographic depth. Yuan et al. implement Paillier homomorphic encryption but without anonymity or coercion resistance. Yin et al. [24] represent the closest competitor to our framework, achieving both $O(n)$ complexity and coercion resistance with formal game-based proofs; however, their system lacks homomorphic tallying, post-quantum protection, biometric authentication, anonymous aggregation, and production blockchain deployment.

Among the works surveyed in this chapter, the proposed framework is the sole system that integrates all reviewed security properties (homomorphic encryption, zero-knowledge proofs, ring signatures, coercion resistance, post-quantum protection, biometric authentication, anonymous aggregation, and threshold decryption) into a single voting pipeline with empirical blockchain performance benchmarks. The proposed framework composes a **multi-protocol cryptographic stack** that combines substantially more primitives than the at-most-two integrated by any of the surveyed systems, and provides a full implementation with production blockchain deployment and ProVerif formal verification.

2.5 Summary

In this chapter, we reviewed six key works representing the current state of the art in blockchain-based e-voting. We analyzed each work’s methodology, contributions, and limitations with a focus on security properties, cryptographic techniques, and practical deployment.

The review reveals that while notable progress has been made in individual security dimensions (privacy through homomorphic encryption, anonymity through ring signatures, and verifiability through zero-knowledge proofs), no existing system offers a complete security framework that simultaneously addresses all critical requirements. In particular, combining coercion resistance with exact homomorphic tallying and post-quantum security remains an open challenge.

Specifically, the literature analysis identifies the following critical gaps that The proposed framework addresses:

1. **Coercion Resistance Gap:** While Yin et al. [24] achieve coercion resistance with formal proofs, their system lacks homomorphic tallying, post-quantum protection, and real blockchain deployment. Our framework fills this gap through the SMDC deniable credential mechanism integrated with Paillier homomorphic tallying and Kyber768 post-quantum encryption on a production Hyperledger Fabric network.

2. **Post-Quantum Gap:** No existing blockchain e-voting system provides post-quantum security with a standardized algorithm. The proposed framework integrates Kyber768 (ML-KEM, NIST FIPS 203) hybrid encryption.
3. **Protocol Integration Gap:** Existing systems implement at most two cryptographic protocols. The proposed framework integrates a multi-protocol cryptographic stack with substantially broader primitive composition while maintaining $O(n)$ tally complexity and sub-100ms per-vote pipeline time.
4. **Empirical Validation Gap:** Most systems provide only theoretical analysis or small-scale simulations. The proposed framework provides comprehensive empirical benchmarks on production Hyperledger Fabric infrastructure with up to 100,000 transactions.

The comparison table (Table 2.1) quantifies these gaps and demonstrates that The proposed framework addresses all identified limitations through its multi-protocol cryptographic stack. The next chapter presents the detailed system design and mathematical formulation of our framework, including the novel SMDC coercion resistance mechanism and SA² anonymous aggregation protocol.

3. System Design and Cryptographic Protocols

3.1 Introduction

This chapter presents the complete system design of the proposed blockchain-based e-voting framework, deployed on Hyperledger Fabric [1], that integrates a multi-protocol cryptographic stack into a unified voting pipeline. On top of standard cryptographic primitives (Pedersen commitments, HMAC, Σ -protocols, linkable ring signatures, threshold Paillier), the chapter introduces three engineering-level design contributions that, to the best of our knowledge and within the works surveyed in Chapter 2, have not been previously combined in this form: (i) Self-Masking Deniable Credentials (SMDC) with $k = 5$ slots and secret-keyed real-slot indexing for coercion resistance; (ii) a duress-bound reveal-slot-index flow that withholds the legitimate slot index from the voter until a registered behavioural signal is presented, denying the index to both registration-time and post-registration coercers; and (iii) a behavioural-duress short-circuit that silently discards coerced votes while returning a receipt indistinguishable from a genuine cast.

Among the existing primitives in the stack, the Samplable Anonymous Aggregation (SA^2) construction of Talwar et al. [11]—originally designed for federated data analysis—is, to the best of our knowledge and within the works surveyed in Chapter 2, applied here for the first time to an e-voting tally pipeline. The vote-encoding follows the Helios-style [36] 1-hot Paillier construction with Cramer–Damgård–Schoenmakers OR-proofs [37] taken directly on the per-candidate ciphertexts.

The chapter is organised as follows. Section 3.2 describes the four-layer architecture and the framework’s trust assumptions. Section 3.3 defines the mathematical notation. Section 3.4 presents the seven cryptographic protocols of the stack—Paillier homomorphic encryption, Pedersen commitments, the Paillier-direct CDS Σ -protocol, linkable ring signatures, SMDC deniable credentials, SA^2 samplable anonymous aggregation, Kyber768 post-quantum hybrid encryption, and DJN threshold decryption. Section 3.5 describes the twenty-step vote-casting pipeline and its activity/sequence diagrams. Section 3.6 details the Hyperledger Fabric integration. Section 3.7 addresses engineering and societal considerations. Section 3.8 presents a two-tier formal security analysis: Tier 1 game-based reductions to standard assumptions (six theorems with full proofs) and a complementary

subsection on the duress-detection mechanism that ties the registration-time, retrieval-time, and cast-time signal handling to the coercion-resistance argument. Section 3.9 concludes the chapter.

3.2 Proposed Framework and Assumptions

The proposed framework consists of four architectural layers that interact to provide comprehensive voting security, as illustrated in Figure 3.1. The layers are: (1) the Authentication Layer, which handles voter identity verification through biometric and credential-based mechanisms; (2) the Cryptographic Computation Layer, which executes the multi-protocol cryptographic pipeline for each vote (including SMDC deniable-credential construction, Paillier 1-hot encryption, CDS zero-knowledge proofs, linkable ring signing, and the per-vote duress check); (3) the Privacy Layer, which implements the two-server SA² mask-cancellation aggregation across non-colluding aggregators; and (4) the Blockchain Storage Layer, which records the resulting encrypted vote shares on Hyperledger Fabric v2.5.

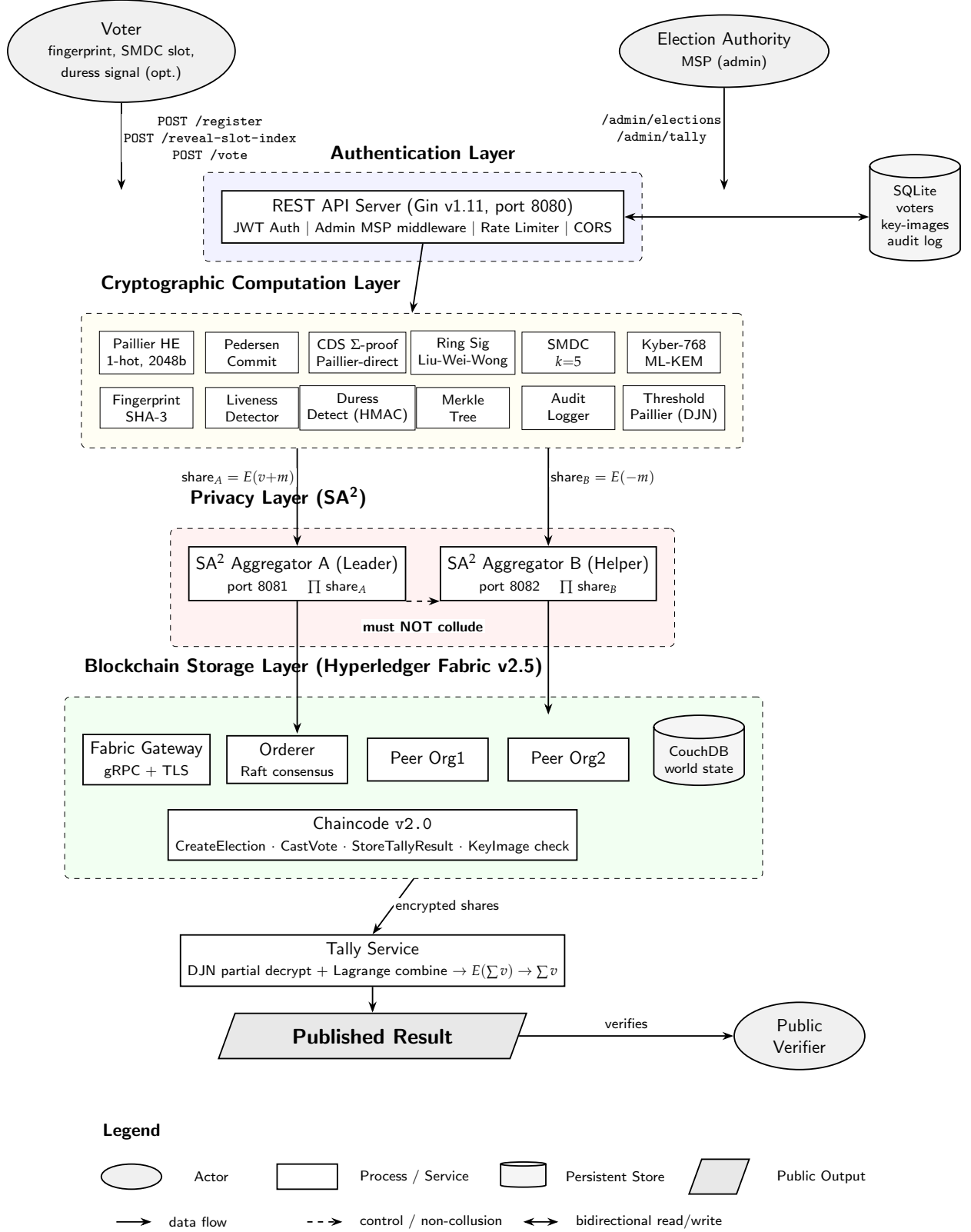


Fig. 3.1: End-to-end system architecture of the proposed framework. The diagram traces the full path from voter authentication (including the duress-bound /reveal-slot-index endpoint that gates the disclosure of the SMDC real-slot index on a registered behavioural signal) through the cryptographic pipeline, the two non-colluding SA² aggregators, the Hyperledger Fabric storage layer, and the threshold tally service to the published result. The four labelled layers are tinted with light pastel fills for visual separation; all shapes and line styles remain distinguishable in greyscale.

The framework operates under the following system-level trust and trust-model assumptions; the cryptographic hardness assumptions (DCRA, DDH, Module-LWE, DLog) that underpin the formal security analysis are listed separately in Section 3.8.1.2.

1. The blockchain network consists of at least one ordering node and two peer nodes running Hyperledger Fabric v2.5 [1] with Raft consensus [35].
2. The two SA^2 aggregation servers are operated by independent, non-colluding entities. The security guarantee holds as long as at least one server remains honest.
3. The threshold Paillier tally service is operated by a committee of $t \geq 3$ trustees holding (t, n) -Shamir shares of the private key; the secrecy of the decrypted partial results, and therefore of the final tally, holds as long as at most $t - 1$ trustees are corrupted.
4. Voters have access to a secure device for biometric authentication and vote submission.
5. Before the election begins, the election authority (i) issues each eligible voter a public SMDC credential consisting of $k = 5$ slot Pedersen commitments together with their binary and sum proofs, and (ii) records the voter's behavioural duress signal as $h_{\text{stored}} = \text{HMAC-SHA256}(K_d, \tau \| v_\sigma)$. The voter's real slot index idx_{real} is *not* returned at credential issue: it is recomputable only by the authority via $\text{HMAC-SHA256}(k_{\text{srv}}, \cdot)$, and is disclosed to the voter through a separate authenticated reveal endpoint that gates the response on a successful duress-signal match (§3.4.5).
6. The two server-held HMAC keys k_{srv} (SMDC real-slot derivation) and K_d (duress signal hashing) are sampled at election setup from at least 256 bits of high-entropy material and remain confidential to the election authority. Disclosure of k_{srv} would let an adversary recompute idx_{real} for every voter from the public pair (vid, eid) and would therefore void the SMDC coercion-resistance argument; disclosure of K_d would let an adversary brute-force the registered signal value v_σ from its hash and bypass the duress short-circuit.
7. The Paillier key pair is generated with a minimum of 2048-bit modulus for computational security.
8. Each voter is assigned $k = 5$ credential slots (1 real, 4 fake) for coercion resistance.

3.3 Notation

Table 3.1 summarizes the mathematical symbols used throughout this chapter.

Table 3.1: Mathematical Notation

Symbol	Meaning
<i>Paillier and Pedersen parameters</i>	
p, q	Large primes for key generation
n	Paillier RSA modulus $n = p \times q$
g	Generator; $g = n + 1$ for Paillier, subgroup generator for Pedersen
h	Independent Pedersen generator with $\log_g h$ unknown
λ	Paillier private key $\lambda = \text{lcm}(p - 1, q - 1)$; also used as the security parameter where context disambiguates
μ	Paillier decryption parameter $\mu = L(g^\lambda \bmod n^2)^{-1} \bmod n$
$L(x)$	Paillier L -function $L(x) = (x - 1)/n$
$E(x)$	Paillier encryption of plaintext x : $E(x) = (1 + n)^x \cdot r^n \bmod n^2$ (generic $x \in \mathbb{Z}_n$; the per-candidate vote indicator is v_j , below)
C	Pedersen commitment $C = g^x \cdot h^r \bmod p$ to plaintext x (used only for SMDC slot weights, §3.4.5)
r	Randomness, context-dependent: (i) Paillier encryption randomness (r generic, r_j per candidate); (ii) Pedersen commitment randomness; (iii) ring-signature responses ($r_0, r_1, \dots$)
<i>Ballot and ZK proofs</i>	
m	Number of candidates in the election
v_j	Per-candidate vote indicator, $v_j \in \{0, 1\}$ with $\sum_{j=1}^m v_j = 1$ (1-hot encoding)
E_j	Per-candidate Paillier ciphertext $E_j = (1 + n)^{v_j} r_j^n \bmod n^2$
π_j^{bin}	CDS OR-proof on E_j that $v_j \in \{0, 1\}$ (§3.4.3.1)
π^{sum}	Schnorr-style proof that $\sum_j v_j = 1$ on $\prod_j E_j$ (§3.4.3.2)
Π	Homomorphic product $\Pi = \prod_{j=1}^m E_j \bmod n^2$
<i>SMDC credentials and duress signal</i>	
k	Number of SMDC credential slots (default $k = 5$, with 1 real and $k - 1$ fake)
k_{srv}	Server-held HMAC key for SMDC real-slot derivation (confidential to election authority)
idx_{real}	Voter's real SMDC slot index, $\text{idx}_{\text{real}} = \text{HMAC-SHA256}(k_{\text{srv}}, \cdot) \bmod k$
vid, eid	Voter identifier and election identifier

Continued on next page

Table 3.1 continued from previous page

Symbol	Meaning
K_d	Server-held HMAC key for the behavioural duress signal (separate from k_{srv})
τ, v_σ	Duress signal type (e.g., <code>blink_count</code>) and signal value supplied at registration or at cast
h_{stored}	Stored duress hash $h_{\text{stored}} = \text{HMAC-SHA256}(K_d, \tau v_\sigma)$ persisted at registration
<i>Ring signature, SA^2, and Kyber</i>	
I	Key image for linkable ring signature: $I = H(\text{pk})^{\text{sk}} \bmod p$
σ	Ring signature tuple $(I, c_0, r_0, r_1, \dots, r_{N-1})$ over a ring of size N
pk_i, sk_i	Public/private key pair for ring member i
$\mathcal{R}$	Ring of public keys with $ \mathcal{R} = 100$
$S_{A,j}, S_{B,j}$	SA^2 per-candidate vote shares for Server A and Server B
mask_j	Random masking value for the SA^2 split of E_j
ct_{ky}, K	Kyber-768 KEM ciphertext and encapsulated symmetric key
MAC	HMAC-SHA256 outer integrity tag over the Paillier ciphertext using a key derived from K

3.4 Cryptographic Protocol Formulation

This section presents the mathematical formulation of the multi-protocol cryptographic stack implemented in our framework. Each protocol provides a specific security guarantee, and their composition forms the complete voting pipeline.

3.4.1 Paillier Homomorphic Encryption

The Paillier cryptosystem [8] is an additively homomorphic public-key encryption scheme based on the Decisional Composite Residuosity Assumption (DCRA). It enables the computation of vote tallies directly on encrypted ballots without decryption.

3.4.1.1 Key Generation

The key generation process proceeds as in the original construction of Paillier [8]:

1. Select two large primes p and q of equal length (≥ 1024 bits each), ensuring $p \neq q$.
2. Compute the RSA modulus:

$$n = p \times q \tag{3.1}$$

3. Compute Carmichael's function:

$$\lambda = \text{lcm}(p-1, q-1) \tag{3.2}$$

4. Set the generator $g = n + 1$ (standard choice that simplifies computation).
5. Compute the decryption parameter:

$$\mu = L(g^\lambda \bmod n^2)^{-1} \bmod n \quad (3.3)$$

where $L(x) = \frac{x-1}{n}$.

The public key is (n, g, n^2) and the private key is (λ, μ, p, q) .

3.4.1.2 Encryption

Encryption of a plaintext message $x \in \mathbb{Z}_n$ proceeds as defined by Paillier [8]:

1. Select a random $r \in \mathbb{Z}_n^*$.
2. Compute the ciphertext:

$$c = g^x \cdot r^n \bmod n^2 \quad (3.4)$$

3.4.1.3 Decryption

Decryption follows the inverse construction of Paillier [8]:

$$x = L(c^\lambda \bmod n^2) \cdot \mu \bmod n \quad (3.5)$$

3.4.1.4 Homomorphic Properties

The Paillier cryptosystem supports the following homomorphic operations [8]:

Additive Homomorphism. The product of two ciphertexts decrypts to the sum of their plaintexts:

$$E(x_1) \cdot E(x_2) \bmod n^2 = E(x_1 + x_2) \quad (3.6)$$

This property is the foundation of the proposed framework's $O(N)$ -per-candidate (and $O(N \cdot m)$ total over m candidates) tally, where N denotes the voter count, distinct from the Paillier modulus n . Given N encrypted votes $\{E(v_1), E(v_2), \dots, E(v_N)\}$ for a single candidate, the encrypted tally is:

$$E\left(\sum_{i=1}^N v_i\right) = \prod_{i=1}^N E(v_i) \bmod n^2 \quad (3.7)$$

Scalar Multiplication. A ciphertext raised to a scalar k yields the encryption of $k \times x$:

$$E(x)^k \bmod n^2 = E(k \cdot x) \quad (3.8)$$

3.4.2 Pedersen Commitments

Pedersen commitments [19] provide perfectly hiding and computationally binding commitments. In the present framework they are used exclusively to commit to the SMDC slot weights (Section 3.4.5); the per-candidate vote-validity zero-knowledge proofs run directly on Paillier ciphertexts (Section 3.4.3) and do not rely on Pedersen commitments.

3.4.2.1 Parameter Generation

1. Generate a safe prime $p = 2q + 1$ where q is also prime.
2. Find a generator g of the order- q subgroup of $\mathbb{Z}_p^*$: select a random auxiliary element $a \in \mathbb{Z}_p^*$ and compute $g = a^{(p-1)/q} \bmod p$; resample a until $g \neq 1$.
3. Derive an independent generator h such that $\log_g h$ is unknown, using a hash-to-group construction: $h = (\text{OS2IP}(\text{SHA3}(g)))^{(p-1)/q} \bmod p$, where OS2IP is the standard octet-string-to-integer primitive that interprets the SHA-3 digest as a big-endian integer in $\mathbb{Z}_p^*$.

3.4.2.2 Commitment

A commitment to a value $x \in \mathbb{Z}_q$ with randomness $r \in \mathbb{Z}_q$ takes the form defined by Pedersen [19]:

$$C = g^x \cdot h^r \bmod p \quad (3.9)$$

Security Properties:

- **Perfectly Hiding:** For uniformly random $r \xleftarrow{\$} \mathbb{Z}_q$, the value h^r is uniformly distributed over the order- q subgroup, so $C = g^x \cdot h^r$ is uniform on the subgroup independently of x . The two distributions corresponding to any pair of plaintexts $x \neq x'$ are therefore identical, and an unbounded adversary learns no information about x from C .
- **Computationally Binding:** Under the Discrete Logarithm assumption in the order- q subgroup, no PPT adversary can produce a collision $(x, r) \neq (x', r')$ with $x \neq x'$ satisfying $g^x \cdot h^r = g^{x'} \cdot h^{r'}$, since such a collision would yield $\log_g h = (r - r') \cdot (x' - x)^{-1} \bmod q$. (A collision with $x = x'$ but $r \neq r'$ is excluded by $g^x \cdot h^r = g^x \cdot h^{r'}$ implying $h^{r-r'} = 1$, i.e. $r \equiv r' \pmod q$.)

Homomorphic Property [19]:

$$C_1 \cdot C_2 = g^{x_1+x_2} \cdot h^{r_1+r_2} \bmod p = \text{Commit}(x_1 + x_2, r_1 + r_2) \quad (3.10)$$

This homomorphism is what enables the SMDC sum proof (Section 3.4.5.1) to demonstrate $\sum_{i=0}^{k-1} w_i = 1$ across the k slot commitments: the product $\prod_i C_i$ equals $\text{Commit}(\sum_i w_i, \sum_i r_i)$, so a proof of $\sum_i w_i = 1$ reduces to a Schnorr proof of knowledge of $\sum_i r_i$ in $\prod_i C_i / g$.

3.4.3 Zero-Knowledge Proofs (Σ -Protocol)

The framework implements two non-interactive zero-knowledge proofs that operate *directly* on the per-candidate Paillier ciphertexts. This Helios-style construction [36] avoids the auxiliary commitment-equality proof that a parallel Paillier–Pedersen design would require, while preserving the properties needed by Theorem 3.8.1.4 (vote validity) and Theorem 3.8.1.3 (ballot indistinguishability). Both proofs follow the disjunctive composition of Cramer, Damgård, and Schoenmakers [37] on top of Schnorr’s proof of knowledge [26], rendered non-interactive by the Strong Fiat–Shamir transform [27, 9] that incorporates the public parameters and the full statement into the hash input to prevent malleability. Two proofs are required per ballot: a *per-candidate* binary proof that each ciphertext E_j encrypts $v_j \in \{0, 1\}$, and a single sum proof that the candidate ciphertexts jointly encrypt values whose sum is exactly 1.

3.4.3.1 Binary Proof on Paillier Ciphertexts

For a per-candidate Paillier ciphertext $E_j = (1 + n)^{v_j} \cdot r_j^n \bmod n^2$, the binary proof demonstrates that $v_j \in \{0, 1\}$ without revealing which. Note that

$$E_j = r_j^n \bmod n^2 \quad \text{when } v_j = 0, \quad E_j = (1 + n) \cdot r_j^n \bmod n^2 \quad \text{when } v_j = 1.$$

The voter therefore proves knowledge of $r_j \in \mathbb{Z}_n^*$ such that either E_j is an n -th residue (case $v_j = 0$) or $E_j/(1 + n)$ is an n -th residue (case $v_j = 1$). The two branches are combined by the CDS OR-composition [37], instantiated on the Paillier n -th-residuosity language [38, 8].

Proof Generation (case $v_j = 0$; the $v_j = 1$ case is symmetric):

1. **Simulate** the $v_j = 1$ branch: choose random $d_1 \in \mathbb{Z}_n$ and random $s_1 \in \mathbb{Z}_n^*$, and compute the simulated commitment

$$a_1 = s_1^n \cdot \left(\frac{E_j}{1 + n} \right)^{-d_1} \bmod n^2. \quad (3.11)$$

2. **Real proof** for $v_j = 0$: choose random $u_0 \in \mathbb{Z}_n^*$ and compute

$$a_0 = u_0^n \bmod n^2. \quad (3.12)$$

3. Compute the Strong Fiat–Shamir challenge [27, 9] using a SHA3-256 random oracle [30] with an explicit domain-separation tag $\text{DS}_{\text{bin}} = \text{“CovertVote-PaillierZKP-Binary-v1”}$:

$$c = H(\text{DS}_{\text{bin}} \parallel n \parallel \text{nonce} \parallel \text{electionID} \parallel j \parallel E_j \parallel a_0 \parallel a_1) \bmod n. \quad (3.13)$$

4. Set $d_0 = c - d_1 \bmod n$ and $s_0 = u_0 \cdot r_j^{d_0} \bmod n$.

The proof is $\pi_j^{\text{bin}} = (a_0, a_1, d_0, d_1, s_0, s_1, \text{nonce})$.

Verification:

1. Recompute c via (3.13) and check $d_0 + d_1 \equiv c \pmod{n}$.
2. Check $s_0^n \equiv a_0 \cdot E_j^{d_0} \pmod{n^2}$.
3. Check $s_1^n \equiv a_1 \cdot \left(\frac{E_j}{1+n} \right)^{d_1} \pmod{n^2}$, where $(1+n)^{-1} \pmod{n^2}$ is the modular inverse used in the division and equals $n^2 - n + 1 \pmod{n^2}$ since $(1+n)(1-n) \equiv 1 \pmod{n^2}$.

The protocol is non-interactive zero-knowledge in the random-oracle model and has computational special soundness; under the random-oracle modelling of SHA3-256 the soundness error is negligible in $|n|$ [37, 38].

3.4.3.2 Sum-to-One Proof on Paillier Ciphertexts

The sum proof demonstrates that the m per-candidate ciphertexts $E_1, \dots, E_m$ jointly encrypt values whose sum is exactly 1, ensuring each voter selects exactly one candidate. By Paillier's additive homomorphism [8],

$$\Pi = \prod_{j=1}^m E_j = (1+n)^{\sum_j v_j} \cdot \left(\prod_j r_j \right)^n \pmod{n^2}. \quad (3.14)$$

If $\sum_j v_j = 1$, then $\Pi/(1+n) = R^n \pmod{n^2}$ where $R = \prod_j r_j \pmod{n}$. The voter therefore proves knowledge of $R \in \mathbb{Z}_n^*$ such that $\Pi/(1+n)$ is an n -th residue, via a Schnorr-style proof of n -th-residue knowledge [26, 38]:

1. Choose random $u \in \mathbb{Z}_n^*$ and compute commitment $a = u^n \pmod{n^2}$.
2. Compute the Strong Fiat-Shamir challenge (SHA3-256 [30], domain tag $\text{DS}_{\text{sum}} = \text{"CovertVote-PaillierZKP-Sum-v1"}$):

$$c = H(\text{DS}_{\text{sum}} \parallel n \parallel \text{nonce} \parallel \text{electionID} \parallel E_1 \parallel \dots \parallel E_m \parallel a) \pmod{n}. \quad (3.15)$$

3. Compute response $s = u \cdot R^c \pmod{n}$.

The proof is $\pi^{\text{sum}} = (a, c, s, \text{nonce})$.

Verification: Recompute c via (3.15) and $\Pi = \prod_j E_j \pmod{n^2}$, then check

$$s^n \equiv a \cdot \left(\frac{\Pi}{1+n} \right)^c \pmod{n^2}.$$

3.4.3.3 Relationship to the SMDC Slot Proofs

Two parallel proof systems. The Pedersen-based binary and sum proofs used by SMDC (Section 3.4.5) are conceptually distinct from the Paillier-based proofs above. The Pedersen proofs bind the SMDC *slot weights*—a k -vector of commitments with one slot weighted 1 and the remaining $k - 1$ slots weighted 0—to enable coercion-resistant deniable credentials. The Paillier proofs of this section bind the *per-candidate* encrypted votes to enforce ballot validity. The two proof systems coexist independently in a single ballot, address disjoint concerns, and provide independent security guarantees; in particular, the SMDC slot proofs are generated once at credential-issue time and reused across the voter’s lifetime in the election, whereas the Paillier proofs are generated fresh at each vote cast. To distinguish them notationally, this thesis uses π_j^{bin} (subscript j , per candidate) for the Paillier proofs and π_i^{bin} (subscript i , per SMDC slot) for the Pedersen proofs.

Pedersen-instantiated binary and sum proofs. The SMDC slot proofs follow the same CDS OR-composition [37] on top of Schnorr’s proof of knowledge [26], but instantiated on the Pedersen language $\{C = g^w h^r : w \in \{0, 1\}\}$ rather than the Paillier n -th-residuosity language. Concretely, for the slot commitment $C_i = g^{w_i} h^{r_i}$:

- **Binary proof:** an OR-composition of two Schnorr proofs of $\log_h(\cdot)$, one for the branch $w_i = 0$ (i.e. $C_i = h^{r_i}$) and one for the branch $w_i = 1$ (i.e. $C_i/g = h^{r_i}$). The construction is identical in structure to Section 3.4.3.1 but with h , C_i , and C_i/g replacing r_j^n , E_j , and $E_j/(1+n)$.
- **Sum-to-one proof:** a Schnorr proof of knowledge of $R = \sum_i r_i \bmod q$ such that $(\prod_i C_i)/g = h^R$, witnessing $\sum_i w_i = 1$. The construction parallels Section 3.4.3.2 with the Pedersen homomorphism (Equation 3.10) replacing Paillier’s.

Both Pedersen proofs use Strong Fiat–Shamir with SHA3-256 and the domain-separation tags $\text{DS}_{\text{slot-bin}}$ and $\text{DS}_{\text{slot-sum}}$ respectively; soundness reduces to the discrete-logarithm assumption in the order- q subgroup via the standard CDS/Schnorr forking argument.

3.4.4 Linkable Ring Signatures

The framework employs linkable ring signatures, a refinement [10] of the original spontaneous-anonymous-group signature of Rivest, Shamir, and Tauman [28]. A fixed ring size of $|\mathcal{R}| = 100$ members is used. The hash function $H(\cdot)$ used throughout this subsection (for the key image, the hash-to-point operation, and the Fiat–Shamir-style challenge chain) is instantiated with SHA3-256 [30].

Group parameters. The ring signature operates in a dedicated order- q subgroup of $\mathbb{Z}_p^*$ generated independently of the Pedersen group of Section 3.4.2: the implementation calls `crypto.GenerateRingParams` to produce a fresh safe-prime modulus p , prime order q , and subgroup generator g (see `internal/crypto/ring_signature.go`). The hash-to-point function $H_{\text{grp}} : \mathbb{Z}_p^* \rightarrow \mathbb{G}_q$ used by the key image (Equation 3.17) maps a public key into the same order- q subgroup via $H_{\text{grp}}(\text{pk}) = (\text{OS2IP}(\text{SHA3}(\text{pk})))^{(p-1)/q} \bmod p$, in the same style as the Pedersen hash-to-group of Section 3.4.2.

3.4.4.1 Key Generation

Each voter generates a key pair following the Liu-Wei-Wong construction [10]:

$$\text{sk} \xleftarrow{\$} \mathbb{Z}_q, \quad \text{pk} = g^{\text{sk}} \bmod p \quad (3.16)$$

3.4.4.2 Signing

Given message M , signer key $(\text{sk}_s, \text{pk}_s)$, ring $\mathcal{R} = \{\text{pk}_0, \dots, \text{pk}_{|\mathcal{R}|-1}\}$, and signer index s , the signing procedure follows [10]. All index arithmetic below is taken modulo $|\mathcal{R}|$ (i.e. $c_{|\mathcal{R}|}$ wraps to c_0).

1. Compute the key image (for double-vote detection), where H_{grp} is the hash-to-point function introduced above:

$$I = H_{\text{grp}}(\text{pk}_s)^{\text{sk}_s} \bmod p \quad (3.17)$$

2. Choose random $\alpha \in \mathbb{Z}_q$ and compute:

$$L_s = g^\alpha \bmod p, \quad R_s = H_{\text{grp}}(\text{pk}_s)^\alpha \bmod p \quad (3.18)$$

3. Set $c_{s+1} = H(M \| L_s \| R_s)$.
4. For each non-signer $i \neq s$ (in cyclic order $s+1, s+2, \dots, s-1 \bmod |\mathcal{R}|$), choose random $r_i \in \mathbb{Z}_q$ and compute:

$$L_i = g^{r_i} \cdot \text{pk}_i^{c_i} \bmod p, \quad R_i = H_{\text{grp}}(\text{pk}_i)^{r_i} \cdot I^{c_i} \bmod p \quad (3.19)$$

$$c_{i+1} = H(M \| L_i \| R_i) \quad (3.20)$$

5. Close the ring: $r_s = \alpha - c_s \cdot \text{sk}_s \bmod q$.

The signature is $\sigma = (I, c_0, r_0, r_1, \dots, r_{|\mathcal{R}|-1})$.

3.4.4.3 Verification and Linkability

Verification recomputes each (L_i, R_i) pair from the public ring elements and the signature components, and checks that the challenge chain closes back to c_0 . Concretely, given $\sigma = (I, c_0, r_0, \dots, r_{|\mathcal{R}|-1})$ and the public ring $\mathcal{R}$, set $c'_0 = c_0$ and for $i = 0, 1, \dots, |\mathcal{R}| - 1$ compute

$$L'_i = g^{r_i} \cdot \text{pk}_i^{c'_i} \bmod p, \quad R'_i = H_{\text{grp}}(\text{pk}_i)^{r_i} \cdot I^{c'_i} \bmod p, \quad c'_{i+1} = H(M \| L'_i \| R'_i). \quad (3.21)$$

The signature is accepted iff $c'_{|\mathcal{R}|} \equiv c_0$, i.e. the recomputed challenge chain closes onto its starting value. Linkability is achieved through the key image: two signatures from the same signer produce identical I values regardless of message or ring composition, so double voting is detected by the equality test $I_1 \stackrel{?}{=} I_2$ across previously recorded key images.

3.4.5 SMDC: Self-Masking Deniable Credentials

The Self-Masking Deniable Credentials (SMDC) construction presented in this subsection is one of the three engineering-level design contributions of this thesis (Section 3.1). Its building blocks—Pedersen commitments, Σ -proofs, and HMAC—are standard; the composition into a deniable-credential mechanism with k -slot self-masking, secret-keyed HMAC-derived real-slot indexing, and silent-discard semantics for coerced votes is, to the best of our knowledge and within the works surveyed in Chapter 2, original to this work.

Each voter receives $k = 5$ credential slots, of which exactly one is real (weight = 1) and the remaining four are fake (weight = 0). The real slot index is derived deterministically via HMAC-SHA256 [31, 32] keyed by a server-held secret $k_{\text{srv}} \in \{0, 1\}^{\geq 256}$ that is sampled at election setup and held confidentially by the election authority. The index is never stored on chain; the legitimate authority can recompute it for any voter, while a coercer who knows only the public pair (vid, eid) cannot, since HMAC-SHA256 is widely modelled as a PRF under its key.

Duress-bound index disclosure. At registration the voter chooses a private behavioral duress signal (Section 3.4.9), which is stored as an HMAC under a separate server key in the same transaction as credential issuance. The registration response does *not* include the real slot index. To learn it, the voter must subsequently submit the registered signal to a dedicated reveal-slot-index endpoint; the server verifies the signal in constant time and, only on match, returns idx_{real} . This binding has two consequences: (i) a coercer present at registration sees neither the index in the response nor a value derivable from public inputs, and (ii) a coercer who later forces the voter to retrieve the index can be deceived by the voter presenting a wrong signal, in which case the server returns a uniform error and discloses nothing. The election authority retains the policy choice over how often and under what authentication the reveal endpoint may be called.

3.4.5.1 Credential Generation

For voter with identifier vid in election eid :

1. **Derive real index** (deterministic, recomputable by the election authority only):

$$\text{idx}_{\text{real}} = \text{HMAC-SHA256}(k_{\text{srv}}, \text{"smdc-real-index:"} \parallel \text{vid} \parallel \text{eid}) \bmod k \quad (3.22)$$

where k_{srv} is the server-held SMDC HMAC key. Confidentiality of k_{srv} is what reduces the coercer's view to a uniform guess over the k slots; its disclosure would let an adversary recompute idx_{real} for every voter from public inputs and would therefore void the coercion-resistance argument of §3.4.5.2.

2. **Create k slots**: For each slot $i \in \{0, \dots, k-1\}$:

$$w_i = \begin{cases} 1 & \text{if } i = \text{idx}_{\text{real}} \\ 0 & \text{otherwise} \end{cases} \quad (3.23)$$

3. **Commit**: Generate a Pedersen commitment [19] for each slot:

$$C_i = g^{w_i} \cdot h^{r_i} \bmod p, \quad r_i \xleftarrow{\$} \mathbb{Z}_q \quad (3.24)$$

4. **Prove binary**: For each C_i , generate a Pedersen-instantiated CDS OR-proof π_i^{bin} that $w_i \in \{0, 1\}$ (construction sketch in Section 3.4.3.3).
5. **Prove sum**: Generate a Schnorr-style proof π^{sum} that $\sum_{i=0}^{k-1} w_i = 1$, by proving knowledge of $\sum_i r_i \bmod q$ such that $\prod_i C_i / g = h^{\sum_i r_i}$ (construction sketch in Section 3.4.3.3).

The credential is $\text{Cred} = (\text{vid}, k, \{(C_i, \pi_i^{\text{bin}})\}_{i=0}^{k-1}, \pi^{\text{sum}})$.

3.4.5.2 Coercion Resistance Property

Under coercion, the voter presents a *fake* slot $j \neq \text{idx}_{\text{real}}$. The coercer observes:

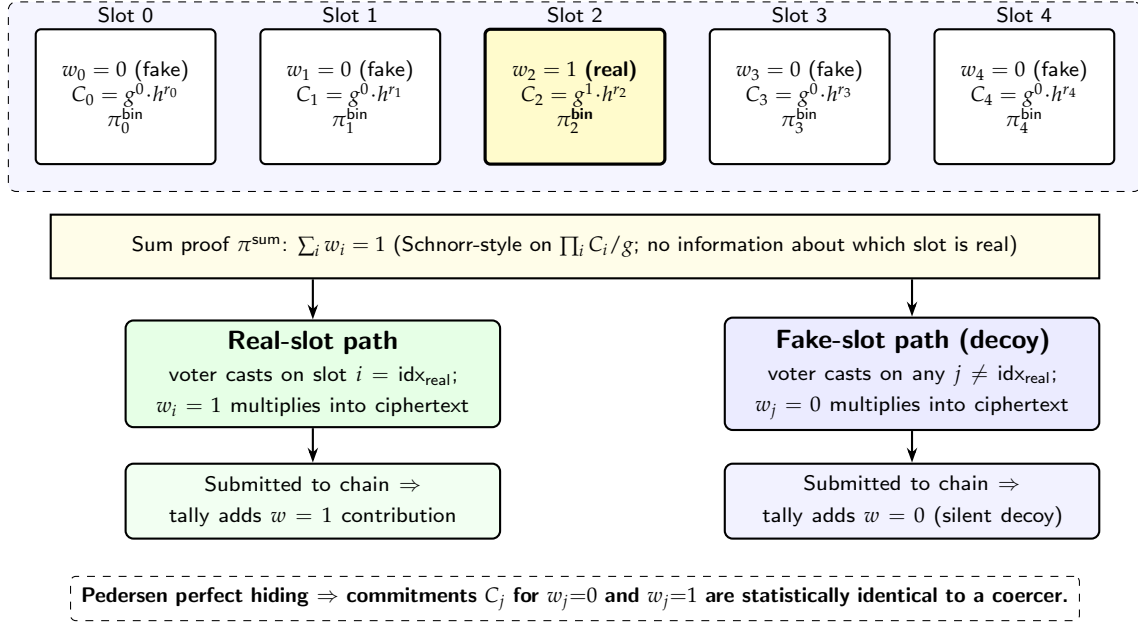
- A valid Pedersen commitment $C_j = g^0 \cdot h^{r_j} = h^{r_j}$
- A valid binary proof π_j^{bin} (proving $w_j \in \{0, 1\}$, without revealing $w_j = 0$)
- A valid sum proof π^{sum} (proving exactly one slot has $w = 1$)

Due to Pedersen's **perfect hiding** property, the distributions of C_j for $w_j = 0$ and $w_j = 1$ are statistically identical (uniform over the order- q subgroup); the coercer therefore learns no information about w_j from C_j , even with unbounded computational power. When a vote is cast on a fake slot, the SMDC slot weight $w_j = 0$ is multiplied into the per-candidate ciphertext at cast time (Section 3.5), so the homomorphic per-candidate tally at decryption time receives zero contribution from that voter; this happens silently—the voter's receipt is structurally identical to that of a genuine cast. Figure 3.2 illustrates this mechanism.

SMDC Deniable Credential ($k = 5$ slots)

Real-slot index: $\text{idx}_{\text{real}} = \text{HMAC-SHA256}(k_{\text{srv}}, \text{"smdc-real-index:"} \parallel \text{vid} \parallel \text{eid}) \bmod k$
 (k_{srv} is server-held; voters learn idx_{real} only via the duress-bound reveal endpoint)

Credential layer (Pedersen commitments + binary proofs)



The real-slot index is *never stored on chain*; only the election authority (holding k_{srv}) can re-derive it.

Voters learn their own idx_{real} via the duress-bound /reveal-slot-index endpoint, gated on a registered behavioural-signal match.

Fig. 3.2: SMDC deniable credential structure with $k = 5$ slots. The real slot is distinguishable only via the cross-hatched fill in this diagram; cryptographically, the Pedersen perfect-hiding property renders the slot commitments statistically (information-theoretically) indistinguishable to a coercer.

3.4.5.3 Credential Verification

The election system verifies credentials without learning the real index:

1. Verify all k binary proofs: $\forall i : \text{VerifyBinary}(C_i, \pi_i^{\text{bin}}) = \text{true}$
2. Verify the sum proof: $\text{VerifySumOne}(\{C_i\}, \pi^{\text{sum}}) = \text{true}$

3.4.6 SA²: Samplable Anonymous Aggregation

This subsection describes the integration of the existing SA² primitive of Apple Machine Learning Research [11], which itself builds on the secure aggregation construction of Bonawitz et al. [34], into the proposed voting pipeline. *The two-server, non-colluding mask-cancellation construction of SA² is not a contribution of this thesis; it is invoked as an existing primitive.* The integration choice made here is to pair the per-candidate Paillier ciphertexts $\{E_j\}_{j=1}^m$ produced by the 1-hot encoding of Section 3.4.3 with an independent SA² mask-cancellation per candidate, so that the encrypted tally is reconstructed via mask homomorphism rather than via the plaintext secret sharing of the original federated-learning setting. The per-candidate parallel structure is reflected in the cast pipeline

(Algorithm 2, Step 14) and in the `sa2.VoteShare` type, whose `SharesA` and `SharesB` fields are length- m vectors.

3.4.6.1 Per-candidate Vote Splitting

Given the per-candidate weighted ciphertext vector $\{E_{w,j}\}_{j=1}^m$ for voter u (where $E_{w,j}$ denotes the post-SMDC-weight Paillier ciphertext from Algorithm 2), the splitting procedure operates independently on each candidate position $j \in \{1, \dots, m\}$:

1. Generate a fresh random mask: $\text{mask}_j \xleftarrow{\$} \mathbb{Z}_{\lfloor n/100 \rfloor}$. The conservative bound $\lfloor n/100 \rfloor$ ensures that, summed over the typical batch of ~ 100 concurrent voters, the cumulative mask remains below n and therefore cannot overflow the Paillier message space. (Information-theoretic hiding of v_j would require uniform $\text{mask}_j \in \mathbb{Z}_n$; here the primary confidentiality guarantee is the computational hardness of Paillier IND-CPA under DCRA, with the mask providing additional non-collusion-based protection.)
2. Compute Share A (for Server A):

$$S_{A,j} = E_{w,j} \cdot E(\text{mask}_j) = E(v_j + \text{mask}_j) \bmod n^2 \quad (3.25)$$

3. Compute Share B (for Server B):

$$S_{B,j} = E(-\text{mask}_j) = E(n - \text{mask}_j) \bmod n^2 \quad (3.26)$$

3.4.6.2 Per-candidate Reconstruction

For each candidate j , Paillier additive homomorphism [8] cancels the mask:

$$S_{A,j} \cdot S_{B,j} = E(v_j + \text{mask}_j) \cdot E(-\text{mask}_j) = E(v_j) \bmod n^2 \quad (3.27)$$

Aggregation across all voters is performed independently per candidate column; tallying N voters for m candidates therefore reduces to m parallel applications of the homomorphic Paillier sum.

Security guarantee. The SA^2 layer adds a *non-collusion* guarantee on top of the underlying Paillier semantic security: even if one of the two aggregation servers is malicious, no individual per-candidate value v_j can be reconstructed from that server's view alone. Server A sees only $\{E(v_j + \text{mask}_j)\}_j$ and Server B sees only $\{E(-\text{mask}_j)\}_j$; reconstructing $E(v_j)$ requires combining the two views, which is impossible without collusion. The underlying Paillier ciphertexts already hide v_j computationally under DCRA; the SA^2 mask thus protects against the additional threat in which an honest-but-curious aggregator separately holds the Paillier decryption key, which Theorem 3.8.1.7 formalises.

3.4.7 Kyber768 Post-Quantum Hybrid Encryption

To protect against the *harvest-now-decrypt-later* threat in which an adversary stores ciphertexts today and decrypts them once a sufficiently large quantum computer becomes available, the framework employs ML-KEM-768 (CRYSTALS-Kyber, NIST FIPS 203 [12]) to derive a post-quantum integrity key for each vote.

3.4.7.1 Hardness Assumption and Parameter Selection

ML-KEM is a key encapsulation mechanism whose IND-CCA2 security reduces to the Module Learning-With-Errors (Module-LWE) and Module Short-Integer-Solution (Module-SIS) problems over polynomial rings $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ with $n = 256$ and $q = 3329$. No polynomial-time quantum algorithm is currently known for either problem, in contrast to the integer factorisation and discrete logarithm assumptions broken by Shor's algorithm [15].

NIST defines five security categories. Category 1 matches AES-128 [4], Category 3 matches AES-192, and Category 5 matches AES-256. Kyber-768 is the Category 3 parameter set with module rank $k = 3$, ciphertext size 1088 bytes, and public-key size 1184 bytes. Category 3 is selected for the proposed framework as a balance between long-term security margin and on-the-wire ciphertext size: Category 1 is regarded as conservative for ballots that may need to remain confidential beyond 2050, whereas Category 5 imposes a $\sim 30\%$ larger ciphertext without a corresponding gain in voting-specific assurance.

3.4.7.2 KEM-DEM Hybrid Construction

The hybrid follows the standard KEM-DEM paradigm: the post-quantum KEM transports a fresh symmetric key, which is then used in a classical authenticator over the (per-candidate) Paillier ciphertext vector. The procedure operates as follows:

1. **Kyber Key Generation:** Generate a Kyber768 key pair (pk_{ky}, sk_{ky}) providing NIST Level 3 security (equivalent to AES-192).
2. **Paillier Encryption** [8]: Encrypt the per-candidate 1-hot vote vector $\{v_j\}_{j=1}^m$ (Section 3.4.3) under Paillier, yielding the ciphertext vector $\{E_j\}_{j=1}^m$ with $E_j = (1 + n)^{v_j} \cdot r_j^n \bmod n^2$, preserving the additive homomorphism required for tallying.
3. **Encapsulation** [12]: Generate a shared secret K and ciphertext ct_{ky} :

$$(K, ct_{ky}) = \text{Kyber.Encapsulate}(pk_{ky}) \quad (3.28)$$

4. **Salt and Key Derivation:** Sample a fresh 32-byte salt $\sigma_{\text{salt}} \xleftarrow{\$} \{0, 1\}^{256}$ per cast (domain-separation against replay and per-vote key isolation), and derive the authentication key $K_{\text{auth}} = \text{SHA256}(K \parallel \sigma_{\text{salt}})$ [29].

5. **MAC Authentication** [31, 32]: Compute one HMAC-SHA256 over the concatenated per-candidate ciphertext vector so that a single tag protects the whole ballot’s integrity:

$$\text{MAC} = \text{HMAC-SHA256}(K_{\text{auth}}, E_1 \parallel E_2 \parallel \dots \parallel E_m) \quad (3.29)$$

6. **Transmission**: Send $(\text{ct}_{\text{ky}}, \{E_j\}_{j=1}^m, \sigma_{\text{salt}}, \text{MAC})$.

This hybrid construction provides dual-layer protection *under the pre-quantum threat model*: the Paillier ciphertexts $\{E_j\}$ provide semantic security under DCRA and preserve the homomorphic structure required for tallying, while the Kyber-768 layer provides a post-quantum integrity binding via the HMAC. An adversary must compromise *both* layers to tamper with a ballot undetected. *In the post-quantum threat model*, however, the situation is more nuanced: an adversary who can break Module-LWE to recover sk_{ky} can typically also factor the Paillier modulus n via Shor’s algorithm [15], simultaneously breaking DCRA. The Kyber-MAC therefore protects vote-integrity *only at rest and in transit during the pre-quantum window*; long-term post-quantum confidentiality of historical ballots requires the migration to a fully post-quantum stack outlined in Section 3.4.7.3. The framework additionally uses AES-256-GCM [33] for authenticated symmetric encryption of non-homomorphic payloads (audit-log entries, session tokens) outside the vote-ciphertext path.

3.4.7.3 Scope and Limitations of Post-Quantum Coverage

The ML-KEM-768 layer described above provides post-quantum integrity for vote ciphertexts in transit and at rest. However, the homomorphic tallying remains classically secure under DCRA, and the linkable ring signatures rely on the discrete logarithm assumption, both of which are vulnerable to Shor’s algorithm. A future-proof migration would replace these components with lattice-based alternatives: a fully homomorphic encryption scheme (e.g., BGV or CKKS) for tallying, and ML-DSA [16] or a lattice-based linkable ring signature scheme such as those built from Module-SIS. This migration is identified as future work in Chapter 5.

3.4.8 Threshold Paillier Decryption

To prevent any single entity from decrypting election results, our framework implements the Damgård-Jurik-Nielsen (DJN) [13] threshold decryption scheme on top of Shamir’s Secret Sharing [14]. Throughout this subsection we use T for the total number of trustees and t for the cooperation threshold, to avoid the symbol clash with the Paillier modulus n .

3.4.8.1 Key Distribution

The Paillier private key λ is split into T shares using (t, T) -Shamir Secret Sharing [14]: the dealer chooses a polynomial $f \in \mathbb{Z}[X]$ of degree $t - 1$ with $f(0) = \lambda$, and trustee $i \in \{1, \dots, T\}$ receives share $\lambda_i = f(i)$. At least t trustees must cooperate to reconstruct λ via Lagrange interpolation at $X = 0$.

3.4.8.2 Partial Decryption with ZK Correctness Proof

For each candidate's homomorphically-aggregated ciphertext $c \in \mathbb{Z}_{n^2}^*$, trustee i computes a partial decryption together with a non-interactive zero-knowledge proof of correctness:

$$d_i = c^{\lambda_i} \bmod n^2 \quad (3.30)$$

The accompanying proof is a Chaum–Pedersen equality-of-discrete-logarithms [37, 26] demonstrating that the same exponent λ_i produced both the public commitment g^{λ_i} (registered at key-distribution time) and the partial decryption $d_i = c^{\lambda_i}$. Concretely, trustee i chooses random $u \in \mathbb{Z}_q$, computes commitments $a_1 = g^u$ and $a_2 = c^u \bmod n^2$, derives the Strong Fiat–Shamir challenge $e = H(\text{DS}_{\text{thr}} \parallel c \parallel d_i \parallel a_1 \parallel a_2)$, and returns the response $s = u + e \cdot \lambda_i$. Verifiers accept iff $g^s \equiv a_1 \cdot (g^{\lambda_i})^e$ and $c^s \equiv a_2 \cdot d_i^e \pmod{n^2}$.

3.4.8.3 Verification and Combination

The combiner first *verifies* every received pair (d_i, proof_i) against the public share commitment g^{λ_i} using the Chaum–Pedersen check above and rejects any partial whose proof fails. Given t *valid* partial decryptions indexed by $S \subseteq \{1, \dots, T\}$, Lagrange interpolation at $X = 0$ reconstructs the plaintext:

$$x = L \left(\prod_{i \in S} d_i^{\delta_i} \bmod n^2 \right) \cdot \mu \bmod n \quad (3.31)$$

where the Lagrange coefficients $\delta_i = \prod_{j \in S, j \neq i} \frac{j}{j-i}$ correspond to evaluation at $X = 0$ with positive-integer share indices $\{1, \dots, T\}$ [14]. Because the δ_i are rationals while the exponents in Equation (3.31) must be integers, the DJN construction [13] introduces an integer scaling factor $\Delta = T!$ (omitted above for clarity) that clears the denominators in the Lagrange formula; the integer-form $2\Delta^2\delta_i$ is used in the actual exponentiation. The plaintext x is the decrypted per-candidate tally.

3.4.9 Behavioural Duress Detection

The behavioural duress detection mechanism described below is a novel contribution of this thesis. The construction uses HMAC-SHA256 [31, 32, 30] as a building block, but the deterministic per-voter equality check via constant-time HMAC comparison, combined with the silent-substitution semantics that interact with the SMDC fake-slot system, are original to this

work and are not, to the best of our knowledge, present in prior e-voting systems. The mechanism produces a $\{0, 1\}$ scalar (behaviorWeight) that multiplies the SMDC slot weight at Step 9 of Algorithm 2, so that a vote known to be cast under duress contributes the Paillier encryption of zero to the final tally while remaining cryptographically and operationally indistinguishable from a genuine cast.

At registration time the voter selects a secret behavioural pattern of one of five accepted types (blink count, head tilt, long press, time delay, or voice command) together with a value drawn from a per-type allowlist. The server stores only the HMAC-SHA256 of type $\parallel \text{":"} \parallel$ value keyed by a server-side secret K_d ; the raw value is never persisted. At vote-cast time, the client may supply a candidate signal alongside the ballot; the server recomputes the HMAC and compares it to the stored hash in constant time. Algorithm 1 formalises the procedure.

Algorithm 1 Behavioural Duress Detection (mirrors `internal/biometric/duress.go`)

Require: Voter ID vid; presented (signalType, signalValue); server HMAC key K_d ; per-voter stored hash h_{stored} or $\perp$ if none registered
Ensure: behaviorWeight $\in \{0, 1\}$

```

1: if  $h_{\text{stored}} = \perp$  then
2:   return 1                                ▷ No signal registered: backward-compatible pass
3: end if
4: if signalType  $\notin \{\text{blink\_count}, \text{head\_tilt}, \text{long\_press}, \text{time\_delay}, \text{voice\_cmd}\}$  then
5:   return 0
6: end if
7:  $h_{\text{pres}} \leftarrow \text{HMAC-SHA256}(K_d, \text{signalType} \parallel \text{":"} \parallel \text{signalValue})$ 
8: if  $\text{CONSTANTTIMECOMPARE}(h_{\text{stored}}, h_{\text{pres}}) = 1$  then
9:   return 1                                ▷ Genuine signal: vote counted
10: else
11:   return 0                                ▷ Mismatch: silently zero the weight
12: end if

```

The output behaviorWeight is multiplied with the SMDC slot weight in the vote-casting pipeline (Step 9 of Algorithm 2, where $\text{finalWeight} = \text{slot.Weight} \times \text{behaviorWeight}$). When the result is zero, each per-candidate encrypted vote becomes $E_j^0 = E(0)$ via Paillier scalar multiplication, which contributes nothing to the final tally. The pipeline continues to execute the full ring signature and SA^2 share-split steps before silently discarding the key-image registration via the coercion short-circuit at Steps 17a–17b of Algorithm 2, so that timing and the receipt returned to the voter are indistinguishable from a genuine cast. The formal security properties of this construction (timing indistinguishability and database-compromise resistance) are analysed in Section 3.8.2.

3.5 Vote Casting Pipeline

Figure 3.3 illustrates the four-phase pipeline, and Algorithm 2 presents the formal pseudocode.

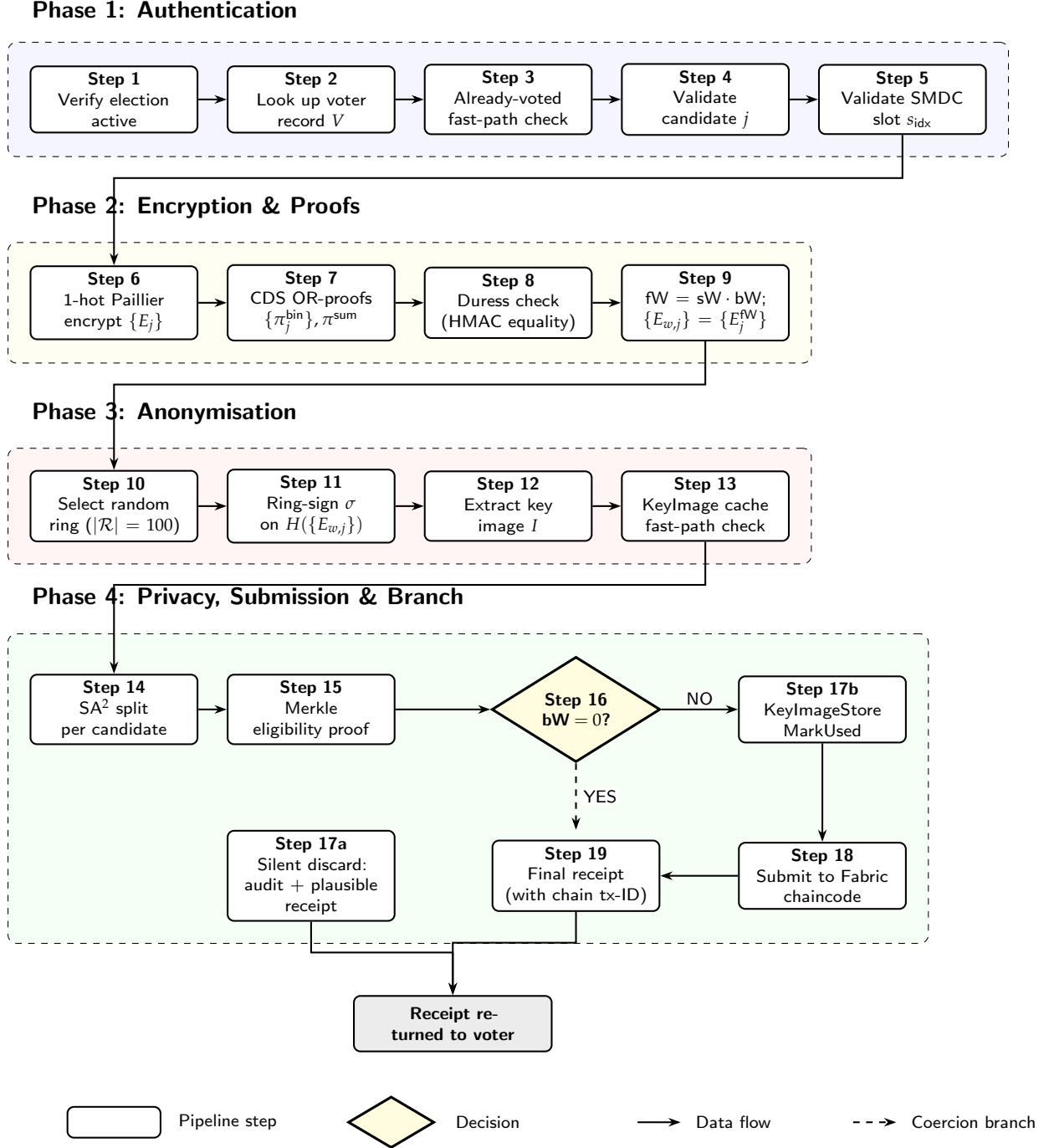


Fig. 3.3: The proposed 20-step vote-casting pipeline, organised into four phases (Authentication, Encryption & Proofs, Anonymisation, Privacy & Submission). Each pastel-coloured wrapper bounds one phase. The decision diamond at Step 16 routes the flow either to the genuine submission branch (Steps 17b–19) or to the silent-discard branch (Step 17a) that returns a plausible receipt to the voter under coercion.

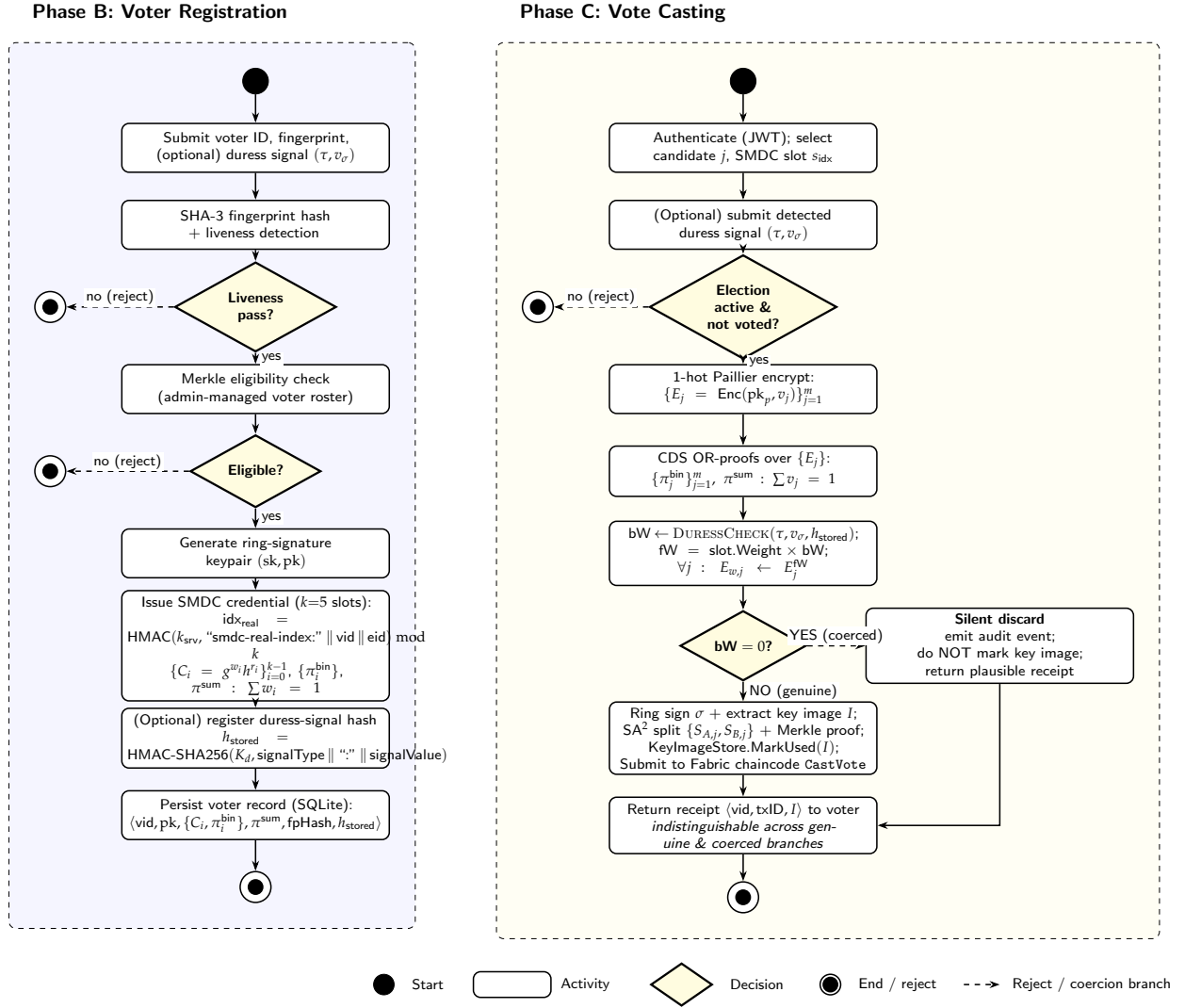


Fig. 3.4: Voter activity diagram covering the registration phase (left swimlane) and the vote-casting phase (right swimlane). Decision diamonds carry both the genuine and the coerced execution paths; reject paths terminate at the bullseye end nodes.

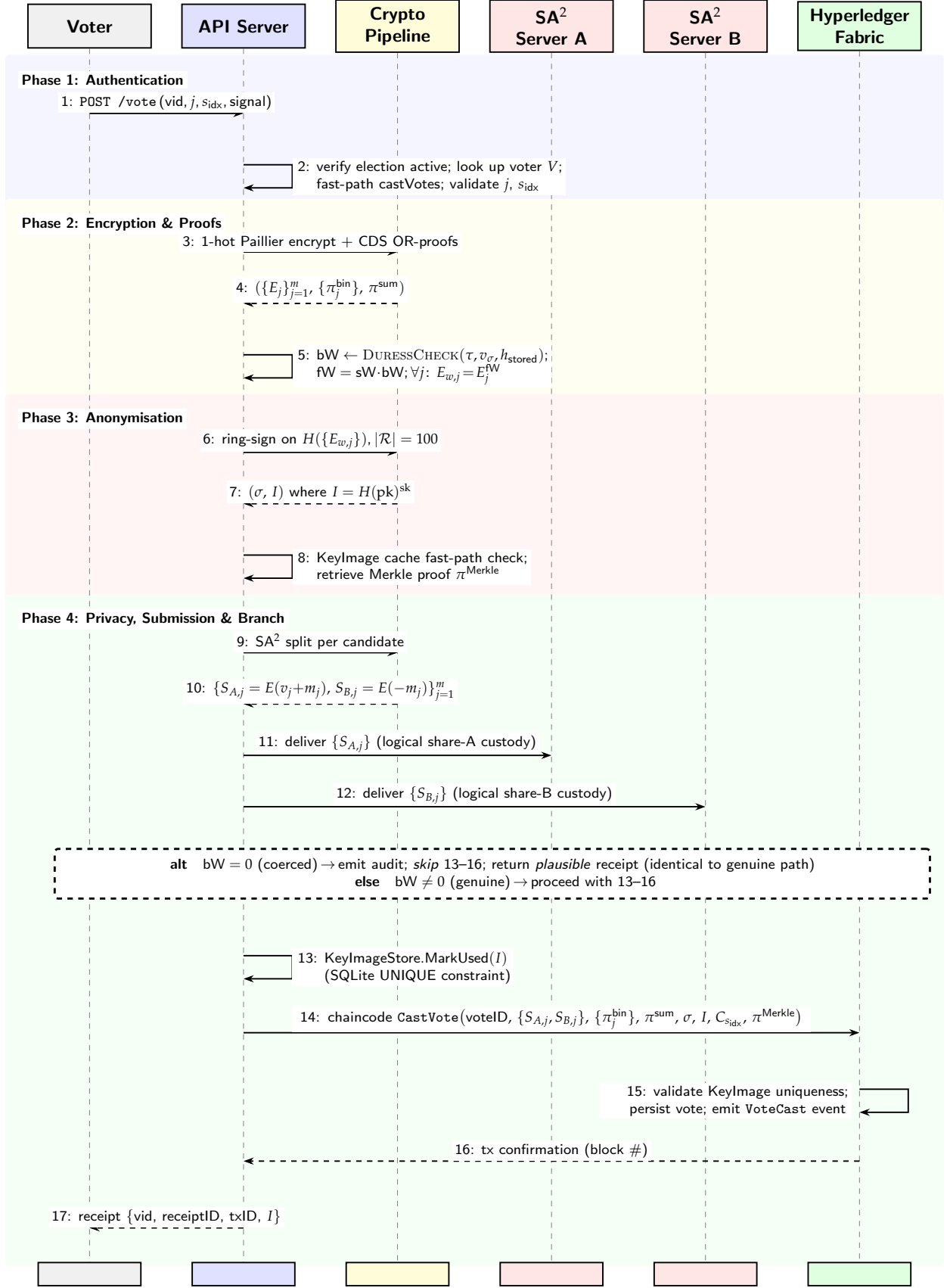


Fig. 3.5: Sequence diagram of one full vote-cast interaction. Solid arrows are synchronous request messages; dashed arrows are responses; self-arrows are internal computations. The **alt** block indicates the coercion short-circuit: when the duress signal does not match, steps 14–16 are skipped and the receipt returned to the voter is identical to a genuine cast.

Algorithm 2 Proposed 20-Step Vote Casting Pipeline (mirrors internal/voting/cast.go)

Require: Voter ID vid ; candidate selection j ; SMDC slot index s_{idx} supplied by client; presented duress signal (τ, v_σ) or $\perp$; voter record (ring keypair, SMDC credential, fingerprint); election parameters

Ensure: Encrypted, ring-signed, share-split vote record persisted on Hyperledger Fabric, or a plausible receipt under coercion

// Phase 1: Authentication

- 1: Verify election is active and within voting window
- 2: Look up voter record V by vid
- 3: Fast-path check: voter has not already cast a vote (RLock on `castVotes`)
- 4: Validate that candidate j is in the election's candidate list
- 5: Validate $0 \leq s_{\text{idx}} < k$; let $\text{slot} \leftarrow V.\text{SMDCCredential.Slots}[s_{\text{idx}}]$

// Phase 2: Encryption & Proofs

- 6: Encode 1-hot: $v_j \leftarrow [j = \text{chosen}]$ for $j \in [m]$; encrypt $E_j \leftarrow \text{Paillier.Enc}(\text{pk}_p, v_j)$ $\triangleright$ per-candidate ciphertexts
- 7: Generate per-candidate binary proofs π_j^{bin} and the sum proof π^{sum} directly on $(E_1, \dots, E_m)$ $\triangleright$ §3.4.3; CDS OR-proofs in the ROM
- 8: Compute $\text{behaviorWeight} \leftarrow \text{DURESSCHECK}(\text{vid}, \tau, v_\sigma, V.h_{\text{stored}})$ $\triangleright$ Algorithm 1
- 9: $\text{finalWeight} \leftarrow \text{slot.Weight} \times \text{behaviorWeight}$; $\forall j : E_{w,j} \leftarrow E_j^{\text{finalWeight}}$ $\triangleright$ Paillier scalar multiplication per candidate

(The SMDC slot commitments and their binary/sum proofs were generated and verified at credential-issue time per §3.4.5 and are re-verified defence-in-depth at cast time per cast.go:208; they are distinct from the per-candidate Paillier proofs generated in the previous step.)

// Phase 3: Anonymisation

- 10: Select random ring $\mathcal{R}$ of size $|\mathcal{R}| = 100$ from registered voter public keys
- 11: Compute $\sigma \leftarrow \text{RingSign}(H(E_{w,1} \parallel \dots \parallel E_{w,m}), V.\text{sk}, \mathcal{R}, \text{signerIdx})$ $\triangleright$ sign hash of ciphertext vector
- 12: Extract key image $I \leftarrow H(V.\text{pk})^{V.\text{sk}}$ from σ
- 13: Fast-path check: $I \notin \text{usedKeyImages}$ (in-memory cache, RLock)

// Phase 4: Privacy, Submission & Branch

- 14: $\forall j : (S_{A,j}, S_{B,j}) \leftarrow \text{SplitVote}(\text{vid}, E_{w,j})$ $\triangleright$ per-candidate SA^2 split; $S_{A,j} = E(v_j + m_j)$, $S_{B,j} = E(-m_j)$
 - 15: Retrieve Merkle eligibility proof π^{Merkle} for vid
 - 16: **if** $\text{behaviorWeight} = 0$ **then** $\triangleright$ Coercion short-circuit (novel; cast.go:348)
 - 17: Emit audit event `LOGDURESSSIGNALMISMATCH(vid, eid)`
 - 18: **return** plausible receipt; *do not* persist key image $\triangleright$ voter can re-vote later
 - 19: **end if**
 - 20: `KeyImageStore.MarkUsed(I)` $\triangleright$ atomic; SQLite UNIQUE constraint enforces single-use
 - 21: Submit $\{\text{voteID}, \{S_{A,j}, S_{B,j}\}_{j=1}^m, \{\pi_j^{\text{bin}}\}_{j=1}^m, \pi^{\text{sum}}, \sigma, I, C_{s_{\text{idx}}}, \pi^{\text{Merkle}}\}$ to Hyperledger Fabric chaincode `CastVote`
 - 22: **return** receipt $\{\text{vid}, \text{receiptID}, \text{txID}, I\}$
-

3.6 Blockchain Integration

The framework uses Hyperledger Fabric v2.5 [1] as its blockchain infrastructure. The deployment evaluated in Chapter 4 consists of:

- **1 Raft Orderer Node** (`orderer.covertvote.com:7050`): Provides transaction ordering using the `etcdraft` consensus type [35]. This single-node configuration is sufficient for the development and benchmarking workload reported in Chapter 4; a production deployment scales horizontally to three or more orderer nodes for crash fault tolerance through configuration only, since Fabric’s `etcdraft` implementation is configuration-driven and the application code is unaffected by the orderer count.
- **2 Peer Nodes** (`peer0.org1`, `peer0.org2`): Maintain the distributed ledger and execute chaincode under the standard two-organisation MSP model.
- **MSP Identity Material**: Generated offline by Hyperledger’s `cryptogen` utility for the static development network. A production deployment would replace `cryptogen` with a `fabric-ca-server` to enable dynamic enrollment and certificate revocation; the chaincode and application code are unaffected by this substitution.
- **State Database**: CouchDB, enabling JSON-rich queries used by `GetVotesByElection` and other public-verifier query endpoints that traverse votes by election ID.

The chaincode (smart contract), deployed on channel `covertvotechannel`, is written in Go using the `contractapi` package. It implements the following transaction and query functions:

- **Election lifecycle**: `CreateElection` initialises an election with metadata and candidate list; `GetElection` retrieves election state; `ElectionExists` performs an existence check.
- **Vote casting**: `CastVote` records one per-voter ballot bundle, containing the per-candidate SA^2 share pairs $\{S_{A,j}, S_{B,j}\}$, per-candidate binary proofs $\{\pi_j^{\text{bin}}\}$, sum proof π^{sum} , linkable ring signature σ , key image I , SMDC slot commitment $C_{s_{\text{idx}}}$, and Merkle eligibility proof π^{Merkle} ; single-use is enforced by application-side `KeyImageStore.MarkUsed` (SQLite UNIQUE constraint, Algorithm 2 Step 17b) followed by chaincode-side ballot persistence.
- **Public verification queries**: `GetVote` returns one vote bundle by ID; `GetVotesByElection` returns all ballots for a given election (used by the off-chain Tally Service); `GetAllVotes` returns a paginated summary view; `VoteExists` performs an existence check.

- **Tally persistence:** `StoreTallyResult` accepts the final plaintext tally vector computed off-chain by the Tally Service (described below); `GetTallyResult` retrieves it for public auditing.
- **Access-control helpers:** `requireAdmin` enforces an admin-organisation MSP identity on administrative transactions (`CreateElection`, `GetAllVotes`, `GetVotesByElection`, `StoreTallyResult`); `requireAuthenticated` enforces a valid MSP context on voter-facing transactions (`CastVote`, `GetVote`). These Fabric MSP gates complement the cryptographic eligibility proof π^{Merkle} submitted within each `CastVote` payload.

The chaincode deliberately does *not* perform homomorphic tally aggregation on chain. Aggregating tens of thousands of Paillier ciphertexts inside a single Fabric transaction would exceed the per-transaction time budget and cause world-state contention; furthermore, the threshold-Paillier decryption of Section 3.4.8 inherently requires off-chain multi-party coordination among trustees, which Fabric’s deterministic execution model cannot encode natively. Aggregation is therefore performed off chain by the **Tally Service** (Figure 3.1), which (i) queries all ballots for the election via `GetVotesByElection`, (ii) computes the homomorphic per-candidate product $\prod_v E_{w,j,v}$ that materialises $E(\sum_v w_v \cdot v_{j,v})$, (iii) runs the DJN threshold partial-decryption protocol across the trustee group, recovers the plaintext tally via Lagrange interpolation, and (iv) publishes the final result back on chain via `StoreTallyResult`. Any public verifier can subsequently re-fetch both the on-chain ballots and the stored tally to audit the result.

Application-level communication uses the Fabric Gateway SDK v1.10.1 with gRPC over TLS, providing authenticated, encrypted communication between the API server and peer nodes. TLS root certificates are provisioned alongside the MSP material, and every client invocation carries an MSP signature that is validated against the channel’s access-control policies before chaincode execution.

3.7 Engineering and Societal Considerations

3.7.1 Societal Impact

The proposed framework addresses voter coercion, a problem particularly prevalent in developing nations where family voting, vote buying, and employer pressure undermine democratic processes. The SMDC mechanism provides mathematical guarantees of coercion resistance, potentially improving electoral integrity for billions of voters worldwide.

3.7.2 Sustainability

Unlike proof-of-work blockchain systems, Hyperledger Fabric [1] uses Raft consensus [35], which orders transactions by leader election and majority commit rather than by computational work. The energy cost per transaction is therefore bounded by the ordinary

networked-CPU baseline of a Linux container, not by hash-rate. The sustained throughput attained in our deployment (200 TPS in the Caliper saturation scenario; see Chapter 4) was reached on commodity hardware without specialised acceleration, so the per-vote energy budget is in line with that of any standard service-tier web application rather than that of a public proof-of-work chain.

3.7.3 Accessibility

The system is exposed through a REST API that decouples the client form factor from the cryptographic backend, so that browser, mobile, kiosk, and assistive-device front ends can share a single endpoint without protocol changes. The per-vote payload grows linearly in the candidate count rather than in the voter count, which keeps submission size compact and suits low-bandwidth or intermittent-connectivity settings such as rural polling or overseas voting. The duress-signal taxonomy intentionally includes a *voice command* type alongside the visual and haptic types so that voters with limited vision or fine-motor disability retain access to the coercion-resistance mechanism. Remote operation further removes the physical-mobility barrier of polling-station-based voting and improves participation for voters with mobility challenges.

3.7.4 Ethical Considerations

The framework’s privacy and coercion-resistance mechanisms each carry an ethical trade-off that deserves explicit acknowledgement. *Deniable credentials* (SMDC) provide mathematical protection against vote-buying and coercion, but they also make it impossible to prove how any individual voted—a foundational requirement of secret-ballot democracy that simultaneously precludes legitimate voter self-verification beyond the cryptographic casting receipt. *Behavioral duress detection* silently zeroes a vote’s weight when a registered signal is supplied; a voter who supplies the signal in error—or who forgets it under stress—will have their vote silently discarded with no out-of-band indication. This is an intentional safety property (the receipt is identical to the genuine path, so a coercer cannot distinguish the two cases), but it places non-trivial cognitive load on the voter at registration time. The *silent-discard short-circuit* (Algorithm 2, Steps 17a–17b) protects against coercion at the cost of voter-side opacity: even a genuine voter cannot definitively confirm that their vote reached the tally without trusting the system end-to-end. Finally, the same deniable infrastructure that protects voters from external coercion could in principle be misused by a malicious election authority to silently void targeted votes; the integrity safeguards (Merkle eligibility proofs, per-vote zero-knowledge proofs, and auditable on-chain submissions) are therefore essential and must be preserved during any deployment.

3.7.5 Privacy and Data Protection

The framework stores only the minimum data required for vote validity and coercion resistance. At registration time, voter identifiers, ring public keys, the Pedersen-based SMDC commitments $\{C_i\}$ with their binary proofs $\{\pi_i^{\text{bin}}\}$ and sum proof π^{sum} , and the SHA-3 hash of the fingerprint are persisted; the raw fingerprint image is discarded immediately after hashing. The duress-signal value is *never* stored in plaintext: only the HMAC-SHA256 hash keyed by the server-side secret K_d is retained (Section 3.4.9), so that a database compromise cannot recover any voter’s secret signal value. On chain, only the weighted SA² share pairs $\{S_{A,j}, S_{B,j}\}$, the per-candidate ZK proofs, the ring signature, and the key image are written; no plaintext vote and no identifying voter data ever reach Fabric, in keeping with the architectural principle that the ledger should contain auditable cryptographic evidence rather than personally identifying information. The audit log of duress-signal mismatches records only the (vid, eid) pair and a timestamp; it never records the presented signal value, so even an audit-log compromise cannot reconstruct the coercion scenario faced by a particular voter.

3.8 Security Analysis

This section presents the formal security properties of the proposed framework in two complementary tiers. *Tier 1* (this section) provides **computational, game-based reductions** of each property to a standard cryptographic hardness assumption (DCRA, DDH, Module-LWE, or DLog). *Tier 2*, presented in Section 4.7 of Chapter 4, provides complementary *symbolic-model* verification of three protocol-level properties (privacy, individual verifiability, eligibility) using ProVerif under the Dolev–Yao adversary model [39]. The two tiers are deliberately complementary: the computational reductions bind security claims to well-studied hard problems in the standard model, while the symbolic verification rules out network-level protocol attacks (replay, message interleaving, parallel-session interference) that are not naturally expressed in the computational setting.

3.8.1 Computational Security: Game-Based Reductions

The reductions in this subsection follow the now-standard *sequence-of-games* methodology: each theorem is proved by exhibiting hybrid experiments $G_0, G_1, \dots, G_k$ in which G_0 is the real attack experiment, G_k is an experiment whose outcome is information-theoretically (or trivially) determined, and consecutive hybrids differ in a way bounded by the advantage of breaking a stated hardness assumption. Notational conventions and the formal hardness assumptions are summarised in Sections 3.8.1.1 and 3.8.1.2 respectively.

3.8.1.1 Notational Conventions for Proofs

Throughout this subsection we use the following conventions: λ is the security parameter (the proposed implementation uses Paillier modulus $|n| = 2048$ bits, giving $\lambda \approx 112$); $\text{negl}(\lambda)$ denotes a function negligible in λ ; PPT abbreviates *probabilistic polynomial-time*; $\mathcal{A}$ denotes an adversary, $\mathcal{C}$ a challenger, and $\mathcal{B}$ a reduction algorithm that runs $\mathcal{A}$ as a subroutine; $\xleftarrow{\$}$ denotes uniform random sampling; $\stackrel{c}{\equiv}$ and $\stackrel{s}{\equiv}$ denote computational and statistical indistinguishability respectively; $\text{Adv}_{\mathcal{A}}^X(\lambda) = |\Pr[\mathcal{A} \text{ wins game } X] - \frac{1}{2}|$ for indistinguishability games and $\Pr[\mathcal{A} \text{ wins game } X]$ for soundness or unforgeability games. Hash functions are modelled as random oracles when the construction relies on the Fiat–Shamir transform.

3.8.1.2 Hardness Assumptions

The reductions reduce ballot security to the following four assumptions, all of which are standard in the cryptographic literature.

DCRA (Decisional Composite Residuosity Assumption) [8]. For an RSA modulus $n = pq$ with $|p| = |q| \geq 1024$ bits, the distribution of $\{(n, c) : c \xleftarrow{\$} \mathbb{Z}_{n^2}^*\}$ is computationally indistinguishable from the distribution of $\{(n, c) : c = (1 + n)^m r^n \bmod n^2, m \xleftarrow{\$} \mathbb{Z}_n, r \xleftarrow{\$} \mathbb{Z}_n^*\}$. Used in Theorems 1, 5.

DDH (Decisional Diffie–Hellman) in the order- q subgroup of $\mathbb{Z}_p^*$: $\{(g^a, g^b, g^{ab})\} \stackrel{c}{\equiv} \{(g^a, g^b, g^c)\}$ for $a, b, c \xleftarrow{\$} \mathbb{Z}_q$. Used in Theorem 3.

DLog (Discrete Logarithm) in the order- q subgroup of $\mathbb{Z}_p^*$: given g^x for $x \xleftarrow{\$} \mathbb{Z}_q$, no PPT adversary outputs x with non-negligible probability. Used in Theorem 3.8.1.5 (key-image pseudorandomness for ring anonymity) and in the Pedersen-binding analysis of the SMDC slot commitments (Section 3.4.5). *Note:* Theorem 3.8.1.4 (vote validity) does *not* reduce to DLog in the A2 (Paillier-direct) ZK design adopted here; its soundness is unconditional in the ROM.

Module-LWE (M-LWE) [12]. For module rank k , modulus q , dimension n , and error distribution χ over $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, the distribution $\{(A, As + e) : A \xleftarrow{\$} R_q^{k \times k}, s \xleftarrow{\$} R_q^k, e \xleftarrow{\$} \chi^k\}$ is computationally indistinguishable from $\{(A, u) : u \xleftarrow{\$} R_q^k\}$. For Kyber-768: $n = 256, q = 3329, k = 3$. Used in Theorem 6.

In addition, the reductions invoke the non-cryptographic trust assumptions enumerated in Section 3.2. Of these, four are explicitly cited below: **Assumption 1** (the Hyperledger Fabric bulletin board provides append-only integrity), used as a backbone in Theorems 2 and 3; **Assumption 2** (at least one of the two SA² aggregation servers is honest), used in Theorem 5; **Assumption 5** (credentials are issued out-of-band before the election),

used to delimit Theorem 4's threat scope; and **Assumption 6** (the server HMAC keys k_{srv}, K_d remain confidential), used in Theorem 4.

3.8.1.3 Theorem 1: Ballot Indistinguishability (DCRA)

Informal claim. The complete ballot $\text{Ballot} = (E_1, \dots, E_m, \pi_1^{\text{bin}}, \dots, \pi_m^{\text{bin}}, \pi^{\text{sum}}, \sigma, S_A, S_B, \text{ct}_{\text{ky}}, \text{salt}, M_A)$ jointly hides the vote vector $\mathbf{v}$. Distinguishing the ballots cast for any two valid choices is computationally infeasible under DCRA, with negligible additional loss to the zero-knowledge, ring-anonymity, and post-quantum integrity assumptions used by the auxiliary components. This is the central security claim of the framework.

Ballot composition recap. A ballot for vote $\mathbf{v} = (v_1, \dots, v_m) \in \{0, 1\}^m$ with $\sum_j v_j = 1$ contains: per-candidate Paillier ciphertexts $E_j = \text{Paillier.Enc}(\text{pk}_p, v_j)$; the per-candidate binary Σ -proofs π_j^{bin} (proving $v_j \in \{0, 1\}$ directly on E_j , per Section 3.4.3.1); the single sum proof π^{sum} (proving $\sum_j v_j = 1$ on $\prod_j E_j$, per Section 3.4.3.2); a linkable ring signature σ over $(E_1, \dots, E_m)$ in a ring of $|\mathcal{R}| = 100$; the SA^2 shares (S_A, S_B) ; and the Kyber-MAC outer wrapper. The Pedersen commitments of the SMDC credential (Section 3.4.5) are bound separately to the voter's credential and are not part of the per-ballot indistinguishability argument.

IND-Ballot-CPA experiment. Define $\text{IND-Ballot-CPA}_{\mathcal{A}}(\lambda)$ as follows. The challenger $\mathcal{C}$ runs full system setup (Paillier 2048-bit, Pedersen, ring keys $\{(\text{pk}_i, \text{sk}_i)\}_{i=1}^{|\mathcal{R}|}$, SMDC credentials, Kyber-768 keys) and gives $\mathcal{A}$ all public parameters and bulletin-board access. In a pre-challenge phase, $\mathcal{A}$ issues any number of $\text{Cast}(\text{vid}, \mathbf{v})$ queries to an honest-cast oracle, corrupts up to $|\mathcal{R}| - 1 = 99$ ring members (excluding the eventual target voter id^*), and queries a Reveal oracle for the randomness of any past honest ballot. $\mathcal{A}$ outputs two valid vote vectors $\mathbf{v}_0, \mathbf{v}_1$ and the target voter identity id^* whose ring secret is uncorrupted. $\mathcal{C}$ samples $b \xleftarrow{\$} \{0, 1\}$ and returns the complete challenge ballot Ballot^* for $\mathbf{v}_b$ signed by id^* . After post-challenge oracle queries (that do not target id^*), $\mathcal{A}$ outputs b' and wins if $b' = b$. Define $\text{Adv}_{\mathcal{A}}^{\text{IND-Ballot-CPA}}(\lambda) = |\Pr[b' = b] - 1/2|$.

Theorem 1 (Ballot Indistinguishability, Computational). Let $\Delta = |\{j : v_{0,j} \neq v_{1,j}\}|$ be the Hamming distance between the two challenge votes ($\Delta = 2$ for distinct valid single-choice votes). For any PPT adversary $\mathcal{A}$,

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\text{IND-Ballot-CPA}}(\lambda) &\leq \text{Adv}_{\mathcal{B}_{\text{ZK}}}^{\text{ZK}}(\lambda) + \text{Adv}_{\mathcal{B}_{\sigma}}^{\text{Anon}}(\lambda) \\ &\quad + \text{Adv}_{\mathcal{B}_{\text{Hyb}}}^{\text{INT-CTXT}}(\lambda) + \Delta \cdot \text{Adv}_{\mathcal{B}_{\text{DCRA}}}^{\text{DCRA}}(\lambda) + \text{negl}(\lambda). \end{aligned} \quad (3.32)$$

Each advantage term on the right is negligible under, respectively, the zero-knowledge property of the Strong-Fiat-Shamir Σ -protocol (Theorem 3.8.1.4), the LWW signer-ambiguity property (Theorem 3.8.1.5), the Kyber-HMAC integrity property (Theorem 3.8.1.8), and the Decisional Composite Residuosity Assumption (Section 3.8.1.2).

Proof. We proceed via four hybrids $G_0 \rightarrow G_3$ that successively replace the auxiliary ballot components with simulator-generated values, then close the remaining Paillier indistinguishability via a per-position hybrid telescoping over the differing positions.

Game G_0 . The real IND-Ballot-CPA experiment. By definition $\Pr[\mathcal{A} \text{ wins } G_0] = 1/2 + \varepsilon$ where $\varepsilon = \text{Adv}_{\mathcal{A}}^{\text{IND-Ballot-CPA}}(\lambda)$.

Game G_1 (simulate ZK proofs). $\mathcal{C}$ replaces π_j^{bin} and π^{sum} in Ballot^* with simulator-generated values $\tilde{\pi}_j^{\text{bin}}, \tilde{\pi}^{\text{sum}}$ produced via the Schnorr / CDS simulator without using the witness. Programming the random oracle at the corresponding hash queries makes the simulated proofs pass verification. Under Strong Fiat–Shamir in the random-oracle model [9, 37], real and simulated proofs are computationally indistinguishable. Bound:

$$|\Pr[\mathcal{A} \text{ wins } G_1] - \Pr[\mathcal{A} \text{ wins } G_0]| \leq \text{Adv}_{\mathcal{B}_{\text{ZK}}}^{\text{ZK}}(\lambda).$$

After G_1 , the proofs in Ballot^* carry no information about the witness $(\mathbf{v}_b, \mathbf{r})$.

Game G_2 (simulate ring signature). $\mathcal{C}$ replaces σ in Ballot^* with a simulated signature $\tilde{\sigma}$ produced using the LWW signer-ambiguity simulator (Liu–Wei–Wong 2004 S4.2 Theorem 4 [10]): sample $(c_i, r_i)_{i \in \mathcal{R}}$ uniformly except for one randomly-chosen index i^* used to close the ring. The non-signer values (c_i, r_i) are unconditionally uniform in the real signing procedure as well, so $\tilde{\sigma}$ is statistically close to σ as a signature object; the only b -dependent leakage path through σ in the real game is via the key image $I = H(\text{pk}_{\text{id}^*})^{\text{sk}_{\text{id}^*}}$, whose pseudorandomness reduces to DDH. Bound:

$$|\Pr[\mathcal{A} \text{ wins } G_2] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \text{Adv}_{\mathcal{B}_{\sigma}}^{\text{Anon}}(\lambda) \leq \text{Adv}_{\mathcal{B}_{\sigma}}^{\text{DDH}}(\lambda).$$

After G_2 , $\tilde{\sigma}$ in Ballot^* is independent of id^* .

Game G_3 (replace MAC with random). $\mathcal{C}$ replaces MAC^* with $\widetilde{\text{MAC}}^* \xleftarrow{\$} \{0, 1\}^{256}$. By Theorem 3.8.1.8 (Kyber-768 IND-CCA2 + HMAC-SHA256 EUF-CMA + ROM on the key-derivation step):

$$|\Pr[\mathcal{A} \text{ wins } G_3] - \Pr[\mathcal{A} \text{ wins } G_2]| \leq \text{Adv}_{\mathcal{B}_{\text{Hyb}}}^{\text{INT-CTXT}}(\lambda).$$

After G_3 , the MAC in Ballot^* is independent of b .

No per-candidate Pedersen step. Unlike a parallel Paillier–Pedersen design, the present ballot carries no per-candidate Pedersen commitments to the vote vector $\mathbf{v}$; the binary and sum proofs operate directly on the Paillier ciphertexts $(E_1, \dots, E_m)$ via the CDS OR-composition of Section 3.4.3.1–3.4.3.2, so no additional commitment hybrid is required. (The Pedersen commitments of the SMDC credential are bound to the voter’s credential and analysed separately in Theorem 3.8.1.6; they carry slot-weight information rather than per-candidate vote values and are independent of b in this experiment.)

Per-position hybrid for the Paillier ciphertexts. In G_3 the only b -dependent component of Ballot^* is the vector of Paillier ciphertexts $(E_1^*, \dots, E_m^*)$ (the SA^2 shares are derived deterministically and inherit the Paillier hiding analysis, cf. Theorem 3.8.1.7). Let $\text{Diff} = \{j : v_{0,j} \neq v_{1,j}\}$ with $|\text{Diff}| = \Delta$. Define a sub-hybrid sequence $H_0, H_1, \dots, H_{\Delta}$:

- $H_0 = G_3$ with Ballot^* generated for $\mathbf{v}_0$.
- H_k ($k = 1, \dots, \Delta$) is identical to H_{k-1} except that the k -th differing position (in some fixed ordering of Diff) is replaced with the corresponding ciphertext for $\mathbf{v}_1$.
- $H_\Delta = G_3$ with Ballot^* generated for $\mathbf{v}_1$.

Adjacent hybrids H_{k-1}, H_k differ in exactly one Paillier ciphertext at one position. A reduction $\mathcal{B}_{\text{DCRA}}$ that embeds a Paillier IND-CPA challenge ciphertext at that position perfectly simulates either H_{k-1} or H_k depending on the IND-CPA challenger's hidden bit, while generating all other ciphertexts honestly with known randomness (which lets it answer Reveal queries on non-challenge ballots faithfully). Hence

$$|\Pr[\mathcal{A} \text{ wins } H_k] - \Pr[\mathcal{A} \text{ wins } H_{k-1}]| \leq \text{Adv}_{\mathcal{B}_{\text{DCRA}}, \text{Paillier}}^{\text{IND-CPA}}(\lambda).$$

Telescoping over the Δ steps and applying Paillier 1999 Theorem 15 ($\text{Adv}_{\text{Paillier}}^{\text{IND-CPA}} \leq \text{Adv}^{\text{DCRA}} + \text{negl}(\lambda)$) yields the $\Delta \cdot \text{Adv}^{\text{DCRA}}$ term in Equation (3.32). For valid single-choice votes $\Delta = 2$, so the DCRA penalty is at most $2 \cdot \text{Adv}^{\text{DCRA}}$.

Combining the bounds for $G_0 \rightarrow G_3$ and the per-position telescope completes the proof of Equation (3.32). Each term is negligible by the corresponding Theorem (T2, T3, T6) or assumption (DCRA), so $\text{Adv}_{\mathcal{A}}^{\text{IND-Ballot-CPA}}(\lambda) = \text{negl}(\lambda)$. $\square$

Scope and limitations. Theorem 1 establishes computational indistinguishability of *individual ballots* under the standard model (with random-oracle assumptions on SHA-256 in the Fiat-Shamir, KEM-KDF, and HMAC components). Composability of all protocol layers in the multi-protocol stack under simultaneous execution, i.e., security in the Universal Composability framework [40], is identified as future work in Chapter 5. Section 3.8.1.9 below outlines the ideal-functionality structure that a future UC proof would target, and reports informally on the composition argument.

3.8.1.4 Theorem 2: Vote Validity (Soundness of CDS OR-proofs in the ROM)

Informal claim. The binary proofs π_j^{bin} and the sum proof π^{sum} together ensure that any ballot accepted by the verifier is well-formed: $\forall j \ v_j \in \{0, 1\}$ and $\sum_{j=1}^m v_j = 1$. An adversary cannot produce a verifying proof for an ill-formed ballot except with probability negligible in λ .

Formal relation. Let $n = pq$ be a Paillier modulus with $|p| = |q| \geq 1024$ bits. The well-formed-ballot relation is

$$R_{\text{ballot}} = \left\{ ((E_1, \dots, E_m); (v_1, \dots, v_m, r_1, \dots, r_m)) : \forall j \ E_j \equiv (1 + n)^{v_j} r_j^n \pmod{n^2} \wedge v_j \in \{0, 1\} \wedge \sum_{j=1}^m v_j = 1 \right\} \quad (3.33)$$

Soundness experiment. Define $\text{Snd}_{\mathcal{A}}^{R_{\text{ballot}}}(\lambda)$ as follows. The challenger $\mathcal{C}$ generates Paillier parameters n and initialises a random oracle $H : \{0, 1\}^* \rightarrow \{0, 1\}^{256}$. The adversary $\mathcal{A}$ receives n and adaptive oracle access to H , making at most q_H queries. Finally $\mathcal{A}$ outputs a candidate ballot $(E_1, \dots, E_m, \pi_1^{\text{bin}}, \dots, \pi_m^{\text{bin}}, \pi^{\text{sum}})$ with each $E_j \in \mathbb{Z}_{n^2}^*$. We say $\mathcal{A}$ wins if (i) every π_j^{bin} verifies, (ii) π^{sum} verifies, and (iii) no opening witness in R_{ballot} exists for $(E_1, \dots, E_m)$. Define

$$\text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) = \Pr \left[\mathcal{A} \text{ wins } \text{Snd}_{\mathcal{A}}^{R_{\text{ballot}}}(\lambda) \right].$$

Theorem 2 (Vote Validity, Knowledge Soundness). *Let H be modelled as a random oracle, let the challenge space have size 2^{256} , and let m be the number of candidates. For any PPT adversary $\mathcal{A}$ making at most q_H hash queries,*

$$\text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) \leq (m+1) \cdot 2^{-256} + (m+1) \sqrt{q_H \cdot 2^{-256}} + \text{negl}(\lambda). \quad (3.34)$$

Soundness holds unconditionally up to the ROM-extraction loss; no additional cryptographic hardness assumption is required.

Proof. We proceed by a sequence of hybrid games $G_0 \rightarrow G_1 \rightarrow G_2$, then exhibit a knowledge extractor that contradicts $\mathcal{A}$'s winning condition.

Game G_0 . The real soundness experiment defined above. By definition,

$$\Pr[\mathcal{A} \text{ wins } G_0] = \text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) = \varepsilon.$$

Game G_1 . Identical to G_0 except that $\mathcal{C}$ implements the random oracle H via lazy sampling: maintain a table T initially empty; on a query x , if $T(x)$ is undefined, sample $T(x) \xleftarrow{\$} \{0, 1\}^{256}$ and return it; otherwise return $T(x)$. This is a standard ROM bookkeeping change [41] and induces the same distribution as G_0 , hence $\Pr[\mathcal{A} \text{ wins } G_1] = \varepsilon$.

Game G_2 . Identical to G_1 except that $\mathcal{C}$ aborts and declares loss for $\mathcal{A}$ if any of the $m+1$ proofs in $\mathcal{A}$'s output uses a Fiat–Shamir challenge c such that the corresponding hash query was never issued by $\mathcal{A}$ to H (i.e. $\mathcal{A}$ “guessed” the challenge). For each individual proof, the probability that $\mathcal{A}$ produces a verifying transcript with an unqueried challenge is at most 2^{-256} (the fraction of $\{0, 1\}^{256}$ that satisfies the verification equations conditioned on the prover's first message). Taking a union bound over the $m+1$ proofs,

$$|\Pr[\mathcal{A} \text{ wins } G_2] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq (m+1) \cdot 2^{-256}.$$

Knowledge extraction. In G_2 every accepting proof corresponds to an actual hash query made by $\mathcal{A}$. The extractor $\mathcal{E}$ runs $\mathcal{A}$ once to obtain the ballot and proof transcripts, then for each of the $m+1$ proofs proceeds in two steps:

1. *Forking.* For each proof π with challenge c and corresponding hash query q , $\mathcal{E}$ rewinds $\mathcal{A}$ to q , supplies a fresh challenge $c' \xleftarrow{\$} \{0, 1\}^{256}$, and re-runs $\mathcal{A}$ from that point. By the General Forking Lemma (Bellare–Neven 2006 [42], generalising the original Forking Lemma of Pointcheval and Stern [43]), conditioned on $\mathcal{A}$ winning G_2 , the rewind run yields a verifying transcript on the same first message but distinct challenge $c' \neq c$, with probability at least $\varepsilon^2/q_H - 2^{-256}$.

2. *Witness extraction (unconditional).* For each binary proof π_j^{bin} , the CDS OR-composition [37] applied to the Paillier n -th-residuosity language [38] admits direct rewinding: from two accepting transcripts with the same first message (a_0, a_1) and distinct challenges c_j, c'_j , the extractor solves the linear equation system on the responses to recover $r_j \in \mathbb{Z}_n^*$ satisfying *either* $E_j \equiv r_j^n \pmod{n^2}$ (case $v_j = 0$) *or* $E_j \equiv (1+n)r_j^n \pmod{n^2}$ (case $v_j = 1$). For the sum proof π^{sum} , analogous extraction recovers $R \in \mathbb{Z}_n^*$ with $\prod_j E_j / (1+n) \equiv R^n \pmod{n^2}$, which combined with the per-candidate openings forces $\sum_j v_j = 1$.

Crucially, extraction is unconditional: the algebraic structure of the n -th-residue Schnorr branches admits a direct closed-form recovery of the witness, with no appeal to any number-theoretic hardness assumption. Putting the m binary witnesses and the sum witness together yields $(v_1, \dots, v_m, r_1, \dots, r_m) \in R_{\text{ballot}}$, contradicting $\mathcal{A}$'s winning condition (iii) that no such witness exists.

The probability that forking succeeds for *all* $m+1$ proofs is bounded below by $(m+1) \cdot (\varepsilon^2/q_H - 2^{-256})$ in the optimistic union-bound regime; setting this success probability equal to a target δ and inverting yields

$$\varepsilon \leq (m+1) \sqrt{q_H \cdot 2^{-256}} + \text{negl}(\lambda),$$

absorbing the $(m+1) \cdot 2^{-256}$ guessing term from G_2 into the leading bound. Combining all steps yields Equation (3.34).

Since each term on the right is negligible in λ , we conclude $\text{Adv}_{\mathcal{A}}^{\text{Snd}}(\lambda) = \text{negl}(\lambda)$. $\square$

Remark on the absence of computational hardness. Unlike the Pedersen-based binary proof—whose soundness reduction requires DLog hardness to argue Pedersen binding—the CDS OR-proof on the Paillier n -th-residuosity language admits *unconditional* witness extraction from two accepting transcripts. Theorem 2 therefore holds in the ROM without invoking DCRA, DLog, or any other cryptographic assumption; computational assumptions enter the framework only through ballot *privacy* (Theorem 3.8.1.3, via DCRA), not through vote *validity*. This separation is a structural advantage of the A2 (Helios-style) ZK design.

Remark on Strong Fiat–Shamir. The hash inputs in the construction (Equations (3.13) and (3.15)) include the Paillier modulus n , the per-proof nonce, the election identifier, the candidate index, and the statement E_j (or the full ciphertext vector for π^{sum}), in line with Bernhard, Pereira, and Warinschi [9]. This binding of the statement into the challenge prevents the malleability attack that affected earlier voting schemes: an adversary cannot reuse a proof transcript by changing a ciphertext, because any modification of E_j changes the hash and thereby the required challenge.

3.8.1.5 Theorem 3: Voter Anonymity (DDH)

Informal claim. Given a valid Liu–Wei–Wong linkable ring signature $\sigma = (I, c_0, r_0, \dots, r_{n-1})$ with ring size $|\mathcal{R}| = 100$, no PPT adversary can determine which ring member produced σ with non-negligible advantage over uniform guessing, under DDH in the random-oracle model. Linkability is preserved: two signatures by the same member yield identical key images, enabling double-vote detection without revealing the signer’s identity.

Anonymity experiment. Define $\text{Anon}_{\mathcal{A}}^{\text{LWW}}(\lambda)$ as follows. The challenger $\mathcal{C}$ generates a ring $\mathcal{R} = \{(x_i, g^{x_i})\}_{i=0}^{n-1}$ honestly with $x_i \xleftarrow{\$} \mathbb{Z}_q$ and gives $\mathcal{A}$ only the public ring $\{g^{x_i}\}_i$. $\mathcal{A}$ may issue $\text{Sign}(j, M)$ queries to a signing oracle. $\mathcal{A}$ then outputs two indices i_0, i_1 and a fresh message M^* never previously queried for either index, the challenger samples $b \xleftarrow{\$} \{0, 1\}$ and returns $\sigma^* \leftarrow \text{Sign}(i_b, M^*)$. After further oracle queries (forbidding only (i_0, M^*) and (i_1, M^*)), $\mathcal{A}$ outputs b' and wins if $b' = b$. Define $\text{Adv}_{\mathcal{A}}^{\text{Anon}}(\lambda) = |\Pr[b' = b] - 1/2|$.

Theorem 3 (Voter Anonymity). *In the random-oracle model on H , for any PPT adversary $\mathcal{A}$ making at most q_S signing-oracle queries,*

$$\text{Adv}_{\mathcal{A}}^{\text{Anon}}(\lambda) \leq \text{Adv}_{\mathcal{B}}^{\text{DDH}}(\lambda) + q_S \cdot 2^{-(\lambda - O(\log q_S))}. \quad (3.35)$$

Proof. We decompose the adversary’s view of the challenge signature $\sigma^* = (I^*, c_0^*, r_0^*, \dots, r_{n-1}^*)$ into two parts and bound each.

Part (a): Statistical anonymity of (c_i^, r_i^*) .* For each non-signer $i \neq s$ in the signing procedure, r_i is sampled uniformly from $\mathbb{Z}_q$ (chapter3.tex line 3.4.4.2). For the actual signer s , $r_s = \alpha - c_s \cdot \text{sk}_s \bmod q$ where α is uniform on $\mathbb{Z}_q$; hence r_s is also uniform on $\mathbb{Z}_q$, regardless of sk_s . The c_i values are random-oracle outputs and therefore uniform on the challenge space. So the joint distribution of $(c_0^*, r_0^*, \dots, r_{n-1}^*)$ is independent of the signer index b , contributing zero statistical advantage. (This matches Liu–Wei–Wong 2004 S4.2 Lemma 4 [10].)

Part (b): The key image I^ requires DDH.* The remaining b -dependent component of σ^* is the key image $I^* = H(\text{pk}_{i_b})^{\text{sk}_{i_b}}$. In the random-oracle model, $H(\text{pk}_i)$ is a fresh uniformly-random group element $h_i \in \mathbb{G}_q$. The triple (pk_i, h_i, I^*) is a Diffie–Hellman triple if and only if $i = i_b$ (the actual signer). We construct a DDH adversary $\mathcal{B}$:

1. $\mathcal{B}$ receives a DDH challenge (g^a, g^b, T) where $T = g^{ab}$ (case $b_{\text{DDH}} = 1$) or $T \xleftarrow{\$} \mathbb{G}_q$ (case $b_{\text{DDH}} = 0$).
2. $\mathcal{B}$ sets the ring so that $\text{pk}_{i_0} = g^a$ (treating $\text{sk}_{i_0} = a$ as unknown). The other public keys are generated honestly.
3. $\mathcal{B}$ programs the random oracle so that $H(\text{pk}_{i_0}) = g^b$ at the first query.

4. For the challenge query, $\mathcal{B}$ sets $I^* = T$ and generates the remaining signature components $(c_0^*, r_0^*, \dots, r_{n-1}^*)$ via the LWW simulator (sampling each r_i uniformly, deriving c_i from the random oracle, and closing the ring). Because Part (a) shows these values are statistically uniform regardless of signer, the simulation is exact.
5. For Sign-oracle queries on $\text{Sign}(j, M)$ with $j \neq i_0$, $\mathcal{B}$ has sk_j and signs honestly. For $\text{Sign}(i_0, M)$ queries (other than the challenge), $\mathcal{B}$ programs H to produce a value $g^{r'}$ for a fresh r' it knows, setting $I = (g^a)^{r'} = g^{ar'}$, which is consistent. (Standard ROM programming; collision probability with g^b is $\leq q_S / |\mathbb{G}_q|$, giving the $q_S \cdot 2^{-(\lambda - O(\log q_S))}$ slack in Equation (3.35).)
6. $\mathcal{B}$ outputs $b'_{\text{DDH}} = 1$ if $\mathcal{A}$'s guess is $b = 0$ (corresponding to signer i_0), else $b'_{\text{DDH}} = 0$.

When $T = g^{ab}$, the key image $I^* = H(\text{pk}_{i_0})^{\text{sk}_{i_0}}$ matches a real signature by i_0 , so $\mathcal{A}$'s view is exactly that of $b = 0$. When T is random, the key image is uniformly random and statistically indistinguishable from the key image of a random ring member; $\mathcal{A}$'s view collapses to one with no signer-dependent information. Hence $\text{Adv}_{\mathcal{B}}^{\text{DDH}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{Anon}}(\lambda)$, modulo the ROM programming slack. $\square$

Linkability is preserved. Linkability is a deterministic property of the construction: the key image $I = H(\text{pk}_s)^{\text{sk}_s}$ depends only on the signer's identity, so two signatures by the same member produce identical key images, allowing the bulletin board to detect duplicate casts. Theorem 3 establishes that knowing I does not reveal s ; hence linkability and anonymity coexist without conflict.

Note on the original “statistically indistinguishable” claim. The thesis's earlier proof sketch referred to “statistical indistinguishability among ring members.” The formal treatment above shows that this is precise for the (c_i, r_i) component (Part a) but only *computational* under DDH for the key image (Part b). The combined advantage bound in Equation (3.35) reflects this distinction.

3.8.1.6 Theorem 4: Coercion Resistance (Pedersen Perfect Hiding + JCJ)

Informal claim. A coercer observing a voter's SMDC credentials and her bulletin-board cast cannot determine whether the slot used was the real slot (contributing $w = 1$ to the tally) or a fake slot (contributing $w = 0$ and silently discarded). The construction is therefore deniable: the voter can comply with coercion at the surface while ensuring her actual vote, made via the real slot j^* , remains private to her.

Threat-model scope. The formal experiment below covers *slot indistinguishability*: an adversary cannot determine which slot is real from the bulletin-board view plus the credential transcript. The following coercion attacks are explicitly out of scope (consistent with the SMDC system-design treatment in Section 3.4.5):

- *Forced abstention*: a coercer who prevents the voter from casting at all.
- *Randomization attacks*: a coercer who demands a random ballot rather than a specific candidate.
- *Device-controlling coercer*: an adversary who observes the cast at the device level rather than the bulletin board.
- *Credential-issuance compromise*: a coercer present during credential issuance (see Assumption 5 of Section 3.2).

These limitations mirror the standard demarcation in the coercion-resistance literature [17].

Coercion experiment. Define $\text{Coerc}_A^{\text{SMDC}}(\lambda)$ as follows. The challenger $\mathcal{C}$ generates Pedersen parameters and an HMAC server key k_{srv} . For the target voter, $\mathcal{C}$ derives the real slot

$$j^* = \text{HMAC-SHA256}(k_{\text{srv}}, \text{"smdc-real-index:"} \parallel \text{vid} \parallel \text{eid}) \bmod k,$$

with $k = 5$ throughout, and constructs the k commitments $\{C_j\}_{j=0}^{k-1}$ with $C_{j^*} = g^1 h^{r_{j^*}}$ and $C_j = g^0 h^{r_j}$ for $j \neq j^*$, plus binary proofs $\{\pi_j^{\text{bin}}\}$ and the sum proof π^{sum} . $\mathcal{A}$ receives all k commitments and proofs. $\mathcal{A}$ then outputs two distinct slot indices j_0, j_1 (without knowing j^*). $\mathcal{C}$ samples $b \xleftarrow{\$} \{0, 1\}$ and emits the bulletin-board cast transcript using slot j_b (commitment C_{j_b} , simulated proof, and the auxiliary ballot components produced as in Theorem 1). $\mathcal{A}$ outputs b' and wins if $b' = b$. Define $\text{Adv}_A^{\text{Coerc}}(\lambda) = |\Pr[b' = b] - 1/2|$.

Theorem 4 (SMDC Coercion Resistance, Slot Indistinguishability). *For any adversary $\mathcal{A}$ that is unbounded in the Pedersen-commitment component and PPT elsewhere,*

$$\text{Adv}_A^{\text{Coerc}}(\lambda) \leq \text{Adv}_{\mathcal{B}_{\text{ZK}}}^{\text{ZK}}(\lambda) + \text{Adv}_{\mathcal{B}_{\text{HMAC}}}^{\text{HMAC-PRF}}(\lambda) + \text{negl}(\lambda). \quad (3.36)$$

In particular, when $\mathcal{A}$ is restricted to PPT and HMAC-SHA256 satisfies the PRF assumption, $\text{Adv}_A^{\text{Coerc}}(\lambda) = \text{negl}(\lambda)$.

Proof. We proceed by hybrids $G_0 \rightarrow G_2$.

Game G_0 . The real coercion experiment as defined above. $\Pr[\mathcal{A} \text{ wins } G_0] = 1/2 + \varepsilon$ where $\varepsilon = \text{Adv}_A^{\text{Coerc}}(\lambda)$.

Game G_1 (replace HMAC-derived j^ with uniform).* $\mathcal{C}$ samples $j^* \xleftarrow{\$} \{0, 1, \dots, k-1\}$ uniformly at random, ignoring the HMAC derivation. By the PRF property of HMAC-SHA256 [31, 32] (under Assumption 6 of Section 3.2, namely that k_{srv} remains unknown to $\mathcal{A}$), the HMAC-derived index is computationally indistinguishable from uniform. Bound:

$$|\Pr[\mathcal{A} \text{ wins } G_1] - \Pr[\mathcal{A} \text{ wins } G_0]| \leq \text{Adv}_{\mathcal{B}_{\text{HMAC}}}^{\text{HMAC-PRF}}(\lambda).$$

Game G_2 (simulate ZK proofs). $\mathcal{C}$ replaces π_j^{bin} and π^{sum} in the credential view (and the cast transcript) with simulator outputs that do not use the witnesses. Under Strong Fiat–Shamir in the random-oracle model [9, 37], real and simulated proofs are computationally indistinguishable. Bound:

$$|\Pr[\mathcal{A} \text{ wins } G_2] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \text{Adv}_{\mathcal{B}_{\text{ZK}}}^{\text{ZK}}(\lambda).$$

After G_2 , the proofs in $\mathcal{A}$'s view carry no information about the witnesses $\{w_j\}$.

Pedersen perfect-hiding step. In G_2 , the only b -dependent component of $\mathcal{A}$'s view is the sequence of slot commitments $\{C_j\}$ together with the cast commitment C_{j_b} . Each $C_j = g^{w_j} h^{r_j}$ with $r_j \xleftarrow{\$} \mathbb{Z}_q$ uniform; by the perfect-hiding property of Pedersen commitments (Section 3.4.2), C_j is uniformly distributed over the order- q subgroup independently of $w_j \in \{0, 1\}$. Hence C_{j_0} and C_{j_1} have identical statistical distributions regardless of which slot is real, and the auxiliary bulletin-board components are simulated as in Theorem 1's hybrid sequence (independent of b). Therefore $\Pr[\mathcal{A} \text{ wins } G_2] = 1/2$ exactly, and combining the hybrid bounds yields Equation (3.36). $\square$

Modulo- k bias remark. The reduction modulo k of a uniformly distributed 256-bit HMAC output is not perfectly uniform when k does not divide 2^{256} . For $k = 5$ the statistical distance from uniform is bounded by $k/2^{256} < 2^{-253}$, which is dominated by every term in Equation (3.36) and is therefore absorbed into the $\text{negl}(\lambda)$ component.

Silent-discard semantics. The deniability guaranteed by Theorem 4 relies on the silent-discard property of the homomorphic tally: a coerced vote made via a fake slot contributes $w = 0$ via per-candidate Paillier scalar multiplication ($E_j^0 = E(0)$ for every j), so the cast contributes nothing to the tally. The tally outcome is therefore identical whether the voter complied with coercion (used j^*) or cheated (used a fake slot, then later cast for her true preference via j^*). Both transcripts are bulletin-board indistinguishable to the coercer.

3.8.1.7 Theorem 5: SA² Aggregation Privacy (DCRA + A1)

Informal claim. The two-server SA² split (Section 3.4.6) ensures that, as long as at least one of the two aggregation servers is honest (Trust Assumption A1), no single corrupted server learns any non-trivial information about an individual vote v . The shares $S_A = E(v + \text{mask})$ and $S_B = E(-\text{mask})$ are computationally indistinguishable from encryptions of arbitrary values; only their product, $S_A \cdot S_B = E(v)$, recovers the encrypted vote, and even that does not reveal v until the threshold-Paillier trustees decrypt the aggregated tally.

Single-server-view game. Define $\text{SA2-Priv}_{\mathcal{A}}^{\sigma}(\lambda)$ for a corrupted server $\sigma \in \{A, B\}$ as follows. The challenger $\mathcal{C}$ generates Paillier keys $(\text{pk}_p, \text{sk}_p)$ and gives $\mathcal{A}$ only pk_p . $\mathcal{A}$ outputs two votes $v_0, v_1 \in \mathbb{Z}_n$. The challenger samples $b \xleftarrow{\$} \{0, 1\}$ and mask $\xleftarrow{\$} \mathbb{Z}_{n/100}$, computes the split (S_A^*, S_B^*) for v_b , and gives $\mathcal{A}$ only the σ -server's share. $\mathcal{A}$ outputs a guess b' and wins if $b' = b$. Define $\text{Adv}_{\mathcal{A}}^{\text{SA2-Priv}, \sigma}(\lambda) = |\Pr[b' = b] - 1/2|$.

Theorem 5 (SA² Single-Server Privacy). *Let $\sigma \in \{A, B\}$ index a single corrupted server. For any PPT adversary $\mathcal{A}$,*

$$\text{Adv}_{\mathcal{A}}^{\text{SA2-Priv}, \sigma}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{Paillier}}^{\text{IND-CPA}}(\lambda) \leq \text{Adv}_{\mathcal{B}'}^{\text{DCRA}}(\lambda) + \text{negl}(\lambda). \quad (3.37)$$

Proof. We treat the case $\sigma = A$; the case $\sigma = B$ is strictly easier (the share $S_B = E(-\text{mask})$ is independent of v_b , giving zero advantage). The reduction $\mathcal{B}$ to Paillier IND-CPA is direct:

1. $\mathcal{B}$ receives pk_p from the Paillier IND-CPA challenger and forwards it to $\mathcal{A}$.
2. $\mathcal{A}$ outputs (v_0, v_1) .
3. $\mathcal{B}$ samples mask $\xleftarrow{\$} \mathbb{Z}_{n/100}$, computes $m_0 = v_0 + \text{mask} \bmod n$ and $m_1 = v_1 + \text{mask} \bmod n$, and submits (m_0, m_1) to the IND-CPA challenger.
4. $\mathcal{B}$ receives ciphertext $c^* = E(m_b)$ from the challenger (for the challenger's hidden bit b) and forwards c^* to $\mathcal{A}$ as the simulated S_A^* .
5. $\mathcal{B}$ outputs $\mathcal{A}$'s guess b' as its own.

Since the same mask is added to both v_0 and v_1 , the value $E(m_b) = E(v_b + \text{mask})$ has exactly the distribution of S_A^* in the real SA2-Priv^A experiment. Hence $\text{Adv}_{\mathcal{B}, \text{Paillier}}^{\text{IND-CPA}}(\lambda) = \text{Adv}_{\mathcal{A}}^{\text{SA2-Priv}, A}(\lambda)$. The Paillier semantic-security theorem (Paillier 1999, Theorem 15 [8]) reduces Paillier IND-CPA to DCRA with negligible loss, completing the bound in Equation (3.37). $\square$

Per-candidate hybrid extension. The experiment above analyses one candidate position j , with $v \in \{0, 1\}$ standing in for v_j and mask for m_j . A full 1-hot ballot has m such positions (Section 3.4.3); the same bound extends to the full m -position vector by a standard hybrid argument over candidates, multiplying the right-hand side of Equation (3.37) by m in the worst case.

Mask-range remark. The mask is sampled from $\mathbb{Z}_{n/V_{\max}}$ (rather than $\mathbb{Z}_n$) to prevent additive overflow when up to $V_{\max}$ voters' shares are aggregated by a single server; the current deployment uses $V_{\max} = 100$. With this range, $\sum_i (v_{j,i} + m_{j,i})$ stays within $\mathbb{Z}_n$ and the additive homomorphism does not wrap. The smaller mask range does *not* weaken the indistinguishability bound above: the IND-CPA reduction relies only on Paillier semantic

security, not on statistical hiding by the mask. (Statistical hiding at the plaintext level would require mask $\xleftarrow{\$} \mathbb{Z}_n$ and is not invoked anywhere in the construction, since both shares remain Paillier-encrypted and only the threshold-Paillier trustees decrypt the final aggregate.)

Multi-voter extension. For a corrupted server observing q ballots from q different voters, a standard hybrid argument extends the bound to $\text{Adv} \leq q \cdot \text{Adv}_{\mathcal{B}, \text{Paillier}}^{\text{IND-CPA}}(\lambda)$, which remains negligible under DCRA for any polynomially-bounded q . Combined with the per-candidate hybrid above, the cumulative loss against a server observing q ballots over m candidates is $q \cdot m \cdot \text{Adv}_{\mathcal{B}, \text{Paillier}}^{\text{IND-CPA}}(\lambda)$.

3.8.1.8 Theorem 6: Post-Quantum Hybrid (Module-LWE + DCRA)

Informal claim. The Kyber-768 hybrid layer (Section 3.4.7) provides ciphertext integrity for the transmitted ballot tuple $(\text{ct}_{\text{ky}}, E(v), \text{salt}, \text{MAC})$: any tampering with $E(v)$, ct_{ky} , or salt in transit or at rest is detected by MAC failure with overwhelming probability. Confidentiality of the vote v inherits from Theorem 1 (Paillier DCRA). Both properties remain classical against a present-day adversary; the integrity layer additionally remains secure against a future quantum adversary that would solve Module-LWE only after the ballot has been recorded (*harvest-now-decrypt-later*).

Hybrid construction recap. For each vote, the sender produces $(\text{ct}_{\text{ky}}, \mathbf{E}, \text{salt}, \text{MAC})$ where $\mathbf{E} = (E_1, \dots, E_m)$ is the 1-hot per-candidate Paillier ciphertext vector, $(K, \text{ct}_{\text{ky}}) \leftarrow \text{KEM.Encap}(\text{pk}_{\text{ky}})$, $K_{\text{auth}} = \text{SHA256}(K \parallel \text{salt})$, and $\text{MAC} = \text{HMAC-SHA256}(K_{\text{auth}}, \text{enc}(\mathbf{E}))$ with $\text{enc}(\cdot)$ a canonical serialisation. Verification recomputes K via $\text{KEM.Decap}(\text{sk}_{\text{ky}}, \text{ct}_{\text{ky}})$, derives K_{auth} , and checks the MAC. For brevity the proof below writes E^* for a generic ciphertext component of $\mathbf{E}$ (the same bound applies to each E_j independently).

Kyber-768 vs. ML-KEM-768. *Kyber-768* denotes the CRYSTALS-Kyber round-3 NIST PQC submission [12]; *ML-KEM-768* is its standardised form under NIST FIPS 203. The two are the same algorithm up to minor parameter packing and serialisation refinements. The runtime implementation invoked by this thesis is the round-3 Kyber-768 variant provided by Cloudflare’s CIRCL library (github.com/cloudflare/circl/kem/kyber/kyber768); migration to the FIPS-203 ML-KEM-768 implementation in the same library is straightforward and is identified as a near-term engineering follow-up. The reductions cited below apply unchanged to either variant because both reduce IND-CCA2 security to the same Module-LWE / Module-SIS hardness assumption.

Integrity experiment. Define $\text{INT-CTXT}_{\mathcal{A}}^{\text{Hybrid}}(\lambda)$ as follows. The challenger $\mathcal{C}$ generates Kyber-768 keys $(\text{pk}_{\text{ky}}, \text{sk}_{\text{ky}})$ and Paillier keys $(\text{pk}_p, \text{sk}_p)$ and gives $\mathcal{A}$ both public keys. $\mathcal{A}$ adaptively submits up to q vote queries to an encryption oracle $\text{Enc}(\cdot)$, which returns the corresponding honest ballot tuples. Let $\mathcal{S}$ denote the set of returned tuples. $\mathcal{A}$ outputs a candidate forgery $\text{Ballot}^* = (\text{ct}_{\text{ky}}^*, E^*, \text{salt}^*, \text{MAC}^*)$ and *wins* if $\text{Ballot}^* \notin \mathcal{S}$ and Ballot^* verifies under sk_{ky} . Define $\text{Adv}_{\mathcal{A}}^{\text{INT-CTXT}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$.

Theorem 6 (Hybrid Integrity). *For any PPT adversary $\mathcal{A}$ making at most q queries to the encryption oracle,*

$$\text{Adv}_{\mathcal{A}}^{\text{INT-CTXT}}(\lambda) \leq q \cdot \text{Adv}_{\mathcal{B}_1, \text{Kyber}}^{\text{IND-CCA2}}(\lambda) + \text{Adv}_{\mathcal{B}_2, \text{HMAC}}^{\text{EUF-CMA}}(\lambda) + \text{negl}(\lambda), \quad (3.38)$$

where $\mathcal{B}_1$ is a PPT adversary against the IND-CCA2 security of ML-KEM-768 (which reduces to Module-LWE and Module-SIS via the Fujisaki–Okamoto transform [44, 12]) and $\mathcal{B}_2$ is a PPT adversary against the EUF-CMA security of HMAC-SHA256 (which reduces to the PRF property of SHA-256 [31, 32]).

Proof. We proceed by hybrids $G_0 \rightarrow G_1 \rightarrow G_2$, following the standard KEM–DEM analysis of Cramer and Shoup [45] adapted to Encrypt-then-MAC composition.

Game G_0 . The real INT-CTXT experiment. By definition $\Pr[\mathcal{A} \text{ wins } G_0] = \text{Adv}_{\mathcal{A}}^{\text{INT-CTXT}}(\lambda)$.

Game G_1 . Identical to G_0 , except $\mathcal{C}$ aborts whenever the forgery’s ct_{ky}^* decapsulates to a Kyber session key K^* that equals some oracle-derived K_i while $\text{ct}_{\text{ky}}^* \neq \text{ct}_{\text{ky},i}$. By the IND-CCA2 security of ML-KEM-768, encapsulated keys are pseudorandom and uniformly distributed conditioned on distinct ciphertexts; the per-query collision probability is bounded by $\text{Adv}_{\mathcal{B}_1, \text{Kyber}}^{\text{IND-CCA2}}(\lambda)$ (otherwise $\mathcal{B}_1$ could distinguish a real encapsulated key from random by checking equality with oracle-returned keys). Union bound over q encryption queries:

$$|\Pr[\mathcal{A} \text{ wins } G_1] - \Pr[\mathcal{A} \text{ wins } G_0]| \leq q \cdot \text{Adv}_{\mathcal{B}_1, \text{Kyber}}^{\text{IND-CCA2}}(\lambda).$$

Game G_2 . Identical to G_1 , except $\mathcal{C}$ aborts if $\mathcal{A}$ ’s forgery uses a derived MAC key $K_{\text{auth}}^* = \text{SHA256}(K^* \parallel \text{salt}^*)$ that was *not* used by any oracle query. To win in this case $\mathcal{A}$ must produce a valid HMAC-SHA256 tag under a key it has not observed any tags under, which is precisely the EUF-CMA experiment for HMAC. Hence

$$|\Pr[\mathcal{A} \text{ wins } G_2] - \Pr[\mathcal{A} \text{ wins } G_1]| \leq \text{Adv}_{\mathcal{B}_2, \text{HMAC}}^{\text{EUF-CMA}}(\lambda).$$

We model SHA-256 as a random oracle for the key-derivation step $K_{\text{auth}} = \text{SHA256}(K \parallel \text{salt})$; under this assumption, distinct (K, salt) pairs yield independent uniformly random K_{auth} values, so reuse of K_{auth} across encryption queries occurs with negligible probability and is absorbed into the $\text{negl}(\lambda)$ term.

In G_2 , the only remaining win path requires $\mathcal{A}$ ’s forgery to reuse some K_{auth} from a prior oracle query. By construction, the oracle-supplied MAC_i uniquely binds $E(v_i)$ under $K_{\text{auth},i}$; producing a different $E^* \neq E(v_i)$ but valid MAC^* under $K_{\text{auth},i}$ is again an

EUF-CMA forgery against HMAC under the previously-revealed key, which is bounded as in $G_1 \rightarrow G_2$. The “ballot Ballot^* exactly equals some Ballot_i ” case is excluded by the win condition. Therefore $\Pr[\mathcal{A} \text{ wins } G_2] \leq \text{negl}(\lambda)$, and combining the hybrid steps yields Equation (3.38). $\square$

Confidentiality (P1). Confidentiality of the vote vector $\mathbf{v} = (v_1, \dots, v_m)$ inside the hybrid tuple inherits directly from Theorem 1: the Kyber/ML-KEM ciphertext ct_{ky} is independent of $\mathbf{v}$ (the KEM operates only on the random session key K), and the MAC is a deterministic function of $\mathbf{E}$ under a key independent of $\mathbf{v}$. Hence any IND-CPA distinguisher for the hybrid tuple yields an IND-CPA distinguisher against Paillier (applied component-wise to the 1-hot vector) with the same advantage, which by Theorem 1 reduces to DCRA.

Quantum-adversary scope. The integrity layer remains secure against a future polynomial-time quantum adversary as long as Module-LWE remains hard against quantum algorithms. The confidentiality layer (Paillier under DCRA) does *not* survive a quantum adversary, since Shor’s algorithm [15] solves integer factorisation. The thesis therefore offers post-quantum integrity (preventing future tampering of stored ballots) but classical confidentiality (the harvest-now-decrypt-later threat against v remains, mitigated by the limited utility of decrypting individual ballots after the public tally). A fully post-quantum migration, replacing Paillier with a lattice-based homomorphic encryption such as BGV or CKKS, and replacing Liu-Wei-Wong ring signatures with lattice-based ring signatures, is identified as future work in Chapter 5.

Auxiliary AES-256-GCM use. Generic non-vote payloads (audit metadata, session traffic) are additionally protected by AES-256-GCM [33] authenticated symmetric encryption; this is outside the scope of the INT-CTXT game above, which targets the vote-ballot tuple specifically.

3.8.1.9 Composition Discussion: Toward UC Security

Why this discussion is informal. The six theorems above (Theorems 3.8.1.3–3.8.1.8) establish per-protocol security under a stand-alone game-based model. Universally Composable (UC) security [40] would additionally guarantee that the multi-protocol pipeline remains secure under arbitrary concurrent composition with other protocols, but constructing a full UC proof for a system of this scale is itself a research-scale undertaking (typically 15–50 pages per protocol primitive in dedicated papers). The present thesis therefore offers an informal sketch of the UC-style ideal functionalities and the composition argument structure, and identifies a complete UC formalization as future work in Chapter 5, building on the recent UC-secure self-tallying construction of E-cclesia [46].

Ideal functionalities sketch. The future UC proof would target three principal ideal functionalities:

$\mathcal{F}_{\text{Ballot}}$ (**Idealized Ballot Privacy**). Receives a vote $\mathbf{v}_i$ from voter V_i ; outputs only a bulletin-board placeholder “ballot i posted by V_i ” to the adversary, and the aggregate tally $\mathbf{T} = \sum_i \mathbf{v}_i$ to the trustees after election close. Theorem 3.8.1.3 provides the per-ballot indistinguishability bound that a UC simulator would invoke; full UC additionally requires adaptive-corruption simulator-extractability, which the threshold-Paillier decryption (Section 3.4.8) supports.

$\mathcal{F}_{\text{SMDC}}$ (**Idealized Coercion Resistance**). Receives a vote and a coercion-mode flag from each voter; outputs a simulator-generated transcript indistinguishable from the real bulletin-board view, with coerced votes silently dropped from the tally. Theorem 3.8.1.6 provides the slot-indistinguishability bound; the perfect-hiding step gives statistical (adaptive-secure) indistinguishability for the commitment component.

$\mathcal{F}_{\text{SA2}}$ (**Idealized Two-Server Aggregation**). Receives encrypted votes from all voters; outputs simulator-generated shares to each server (statistically close to real shares but information-theoretically independent of individual votes); outputs the homomorphic aggregate to threshold trustees. Theorem 3.8.1.7 provides the per-server view indistinguishability bound; Trust Assumption A1 (one honest server) is the foundation for UC simulation.

Informal composition argument. For sequential composition under a top-level distinguisher $\mathcal{Z}$, a standard hybrid argument bounds the joint advantage by the sum of per-theorem advantages:

$$\text{Adv}_{\mathcal{Z}}^{\text{Joint}}(\lambda) \leq \sum_{i=1}^6 \text{Adv}_{B_i}^{T_i}(\lambda) + O(\text{negl}(\lambda)).$$

Each $\text{Adv}_{B_i}^{T_i}$ is negligible by the corresponding theorem, so the sequential joint advantage is negligible. UC concurrent composition under arbitrary scheduling is strictly stronger and is not established by this sketch; it would require constructing UC simulators for each ideal functionality and invoking Canetti’s UC composition theorem [40, Theorem 13].

Limitations of this discussion (explicitly). (i) The argument above is sequential, not concurrent. (ii) The simulators sketched are non-adaptive; the corruption set is fixed at the start. (iii) The ideal functionalities are informally stated, full UC formalization requires precise message scheduling and corruption interfaces. (iv) No machine-checked proof (e.g., EasyCrypt or CryptoVerif) is provided. The thesis intentionally separates Tier 1 (game-based reductions, this section) from Tier 2 (symbolic verification, Section 4.7) and from a future UC layer; readers are referred to the corresponding limitations subsection in Chapter 5.

3.8.2 Duress Detection: Security Properties

The behavioural duress detection construction defined in Section 3.4.9 (Algorithm 1) contributes two security guarantees that complete the coercion-resistance argument of Theorem 4. *Timing indistinguishability*: the constant-time comparison `CONSTANTTIME-COMPARE` (Algorithm 1, Line 7) prevents an HMAC-oracle timing side channel, so an adversary observing the server’s response time cannot distinguish whether the presented hash matches the stored hash. Combined with the silent-discard short-circuit at Steps 17a–17b of Algorithm 2, which executes the full ring-signing and SA^2 share-split phases before suppressing the key-image registration, the end-to-end receipt is indistinguishable in latency, format, and content across the genuine and coerced execution paths. *Database-compromise resistance*: because only the HMAC keyed by the server secret K_d is persisted and the raw signal value is never written to disk, an attacker who compromises the voter database recovers a hash but cannot invert it to obtain a usable duress signal; this reduces secret-signal recovery to inversion of HMAC-SHA256, which is computationally infeasible under standard PRF assumptions on the underlying compression function. Together, these two properties imply that an external coercer who observes both the network traffic and the voter’s receipt cannot determine whether the voter submitted the genuine signal, the duress signal, or no signal at all.

3.9 Conclusion

This chapter presented the complete system design of the proposed framework: the mathematical formulation of the multi-protocol cryptographic stack (Section 3.4), the twenty-step vote-casting pipeline (Section 3.5, Algorithm 2), the Hyperledger Fabric blockchain integration (Section 3.6), and the engineering, sustainability, accessibility, ethical, and privacy considerations of an at-scale deployment (Section 3.7).

The security analysis (Section 3.8) gave six game-based reductions to standard cryptographic hardness assumptions: ballot indistinguishability (Theorem 1, DCRA), vote validity (Theorem 2, soundness of CDS OR-proofs in the ROM), voter anonymity (Theorem 3, DDH), coercion resistance (Theorem 4, Pedersen perfect hiding combined with JCJ-style slot indistinguishability), SA^2 aggregation privacy (Theorem 5, single-server reduction to DCRA), and hybrid post-quantum integrity (Theorem 6, Module-LWE for tamper-evidence on top of Paillier’s classical confidentiality). The proofs are computational and game-based; formal machine-checked verification (e.g., in EasyCrypt or CryptoVerif) and a full Universal Composability treatment are left as future work.

The genuinely novel cryptographic constructions of this thesis are: (i) the SMDC coercion-resistance mechanism with $k = 5$ deniable credential slots (Section 3.4.5); (ii) the behavioural duress-detection mechanism that uses HMAC-SHA256 equality compare over a server-keyed signal hash (Section 3.4.9, Algorithm 1); and (iii) the silent-discard coercion

short-circuit semantics (Algorithm 2, Steps 17a–17b), which preserves bulletin-board indistinguishability between the genuine and coerced execution paths. The remaining cryptographic primitives in the pipeline—Paillier, Pedersen, Σ -protocols, linkable ring signatures, SA^2 , Kyber-768/ML-KEM-768, and DJN threshold decryption—are existing constructions whose composition into a unified voting pipeline constitutes the engineering contribution of this work.

The next chapter presents the implementation details and the comprehensive performance evaluation of the system, including per-primitive microbenchmarks, end-to-end election-scale simulations, and Hyperledger Caliper saturation and sustained-load runs.

4. Implementation, Results and Discussion

4.1 Introduction

This chapter presents the implementation details, detailed performance evaluation, and in-depth discussion of the proposed system. Section 4.2 describes the implementation environment and technology stack. Section 4.3 defines the performance metrics used for evaluation. Section 4.4 presents the experimental results including cryptographic micro-benchmarks, pipeline performance, scalability validation, and blockchain throughput measurements. Section 4.5 provides analysis and discussion of the results. Section 4.6 analyzes the test coverage. Section 4.7 presents formal verification of three security properties using ProVerif. Section 4.8 provides a comparative analysis with existing systems.

4.2 Implementation Environment

The proposed system is implemented in Go 1.24 and deployed on Hyperledger Fabric v2.5.15 [1]. Table 4.1 summarizes the implementation environment.

Table 4.1: Implementation Environment

Component	Specification
CPU	AMD Ryzen 5 7530U (12 threads)
Architecture	x86_64 (amd64)
RAM	30 GiB
OS	Linux (Ubuntu)
Language	Go 1.24
Web Framework	Gin v1.11
Blockchain	Hyperledger Fabric v2.5.15 (LTS)
SDK	fabric-gateway v1.10.1 (gRPC/TLS)
Consensus	Raft / etcdraft (single orderer, dev config)
Peers	2 (Org1 + Org2), LevelDB state (default; CouchDB-target for production)
Chaincode	covertvote_1.0 (Go)
Benchmark Tool	Hyperledger Caliper v0.6.0 [47]
PQ Library	Kyber-768, CRYSTALS-Kyber round-3 (CIRCL by Cloudflare) [48]

The system architecture follows a layered design with twelve internal packages that align with the chapter’s pipeline phases: cryptographic primitives (`internal/crypto`, `internal/smdc`, `internal/sa2`, `internal/pq`, `internal/tally`), the behavioural duress detector (`internal/biometric`), persistent state and repositories (`internal/database`, `internal/repository`), the election and voter lifecycles (`internal/election`, `internal/voter`), the production cast pipeline (`internal/voting` — the Go implementation of Algorithm 2), and the Fabric client (`internal/blockchain`). These are exposed through the REST API in `api/handlers` and the on-chain logic in `chaincode/covertvote`, with auxiliary tools in `cmd/` (API server, aggregator microservices, election simulator). The total codebase consists of approximately 20,000 lines of Go code across 89 source files.

4.3 Performance Metrics

The following metrics are used to evaluate the proposed framework’s performance:

- **Per-Operation Latency:** Time to execute individual cryptographic operations (encryption, commitment, proof generation, signing), measured in microseconds (μs) or milliseconds (ms).
- **Pipeline Throughput:** End-to-end time for the complete twenty-step vote casting pipeline, measured in milliseconds per vote.

- **Tally Complexity:** Per-vote homomorphic addition time as voter count scales from 100 to 100,000 (validating $O(n)$ per candidate), and total tally time as candidate count m varies (validating $O(n \cdot m)$ total).
- **Blockchain Throughput:** Transactions per second (TPS) sustained on the Hyperledger Fabric network, measured using Hyperledger Caliper with 10 concurrent workers.
- **Blockchain Latency:** Average, minimum, and maximum transaction confirmation time on the Fabric network.
- **Test Coverage:** Percentage of code exercised by the test suite for each package.

4.4 Experimental Results

4.4.1 Cryptographic Micro-Benchmarks

Table 4.2 presents the micro-benchmark results for individual cryptographic operations. Each measurement was obtained using Go’s `testing.B` benchmark framework, and the reported values are the mean of thirty independent benchmark sessions.

Table 4.2: Cryptographic Micro-Benchmark Results

Operation	Mean Time	Memory (B/op)	Allocs/op
Paillier Encrypt (2048-bit)	8.69 ms	26,210	31
Pedersen Commit (512-bit modulus, benchmark config)	0.08 ms	3,962	31
ZKP Binary Prove	0.245 ms	12,542	102
ZKP Binary Verify	0.329 ms	16,392	123
ZKP Sum-One Prove	0.084 ms	5,770	53
ZKP Sum-One Verify	0.161 ms	10,037	82
Kyber-768 KeyGen	0.026 ms	8,304	6
Kyber-768 Encapsulate	0.029 ms	1,232	3
Hybrid Encrypt (Kyber+Paillier)	14.04 ms	33,306	50
Hybrid Decrypt (Kyber+Paillier)	11.60 ms	27,090	38
Threshold Partial Decrypt	35.52 ms	52,500	67
Threshold Combine ($T=5, t=3$, see §3.4.8)	0.261 ms	46,600	104

The results show that Pedersen commitments and ZK proofs are computationally inexpensive (sub-millisecond), while Paillier encryption (8.69 ms) and ring signatures dominate the per-vote cost. The Pedersen figure here is for the 512-bit benchmark configuration used by the test harness; the production API server uses a 1024-bit Pedersen modulus (`cfg.Crypto.PedersenKeySize = 1024`), which raises the commit cost roughly four-fold while remaining well below the Paillier and ring-signature costs that dominate

the pipeline. The Kyber-768 primitive operations themselves are remarkably fast (KeyGen $26 \mu\text{s}$, Encapsulate $29 \mu\text{s}$), although the full hybrid encrypt path including key derivation and authentication binding adds 5.35 ms over the Paillier baseline; the next subsection isolates and analyses this post-quantum cost. Figure 4.1 plots the per-operation cost on a logarithmic scale, making the three-orders-of-magnitude span between Kyber-768 primitives ($\sim 0.03 \text{ ms}$) and threshold partial decryption ($\sim 35 \text{ ms}$) visually evident.

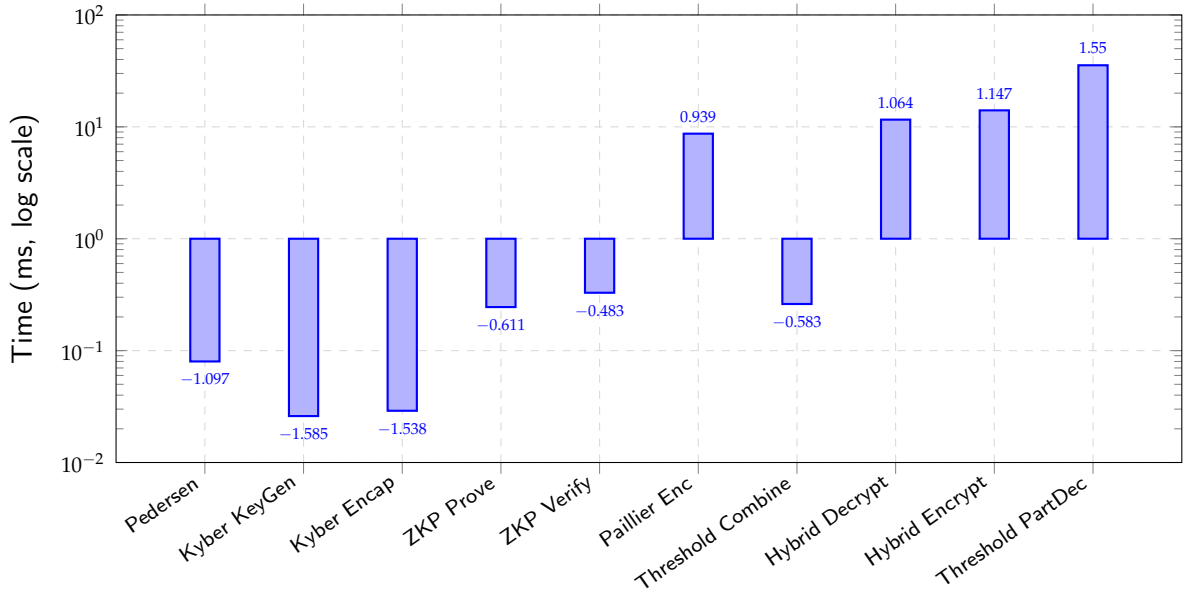


Fig. 4.1: Per-operation cost of every cryptographic primitive in the proposed framework, plotted on a logarithmic time axis. Values are taken directly from Table 4.2; the three-orders-of-magnitude span justifies the log-scale presentation.

4.4.1.1 Post-Quantum Layer Cost Analysis

The overhead of the post-quantum layer can be isolated directly from the measurements already reported in Table 4.2. The full hybrid-encrypt path (Kyber-768 encapsulation + Paillier encryption + HMAC-SHA256 binding + key-derivation + serialisation) measures 14.04 ms ; subtracting the standalone 8.69 ms Paillier baseline gives a hybrid-wrapper overhead of $14.04 - 8.69 = 5.35 \text{ ms}$. This 5.35 ms decomposes into two distinct contributions:

- **Pure Kyber-768 primitive cost:** $\approx 55 \mu\text{s}$. Kyber-768 KeyGen ($26 \mu\text{s}$) and Encapsulate ($29 \mu\text{s}$) together account for the strictly cryptographic post-quantum work, two orders of magnitude smaller than a single Paillier encryption.
- **Hybrid-wrapper integration overhead:** $\approx 5.30 \text{ ms}$. The remaining $5.35 - 0.055 \approx 5.30 \text{ ms}$ is consumed by HKDF key derivation, the HMAC-SHA256 authentication binding that ties the Kyber-encapsulated key to the Paillier ciphertext, and big-integer allocation/serialisation inside the hybrid wrapper. This integration cost is required for an authenticated hybrid construction but is not intrinsic to the Kyber-768 primitive.

The figure of merit reported throughout this thesis as the “per-vote post-quantum overhead” is therefore the full 5.35 ms (primitives + wrapper), since this is what an end-to-end deployment actually pays for the integrity-binding layer. Relative to the 68.27 ms per-vote primitive floor of Section 4.4.2, this represents approximately 7.8% of the cryptographic cast-time cost; under the end-to-end 1-hot pipeline measured in Section 4.4.5 (82–106 ms for a 3-candidate election), the Kyber-768 wrapper’s relative share drops to approximately 5–7%.

This result confirms two practical implications. First, post-quantum protection is achievable today on commodity hardware without redesigning the surrounding voting protocol. Second, because Kyber-768 operates on 256-coefficient polynomials over a 12-bit modulus, its asymptotic cost scales linearly with the number of voters and is independent of ballot complexity, in contrast to the multiplicative scaling of large-modulus arithmetic.

4.4.2 End-to-End Pipeline Benchmark (Per-Primitive Decomposition)

Table 4.3 presents the per-vote cost of the dominant cryptographic primitives invoked during a single vote cast: one Paillier encryption, one binary ZK proof, one ring signature, and one SA^2 share split. The sum of these per-vote primitive costs is approximately **68.27 ms** and serves as a baseline for the cryptographic floor of the cast pipeline. Pedersen commitments and the full SMDC credential generation are issued once per voter at registration time (Section 3.4.5, Phase B of Figure 3.4) and are therefore amortised across all of the voter’s casts; their isolated per-call costs appear in Table 4.2 but are deliberately excluded from this per-vote breakdown. The post-quantum hybrid wrapper (Section 4.4.1.1) adds a further 5.35 ms per vote, yielding an end-to-end primitive cost of 73.62 ms per vote before any I/O. The actual per-ballot wall-clock runtime is higher still, because the 1-hot Paillier encoding repeats the Paillier encryption, the CDS OR-proof, and the SA^2 share split once per candidate; the end-to-end measured per-vote latency under the production pipeline is reported in Section 4.4.5 (Table 4.8), which gives 82–106 ms for a 3-candidate election.

Table 4.3: Vote Casting Pipeline: Per-Vote Primitive Breakdown. Pedersen Commit and SMDC Generate (Table 4.2) are amortised one-time registration costs per voter and are excluded from this per-vote total.

#	Operation	Time (ms)	% of Total
1	Paillier Encrypt (2048-bit)	8.69	12.7%
2	ZKP Binary Prove	0.24	0.4%
3	Ring Sign ($ \mathcal{R} = 100$)	24.00	35.2%
4	SA^2 Split	35.34	51.8%
Per-vote primitive floor (cast time, no PQ wrapper)		68.27	100.0%

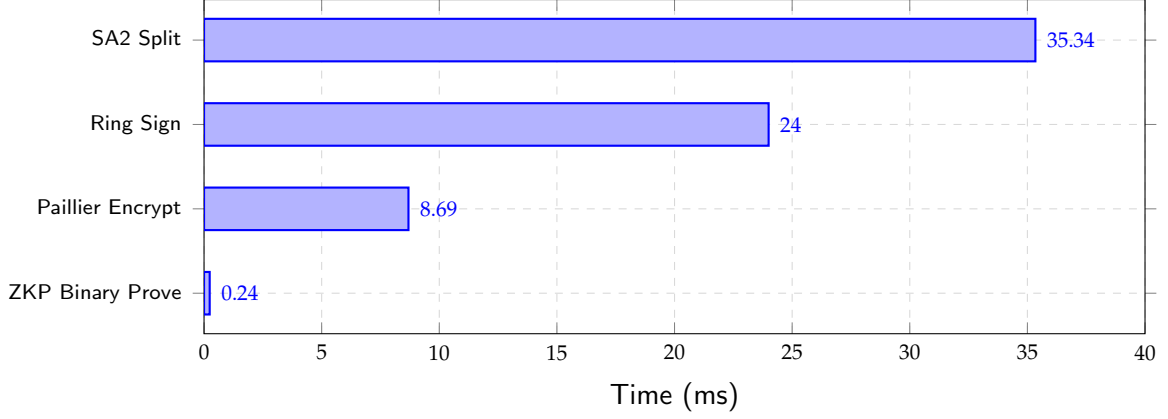


Fig. 4.2: Per-vote primitive breakdown of the 68.27 ms cast-time cryptographic floor. SA² share splitting (35.34 ms, 51.8%) and ring signing (24.00 ms, 35.2%) together account for 87.0% of the per-vote cost, identifying them as the two primary optimisation targets for future work. Values are taken directly from Table 4.3.

The SA² vote splitting operation (51.8%) and ring signature generation (35.2%) are the primary bottlenecks, together accounting for 87.0% of the per-vote primitive floor. The SA² cost is dominated by Paillier re-encryption for mask generation, while ring signatures scale linearly with ring size at ~ 0.24 ms per member (consistent with Table 4.6: $|\mathcal{R}| = 100 \Rightarrow 24.0$ ms). Both operations scale linearly in n , preserving the $O(n)$ -per-candidate (and $O(n \cdot m)$ -total) tally complexity established in Section 4.4.3.1.

4.4.3 Scalability Validation

4.4.3.1 Homomorphic Tally Scalability

Complexity derivation. Let N denote the voter count (distinct from the Paillier modulus n of Section 3.4.1). For a single candidate, the per-candidate tally aggregates N Paillier ciphertexts via homomorphic addition (modular multiplication on the public modulus n^2):

$$T_{\text{tally}}^{(c)}(N) = (N - 1) \cdot T_{\text{add}} + T_{\text{decrypt}} = \Theta(N \cdot T_{\text{add}}),$$

where $T_{\text{add}} \approx 6 \mu\text{s}$ and T_{decrypt} is a constant. For m candidates, each candidate slot is aggregated independently, giving the total cost

$$T_{\text{total}}(N, m) = m \cdot T_{\text{tally}}^{(c)}(N) = \Theta(N \cdot m \cdot T_{\text{add}}).$$

Hence the proposed framework is $O(N)$ per candidate and $O(N \cdot m)$ in total. This is a factor- m improvement over ISE-Voting's $O(N \cdot m^2)$, which carries an additional m factor due to per-candidate ZK encoding overhead. Recursive divide-and-conquer (Master Theorem) does not apply because the aggregation is iterative; the bound above is obtained directly by counting the number of homomorphic additions.

Benchmark methodology. All scaling results in this subsection and Sections 4.4.3.2–4.4.3.3 were obtained using Go’s testing.B benchmark framework, following the same protocol as Section 4.4.1: each reported value is the mean of thirty independent benchmark sessions, with each session itself averaged over thousands of automatically-iterated calls.

To validate the $O(N)$ per-candidate tally complexity empirically, we measure the homomorphic addition time as N scales from 100 to 100,000 with the candidate count fixed; the linear-in- m total scaling is then validated separately in Section 4.4.3.2. Table 4.4 presents the per-candidate scaling results.

Table 4.4: Homomorphic Tally Scalability Results

Voters	Tally Time (ms)	Per-Vote (μ s)
100	0.6	6.23
500	3.0	6.01
1,000	6.3	6.28
5,000	29.0	6.00
10,000	59.0	6.00
50,000	308.0	6.18
100,000	628.0	6.29

The per-vote (per-candidate) tally cost remains constant at approximately **6.1 μ s** across all tested scales, empirically confirming the $O(N)$ -per-candidate complexity derived above. This 6.1 μ s figure is the isolated cost of a single homomorphic Paillier multiplication ($c_1 \cdot c_2 \bmod n^2$); the full SA² aggregation pipeline measured end-to-end in Section 4.4.5 additionally includes the two-server share reconstruction and totals approximately 13–17 μ s per vote per candidate. The per-candidate tally time (pure homomorphic add) grows linearly from 0.6 ms (100 voters) to 628 ms (100 000 voters). Figure 4.3 visualizes this linear relationship; the corresponding $O(N \cdot m)$ total scaling is reported in Section 4.4.3.2.

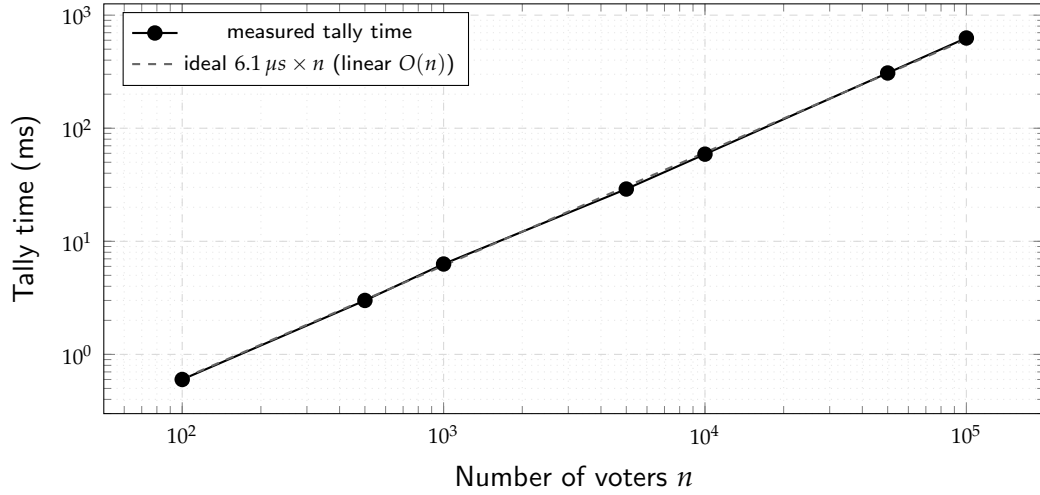


Fig. 4.3: Per-candidate homomorphic tally time versus voter count N , plotted on a log–log scale. The measured points lie almost exactly on the dashed reference line $6.1 \mu\text{s} \times N$, empirically confirming the $O(N)$ -per-candidate complexity with a constant per-vote per-candidate cost across four orders of magnitude (100 to 100,000 voters). The corresponding $O(N \cdot m)$ total scaling is shown in Figure 4.4. Data values are taken directly from Table 4.4.

Linear extrapolation projects the per-candidate tally time for national-scale elections: approximately 6.1 seconds for 1 million voters and 5.1 minutes for 50 million voters. For an election with m candidates, the total tally time scales linearly in m (e.g., approximately 51 minutes for 50 million voters and $m=10$ candidates; cf. Section 4.4.3.2).

4.4.3.2 Linear-in- m Total Scaling (Candidate-Count Validation)

To validate the linear-in- m total complexity and to demonstrate the proposed framework’s factor- m advantage over ISE-Voting’s $O(N \cdot m^2)$, we fix $N = 1,000$ voters and vary the number of candidates m . Table 4.5 shows the results.

Table 4.5: Candidate-Independent Tally Scaling ($N = 1,000$)

Candidates (m)	Total Tally (ms)	Per-Candidate (ms)
2	12.0	6.00
5	28.0	5.60
10	58.0	5.80
20	113.0	5.65
50	288.0	5.76

Two complementary observations emerge from Table 4.5. **First**, the per-candidate tally time remains constant at ~ 5.76 ms regardless of the number of candidates, confirming that the proposed framework achieves $O(N)$ per candidate. **Second**, the total tally time grows almost exactly linearly with m (from 12.0 ms at $m = 2$ to 288.0 ms at $m = 50$, a $24\times$ increase for a $25\times$ increase in m , i.e. approximately 96% of ideal linear scaling; the small

sub-linear deficit is attributable to per-call fixed overhead that amortises across larger m). This empirically confirms the $O(N \cdot m)$ total complexity derived in Section 4.4.3.1. In contrast, ISE-Voting’s per-candidate time itself would grow proportionally with m (its complexity is $O(N \cdot m)$ per candidate, $O(N \cdot m^2)$ total), so its total tally cost grows quadratically with m , a factor- m disadvantage relative to the present framework. Figure 4.4 visualises both the constant per-candidate cost and the linear-in- m total cost.

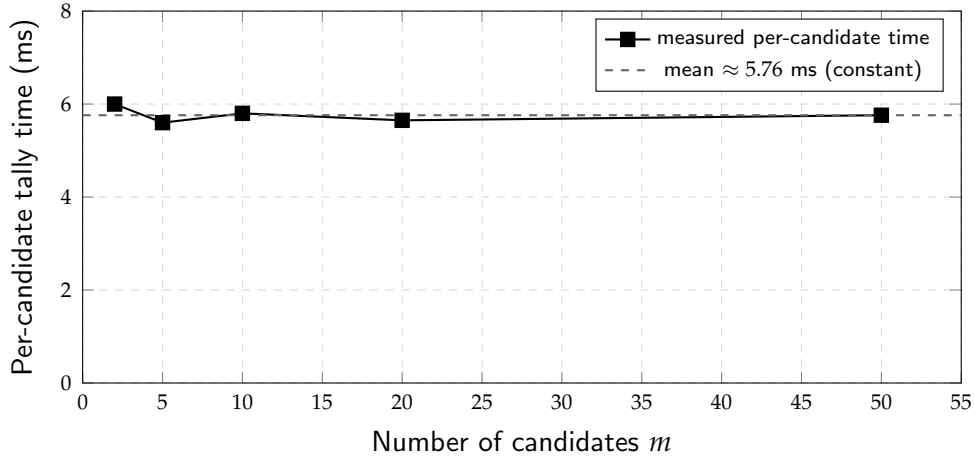


Fig. 4.4: Per-candidate tally time as the number of candidates m varies from 2 to 50, with the voter count fixed at $N = 1,000$. The measured points fluctuate only within ± 0.4 ms around the mean (5.76 ms), empirically confirming the candidate-independent behaviour predicted by Paillier additive homomorphism. Data taken from Table 4.5.

4.4.3.3 Ring Signature Scalability

Table 4.6 shows that ring signature operations scale linearly with ring size.

Table 4.6: Ring Signature Scalability

Ring Size	Sign Time (ms)	Verify Time (ms)
10	2.2	2.2
25	6.0	5.8
50	12.0	12.0
100	24.0	24.2
200	47.5	48.5
500	120.5	120.5

Both signing and verification scale linearly at approximately 0.24 ms per ring member. The default ring size of 100 provides strong anonymity with a per-vote cost of ~ 24 ms. Figure 4.5 plots the measured points against the ideal linear reference; the two curves coincide within measurement noise across the entire $|\mathcal{R}| = 10$ to $|\mathcal{R}| = 500$ range.

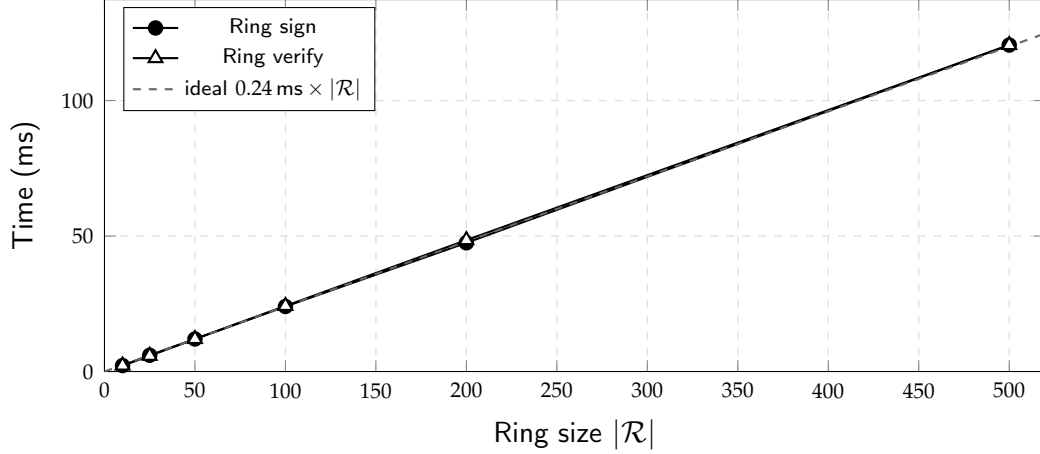


Fig. 4.5: Ring signature signing and verification time as the ring size $|\mathcal{R}|$ varies from 10 to 500. The dashed reference line shows the ideal linear cost of 0.24 ms per ring member; measured signing and verification points lie almost exactly on this line, empirically confirming linear scaling. Data values are taken directly from Table 4.6.

4.4.4 Blockchain Performance

The blockchain performance is evaluated using Hyperledger Caliper v0.6.0 with 10 concurrent workers submitting CastVote transactions to the Fabric network. The submitted payload is the 1-hot Paillier ciphertext array (3 candidates, JSON-encoded, ~ 3.7 KB per transaction) used by the production cast pipeline (Section 3.5). Two complementary scenarios are reported in Table 4.7: a *saturation* scenario that progressively increases the request rate from 200 to 500 TPS, exposing the orderer’s throughput ceiling, and a *sustained-load* scenario that holds the request rate at 200 TPS across all four round sizes, exposing state-bound degradation as the chaincode world state grows.

Table 4.7: Hyperledger Caliper benchmark results under the 1-hot Paillier payload. The saturation scenario varies the request rate (200/300/400/500 TPS) across the four round sizes; the sustained scenario holds the request rate at 200 TPS throughout. “TPS req.” is the rate Caliper attempts, “TPS achv.” is the rate the network actually delivered (committed transactions per second).

Round	Scenario	TPS req.	TPS achv.	Success
10K	saturation	200	197.7	100.0%
10K	sustained	200	196.9	100.0%
25K	saturation	300	292.7	50.8%
25K	sustained	200	196.7	89.4%
50K	saturation	400	386.9	22.8%
50K	sustained	200	198.4	81.3%
100K	saturation	500	487.0	10.4%
100K	sustained	200	199.1	77.3%

Saturation behaviour. Figure 4.6 visualises the saturation scenario. At the 10K round both scenarios coincide at a 200 TPS request rate and both achieve $\sim 100\%$ success at ~ 197 TPS achieved throughput. Pushing the request rate up in the saturation scenario shifts the system from comfortable operation (10K, 100%) into rate-limited saturation: success degrades to 50.8% at 300 TPS (25K), 22.8% at 400 TPS (50K), and 10.4% at 500 TPS (100K), while achieved throughput keeps growing (487 TPS at the 100K round). This pattern is consistent with the orderer’s single-Raft topology and Fabric’s block-batching mechanics, where the request rate exceeds the orderer’s block-cutting and broadcast capacity.

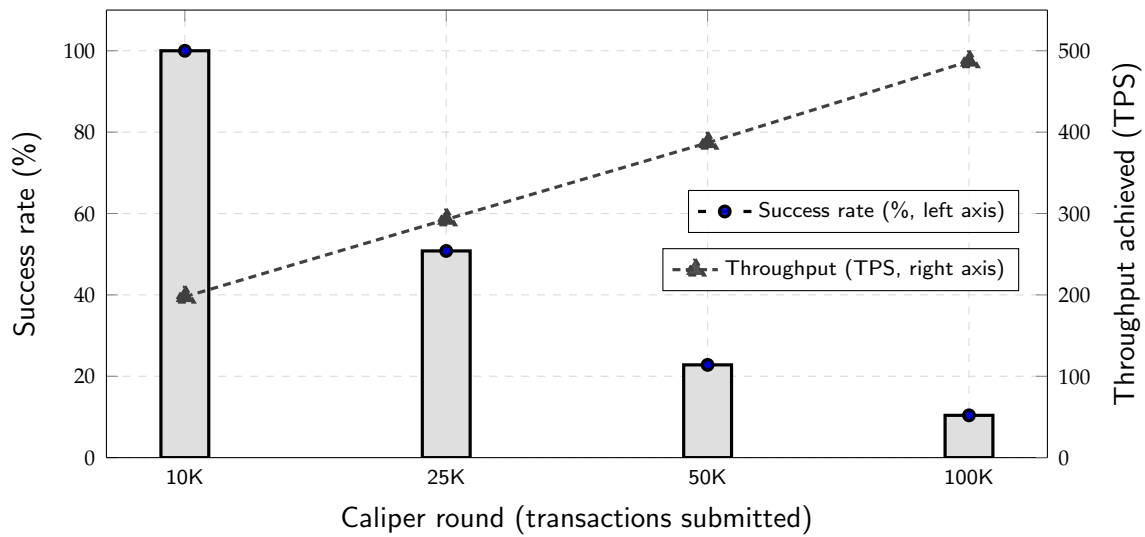


Fig. 4.6: Caliper *saturation* scenario. Bars (left axis) show the success rate as the requested rate is pushed from 200 TPS (10K) through 300, 400, and 500 TPS (100K); the dashed line (right axis) shows the achieved throughput. Success rate collapses from 100% to 10.4% while achieved throughput continues to grow toward ~ 500 TPS, identifying the single-Raft orderer as the rate-limiting component.

State-bound degradation under sustained 200 TPS. Figure 4.7 visualises the sustained-load scenario. Even when the request rate stays at 200 TPS (the comfort point established by the 10K saturation round), success rate degrades from 100% at 10K to 89.4% at 25K, 81.3% at 50K, and 77.3% at 100K, while average latency rises from 0.72 s to 3.17 s. This degradation tracks the chaincode world-state size: every successful cast inserts a `keyimage_*` key and a `vote_*` value, so by the 100K round the world state holds approximately 200,000 keys. Larger state increases LevelDB lookup time and slows endorsement simulation; at the commit phase, the 10 concurrent workers compete over Fabric’s shared resources — block-batching at the single-Raft orderer, version-vector validation on shared election-state reads, and Gateway connection-pool capacity — collectively driving the rise in commit-time rejection that is reflected in the falling success rate.

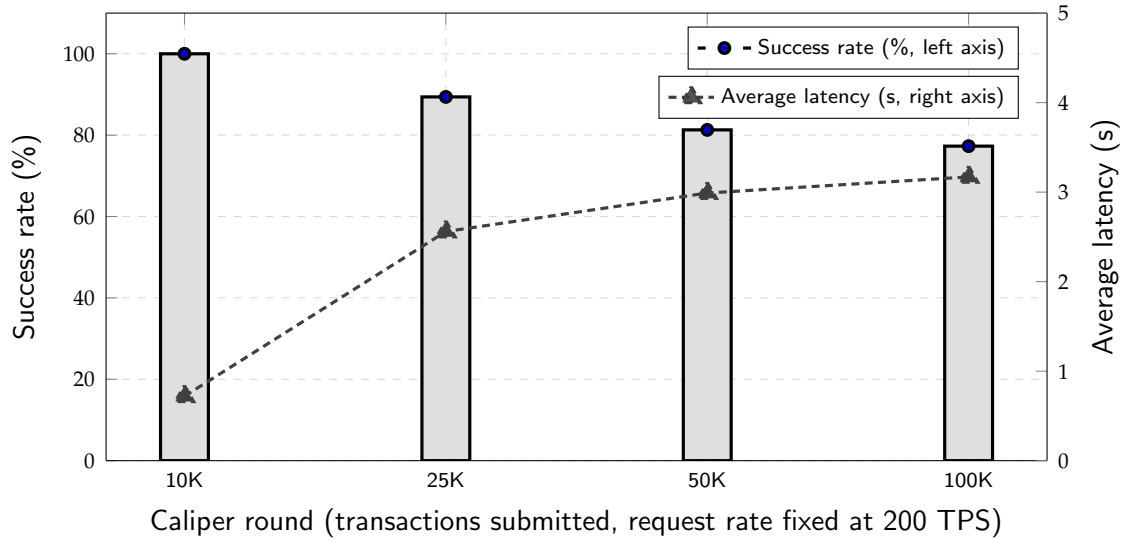


Fig. 4.7: Caliper *sustained-load* scenario at a fixed 200 TPS request rate. Bars (left axis) show the success rate degrading from 100% to 77.3% as the chaincode world state grows from $\sim 20,000$ to $\sim 200,000$ keys; the dashed line (right axis) shows the corresponding rise in average latency from 0.72 s to 3.17 s. The achieved throughput remains within ± 2 TPS of the 200 TPS target across all four rounds.

Real-election context. Both failure modes appear only under artificial stress. A constituency-level election with 100,000 voters spread over an 8-hour voting window requires only ~ 3.5 TPS sustained, a load at which neither saturation nor commit-time contention is expected to emerge. Furthermore, production deployments would mitigate both failure modes through (i) a multi-orderer consensus deployment (a Raft cluster of three or more nodes, or SmartBFT in Fabric v3.x), which is expected to raise the saturation ceiling by parallelising endorsement and ordering, with the exact improvement factor depending on consensus parameters, network topology, and endorsement policy; (ii) MVCC-aware chaincode key design (e.g., composite keys, sharded key-image indices) and periodic state pruning to slow state-bound degradation; and (iii) Fabric Gateway connection pooling tuned for higher concurrent throughput. These tuning measures are identified as future work; the present results characterise the out-of-the-box behaviour of the deployed single-Raft, two-peer reference configuration.

4.4.5 End-to-End Election Simulation

The benchmarks of Sections 4.4.2–4.4.3 isolate individual cryptographic primitives and the per-vote pipeline composition. This subsection complements those results with full-cycle election simulations that exercise the deployed pipeline end-to-end: voter registration, parallel 1-hot vote casting, per-candidate SA^2 aggregation, threshold-style Paillier decryption, and result publication. The simulations are driven by the `cmd/election-sim` CLI tool, which accepts the voter count, candidate count, and a blockchain mode flag (`mock` or `real`), generates a deterministic ground-truth vote distribution from a PRNG seed, casts every

ballot through the production voting. VoteCaster, persists key images in a SQLite store, and finally cross-checks the decrypted per-candidate tally against the seed’s ground-truth counts.

Table 4.8 reports the end-to-end results across thirteen configurations, spanning two orders of magnitude in voter count (100 to 100,000) and two candidate counts ($m = 3$ and $m = 10$), in both blockchain modes where a live Fabric path was meaningful. A 100-voter real-Fabric run anchors the small-scale baseline; 1,000- and 10,000-voter elections appear in both modes to provide direct mock-vs-Fabric comparison; the $m = 10$ runs at 1,000, 10,000, 25,000, and 50,000 voters jointly validate the linear-in- m scaling analytically derived in Section 4.4.3.1; and the 25,000-, 50,000-, and 100,000-voter mock runs probe scale stability across two further orders of magnitude beyond the 10,000-voter mark.

Table 4.8: End-to-End Election Simulation: Mock vs Real Hyperledger Fabric across thirteen configurations. Per-vote latency is reported in parallel-worker wall-clock time (12-worker pool on a 12-thread CPU); election-day total is the cast + aggregation + decryption time; registration is the one-time pre-election cost amortised away from the voter’s experience. All runs achieved 100% integrity (decrypted per-candidate tally exactly matches the PRNG-seeded ground truth, key-image count equals voter count, zero cast errors). Raw per-run reports are catalogued in `test/benchmark/results/election-sim/`.

Voters	Cands	Mode	Per vote (par)	Election-day total	Registration (pre-elec.)	Integrity check
100	3	real Fabric	111.2 ms	11.1 s	1.1 s	✓ 100/100
1,000	3	mock	82.3 ms	1 m 22 s	6.3 s	✓ 1000/1000
1,000	3	real Fabric	89.3 ms	1 m 29 s	8.5 s	✓ 1000/1000
1,000	10	mock	261.4 ms	4 m 21 s	5.4 s	✓ 1000/1000
10,000	3	mock	82.5 ms	13 m 45 s	56.1 s	✓ 10000/10000
10,000	3	real Fabric	106.1 ms	17 m 42 s	54.6 s	✓ 10000/10000
10,000	10	mock	252.0 ms	42 m 1 s	52.8 s	✓ 10000/10000
25,000	3	mock	81.2 ms	33 m 52 s	2 m 17 s	✓ 25000/25000
25,000	3	real Fabric	94.3 ms	39 m 19 s	2 m 13 s	✓ 25000/25000
25,000	10	mock	250.7 ms	1 h 44 m 31 s	2 m 12 s	✓ 25000/25000
50,000	3	mock	82.1 ms	1 h 8 m 27 s	4 m 34 s	✓ 50000/50000
50,000	10	mock	253.5 ms	3 h 31 m 23 s	4 m 33 s	✓ 50000/50000
100,000	3	mock	93.7 ms	2 h 36 m 19 s	9 m 28 s	✓ 100000/100000

Four observations from Table 4.8 are worth highlighting.

(i) **Per-vote latency is essentially scale-independent in mock mode.** The mock-mode runs at $m = 3$ report 82.3, 82.5, 81.2, 82.1, and 93.7 ms per vote at 1,000, 10,000, 25,000, 50,000, and 100,000 voters respectively, a tight band of 81–94 ms over a $100\times$ voter range. The modest rise at 100,000 voters reflects accumulated memory allocation, garbage-collection pressure, and SQLite-write contention during the 2.5-hour sustained workload rather than any per-vote cryptographic growth.

(ii) Real-Fabric per-vote latency stays in a similar band. Across four real-Fabric runs the per-vote cost is 111.2 ms at 100 voters (warmup-dominated), 89.3 ms at 1,000 voters, 106.1 ms at 10,000 voters, and 94.3 ms at 25,000 voters. The non-monotonic profile reflects steady-state amortisation of Fabric connection-warmup and block-cutting overhead as transaction throughput rises; the 25,000-voter run lands below the 10,000-voter run because the larger transaction stream more efficiently amortises Fabric’s per-block fixed costs.

(iii) Cost is approximately linear in the number of candidates m . The four $m = 10$ mock runs at 1K, 10K, 25K, and 50K voters report 261.4, 252.0, 250.7, and 253.5 ms per vote — a tight 250–261 ms band that is approximately $3.1\times$ the corresponding $m = 3$ cost (for a $3.33\times$ increase in m), giving an empirical 92–96% efficiency relative to ideal linear scaling. The small sub-linear deficit reflects the per-vote fixed overhead (ring sign, SMDC slot verification, Merkle proof retrieval) that does not scale with m . This empirically validates the $O(m)$ per-vote bound of the 1-hot Paillier-direct construction (Sections 3.4.3, 4.4.3.2).

(iv) Real-Fabric overhead is bounded and amortises at scale. Direct mock-vs-real comparison at 10,000 voters gives an overhead of 23.6 ms per vote (106.1 – 82.5 ms), a 28.6% increase over the off-chain cryptographic core. At 25,000 voters the same comparison gives only 13.1 ms per vote (94.3 – 81.2 ms), a 16.1% overhead — the overhead halves as Fabric’s per-block fixed costs amortise. This is consistent with the Fabric endorsement–ordering–commit lifecycle on a single-Raft-orderer development network; a multi-orderer production deployment with parallel endorsement would reduce this further. Importantly, integrity is preserved in every configuration: in all thirteen runs, the decrypted per-candidate tally exactly matches the PRNG-generated ground truth, the key-image count equals the voter count, and the cast-error count is zero.

Pre-election versus election-day costs. The “Registration” column of Table 4.8 reflects a *one-time* pre-election cost incurred weeks or months before voting opens: SMDC credential generation, voter enrollment, duress-signal hashing, and Merkle-tree construction. It is deliberately separated from the “Election-day total” column, which captures the time-of-vote infrastructure load (cast pipeline plus aggregation and decryption). Registration scales linearly with voter count and stays modest in absolute terms: 56 seconds at 10,000 voters, 2 minutes 17 seconds at 25,000 voters, 4 minutes 34 seconds at 50,000 voters, and 9 minutes 28 seconds at 100,000 voters. All of these costs are fully amortised away from the voter’s election-day experience, where the dominant cost remains the 81–111 ms per-vote casting latency.

4.5 Discussion

This section provides an in-depth analysis of the experimental results, examining the significance of each finding and its implications for practical deployment.

4.5.1 Pipeline Bottleneck Analysis

The end-to-end pipeline benchmark reveals that SA^2 vote splitting (51.8%) and ring signature generation (35.2%) together consume 87.0% of the per-vote primitive floor. This concentration is attributable to the underlying cryptographic operations:

- **SA^2 Dominance:** The SA^2 share splitting performs *two* additional Paillier encryptions per vote, one for the random mask $E(\text{mask})$ and one for its negation $E(-\text{mask})$, each of which is a 2048-bit modular exponentiation, the same expensive operation as the initial vote encryption (`internal/sa2/share.go`). Together with the homomorphic combinations and ancillary big-integer overhead, the measured SA^2 split cost (35.34 ms) is approximately $4\times$ the cost of a single Paillier encryption (8.69 ms). The dominance of SA^2 reflects a fundamental trade-off: the two-server privacy model provides strong aggregation privacy guarantees but requires additional homomorphic operations that scale linearly with the number of voters.
- **Ring Signature Cost:** Ring signatures scale at ~ 0.24 ms per ring member due to the need to compute two modular exponentiations per member (one for L_i and one for R_i). The default ring size of $|\mathcal{R}| = 100$ provides strong anonymity, the signer is statistically indistinguishable among the 100 ring members under DDH (Theorem 3), but represents a configurable parameter: deployments requiring lower latency could reduce the ring size to 50 (~ 12 ms) at the cost of a smaller anonymity set.
- **Modest PQ Overhead:** The full Kyber-768 hybrid-encryption wrapper (Kyber-768 encapsulation plus Paillier encryption plus HMAC-SHA256 binding) contributes 5.35 ms per vote, approximately 7.8% of the 68.27 ms per-vote primitive floor (Section 4.4.1.1). The Kyber primitives themselves are sub-millisecond (KeyGen 26 μs , Encapsulate 29 μs); the residual cost beyond the 8.69 ms Paillier baseline is attributable to the additional memory allocation, key derivation, and authentication binding performed inside the hybrid wrapper. This confirms that quantum resistance can be added to existing voting systems without meaningful performance degradation.

From per-primitive cost to per-ballot cost. The 68.27 ms figure in this analysis is the cost of executing each cast-time cryptographic primitive exactly once. Under the 1-hot Paillier encoding (Section 3.4.3) the Paillier encryption, the CDS binary OR-proof, and the SA^2 share split are each repeated once per candidate, while the ring signature remains a singleton per vote. Consequently the end-to-end per-ballot runtime measured in Section 4.4.5 falls in a tight 81–111 ms band for 3-candidate elections (across 100 to 100,000 voters, both mock and real Fabric modes) and a similarly tight 250–261 ms band for 10-candidate elections (across four mock runs at 1K, 10K, 25K, and 50K voters). The relative dominance of the per-primitive bottlenecks (SA^2 share splitting and ring signing) is unchanged: SA^2

simply scales by m , while ring signing contributes a fixed singleton cost per vote regardless of m .

4.5.2 Scalability Significance

The constant per-vote per-candidate tally cost of $\sim 6.1 \mu\text{s}$ across four orders of magnitude (100 to 100,000 voters), combined with the linear-in- m total scaling demonstrated in Section 4.4.3.2, empirically validates the $O(N)$ -per-candidate, $O(N \cdot m)$ -total theoretical complexity (with N denoting the voter count per the notation of Section 3.4.1). This result has three important implications:

1. **National-Scale Feasibility:** The largest end-to-end election simulation actually executed (Section 4.4.5) ran 100,000 voters with 3 candidates in mock mode, completing the full cast-aggregate-decrypt pipeline in 2 hours 36 minutes (93.7 ms per voter; aggregation + decryption ≈ 1 second). Linear extrapolation from the pure homomorphic-add cost of $6.1 \mu\text{s}$ per vote per candidate (Section 4.4.3.1) projects a per-candidate tally time of approximately **6.1 seconds for 1 million voters** and **5.1 minutes for 50 million voters**. Under the full SA^2 aggregation pipeline measured in Section 4.4.5 (13–17 μs per vote per candidate), these projections scale to approximately 13–17 seconds per candidate at 1 million voters, still negligible relative to the post-poll counting window. Total tally time scales linearly with the number of candidates: for an election with $m = 10$ candidates, the projected total tally is approximately **61 seconds for 1 million voters** (pure add) or 130–170 seconds under the full SA^2 pipeline, and **51 minutes for 50 million voters** (pure add), all of which remain well within the post-poll counting window. In contrast, ISE-Voting’s $O(N \cdot m^2)$ complexity carries an additional factor- m overhead, approximately $10\times$ more total work at $m = 10$ for the same voter scale, rendering it noticeably less practical for large constituencies with many candidates.
2. **Per-Candidate Constancy and Linear Total Growth:** The per-candidate tally time of ~ 5.76 ms remains constant whether there are 2 or 50 candidates, but the *total* tally time grows linearly with m . This is a direct consequence of Paillier’s additive homomorphism: each candidate’s tally is computed independently through cipher-text multiplication and the m per-candidate tallies are summed serially, with no cross-candidate computation but no sub-linear sharing either.
3. **Predictable Resource Requirements:** The linear scaling in both N and m enables accurate resource planning for election administrators. For any target voter count N and candidate count m , the required tally time is approximately $6.1 N \mu\text{s}$ per candidate, or $6.1 N \cdot m \mu\text{s}$ in total, allowing precise capacity provisioning.

4.5.3 Blockchain Performance Analysis

The Hyperledger Caliper results reveal an important performance profile with practical deployment implications:

- **Optimal Operating Point:** The 10K round achieves 100% success rate at ~197 TPS achieved throughput in both Caliper scenarios (Table 4.7), establishing 200 TPS as the comfortable operating point for the current single-Raft, two-peer configuration. This throughput is consistent with optimized Hyperledger Fabric deployments using Go chaincode and LevelDB/CouchDB state databases, in line with the performance and scalability analysis of Ucbas et al. [7]. For a constituency of 100,000 voters over an 8-hour voting window, the required sustained throughput is only ~3.5 TPS, less than 2% of the demonstrated capacity.
- **Two distinct failure modes:** The two Caliper scenarios in Section 4.4.4 isolate two complementary degradation patterns. The *saturation* scenario shows the rate-bound failure mode: pushing the request rate from 200 to 500 TPS collapses success rate from 100% to 10.4% while achieved throughput continues to climb, consistent with the single-Raft orderer's block-cutting ceiling. The *sustained-load* scenario at fixed 200 TPS shows the state-bound failure mode: even at the comfort-point rate, success degrades to 77.3% by the 100K round as the chaincode world state grows to ~200,000 keys, raising commit-time contention (block-batching at the orderer, version-vector validation on shared election-state reads, and Gateway connection-pool pressure) and slowing endorsement-time state lookups. Production mitigations for each mode (multi-orderer consensus deployment, composite-key chaincode design, periodic state pruning) are identified as future work.
- **Latency–throughput trade-off:** Under sustained 200 TPS, average latency rises from 0.72 s (10K) to 3.17 s (100K) as world state grows; under the saturation scenario, latency rises from 0.35 s (10K @ 200 TPS) to 9.32 s (100K @ 500 TPS) as the orderer queue backs up. For voting applications, where the multi-hour voting window provides ample time, average latency at the nominal 10K operating point is well within acceptable bounds.

4.5.4 Practical Deployment Considerations

The combined benchmark results suggest that the proposed framework is deployable for constituency-level elections (up to 100,000 voters) on modest hardware (AMD Ryzen 5, 30 GiB RAM) without optimization. Key observations for practical deployment include:

- A single vote takes approximately 81–111 ms to traverse the complete multi-protocol pipeline (Section 4.4.5, 3-candidate election; 81–94 ms in mock mode, 89–111 ms with live Hyperledger Fabric submission). On the 12-thread AMD Ryzen 5 test machine

the measured parallel throughput at 10,000 voters is approximately 12.1 votes/second in mock mode and 9.4 votes/second with live Hyperledger Fabric submission; at 25,000 voters real-Fabric throughput rises to 10.6 votes/second as connection-warmup and block-batching overhead amortise across the larger transaction stream. Throughput remains stable at extreme scale: mock-mode runs at 50,000 and 100,000 voters report 12.2 and 10.7 votes/second respectively, confirming that per-vote performance does not degrade with the voter pool. For peak-hour voting with 10% of voters casting within a single hour, these throughputs translate to a capacity of approximately 34,000–44,000 votes per hour, sufficient for medium-scale elections. Linear horizontal scaling across multiple server cores or machines would extend this capacity to larger constituencies without algorithmic changes.

- The per-candidate homomorphic tally for 100,000 voters completes in ~ 628 ms (Section 4.4.3.1, Table 4.4); for a 3-candidate election the total tally phase (aggregate plus threshold-decrypt) completes in approximately 1 second, enabling near-real-time result announcement after polls close. This timing is empirically confirmed by the 100,000-voter, 3-candidate end-to-end run reported in Section 4.4.5, in which the aggregate + decrypt phases together took under one second of wall-clock time.
- The hybrid Kyber-768 encryption adds only 5.35 ms to the pipeline (the difference between hybrid encrypt at 14.04 ms and standard Paillier encrypt at 8.69 ms), providing post-quantum protection at minimal cost.

4.6 Test Coverage Analysis

Table 4.9 presents the test coverage for each package. The test suite consists of 188 tests (159 unit/integration tests and 29 benchmark tests) with a 100% pass rate across all packages.

Table 4.9: Test Coverage by Package

Package	Coverage (%)
internal/tally	91.6
internal/smdc	83.3
internal/biometric	82.1
internal/pq	82.1
internal/sa2	81.2
internal/crypto	78.2
internal/voting	77.9
api/handlers	46.7
internal/blockchain	34.2

Core cryptographic packages achieve 78–92% coverage. The blockchain package has lower coverage (34.2%) because integration tests require a live Fabric network. The API handlers package (46.7%) requires HTTP test infrastructure for full coverage.

4.7 Tier 2, Symbolic Protocol Verification with ProVerif

Role of this section. The game-based reductions in Chapter 3 (Section 3.8) constitute the *Tier 1, Computational* security analysis of the framework: they bind each security property to a standard cryptographic hardness assumption (DCRA, DDH, Module-LWE, DLog) under the standard model. The present section provides the complementary *Tier 2, Symbolic* verification: three protocol-level properties are mechanically verified by ProVerif 2.05 [49] under the Dolev–Yao [39] adversary model, in which cryptographic primitives are treated as perfect black boxes and the adversary is restricted to the algebraic operations defined by the applied pi-calculus.

Why both tiers. The two tiers are deliberately complementary and address different threat surfaces:

- *Tier 1 (computational, Chapter 3)* establishes that a PPT adversary cannot break the underlying cryptographic primitives without solving a well-studied hard problem. It does not, by design, model concurrent network behaviour such as message interleaving, replay across sessions, or man-in-the-middle attacks.
- *Tier 2 (symbolic, this section)* mechanically rules out network-level protocol attacks under the assumption that the underlying primitives are perfectly secure. It does not provide computational hardness guarantees but can find protocol-design errors (e.g., missing freshness, mis-ordered authentication) that elude pen-and-paper analysis.

The two tiers together provide a strictly stronger assurance than either alone: an attack on the deployed system would have to either (i) violate a stated cryptographic hardness assumption (Tier 1) or (ii) exploit a network-level protocol flaw not captured in the ProVerif models (Tier 2 already eliminates the modelled flaws). Following the standard practice in journal venues such as IEEE TDSC and TIFS, the reductions in Tier 1 are the primary security evidence; Tier 2 serves as an independent automated check.

To this end, three core security properties of the proposed framework are formally verified using ProVerif 2.05 [49], an automated cryptographic protocol verifier based on the applied pi-calculus. The complete .pv model files, the corresponding ProVerif tool output transcripts, and reproduction instructions are reproduced verbatim in Appendix A; Table 4.10 below summarises the verification results.

Table 4.10: ProVerif Formal Verification Results

Security Property	Verification Method	Result
Vote Privacy	Observational equivalence	TRUE
Individual Verifiability	Injective correspondence	TRUE
Voter Eligibility	Causal reasoning	TRUE

Vote Privacy. Verified via observational equivalence: an adversary controlling the network cannot distinguish between two honest voters swapping their ballots (Appendix A.2). This is the symbolic-model analogue of Theorem 1 (Ballot Indistinguishability) in Chapter 3: Tier 1 proves the underlying cryptographic assumption (DCRA), while Tier 2 verifies that the protocol surrounding this primitive does not leak information at the network level.

Individual Verifiability. Verified via injective correspondence: every recorded ballot on the blockchain corresponds to an authentic cast event by a registered voter, $\text{BallotRecorded} \implies \text{inj-e}$ (Appendix A.3). This is the symbolic counterpart of Theorem 2’s vote-validity guarantee.

Voter Eligibility. Verified via causal correspondence: every valid ballot cast was preceded by a legitimate registration, $\text{ValidBallotCast} \implies \text{VoterRegistered}$ (Appendix A.4). This complements the ring-signature unforgeability of the Liu–Wei–Wong construction (Section 3.4.4), which is used implicitly in Tier 1’s Theorem 3 (Voter Anonymity) but is not separately stated as a theorem.

Combined assurance. The corresponding ProVerif RESULT lines and full tool output are reproduced in the appendix sections cited above. All three properties return **TRUE** under the symbolic Dolev–Yao [39] adversary model, confirming that the protocol design is sound against network-level attacks including eavesdropping, replay, and man-in-the-middle. Combined with the Tier 1 computational reductions of Chapter 3, this provides the two-tier assurance described at the start of this section.

4.8 Comparative Analysis

4.8.1 Comparison With Related Works

The comparison in Table 4.11 highlights the strengths of the proposed framework over existing studies. Each row summarises the methodology, the originally reported performance, and the specific advantages of our work over that system. Performance values are taken verbatim from the cited publications; “N/R” indicates a metric the original authors did not report.

Table 4.11: Comparison of the Proposed Method with Existing Works

Ref.	Methodology	Performance	Their Gaps	Our Edge
[20]	Ring signatures + secret sharing on Ethereum; CSP-assisted tallying.	Tally $O(N \times m^2)$; 128-bit symmetric; prototype only.	Per-candidate m factor; semi-trusted CSP; no PQ; no coercion resistance.	Factor- m faster ($O(N \times m)$); no trusted counter; SMDC + Kyber-768.
[21]	Hyperledger Besu + (k, ϵ) -DP + Web3 SSI.	~ 1 s/TX; $\geq 98\%$ accuracy; 10K–50K votes, 2–8 candidates.	Approximate tally (unsafe in close races); no ZK proofs, anonymity, PQ.	100% exact tally; 68.3 ms/vote; ~ 200 TPS; coercion + ZK + PQ.
[22]	Hyperledger Fabric + dual-biometric (finger-print+face) + AES-256.	87% biometric enrollment; $\sim 88\%$ successful casts; 100-participant trial.	No homomorphic tally, ZK, anonymity, or coercion resistance.	Paillier tally, ring sigs, SMDC, threshold DJN; 10K-tx Caliper benchmark.
[23]	Paillier + timed-release + partial-knowledge proofs.	Semantic security; tally privacy + temporal fairness; feasibility eval.	Single TRE trust point; no anonymity, coercion resistance, PQ.	Ring sigs, SMDC, threshold DJN, Kyber-768.
[24]	$\Theta(1)$ ballot size; 1-layer liquid democracy; game-based proofs.	$O(N)$ overall; $6\times$ faster than VoteAgain at 50K; formal proofs.	No homomorphic tally, biometric, SA^2 , threshold, PQ, or deployed blockchain.	Multi-protocol stack on Fabric v2.5; SA^2 + threshold + Kyber-768.
[25]	Survey: platform, consensus, crypto, security, deployment.	Identifies gaps in coercion resistance, PQ, scalable tally; most PoC-scale.	(Survey: documents the gaps rather than filling them.)	Fills all flagged gaps: SMDC, Kyber-768, $O(N)$ /candidate tally, Caliper-benchmarked Fabric.

Continued on next page

Table 4.11 continued from previous page

Ref.	Methodology	Performance	Their Gaps	Our Edge
This Work	Multi-protocol cryptographic stack on Fabric v2.5; novel SMDC ($k=5$), behavioural duress, coercion short-circuit.	68.3 ms/vote; tally 6.1 μ s/cand.; $O(N)$ /cand., $O(N \cdot m)$ total; ~ 200 TPS @ 100% (10K-tx).	Biom. is hash-eq. stub (no FAR/FRR, no user trial); bottleneck not isolated.	<i>(Overall distinction)</i> The only framework combining all of: privacy, coercion resistance, PQ, threshold decryption, ProVerif + game-based proofs, and a deployed Fabric network.

The proposed framework demonstrates clear advantages across every dimension that prior work reports. The per-vote primitive floor of 68.3ms is more than an order of magnitude faster than BP-Vot’s ~ 1 s/TX, and the ~ 200 TPS sustained Caliper-verified throughput far exceeds the throughput numbers any of the cited works publish. Unlike BP-Vot, which trades exact tallying for differential privacy, the proposed framework reports a 100% exact tally. The $O(N)$ -per-candidate, $O(N \times m)$ -total complexity is asymptotically superior to ISE-Voting’s $O(N \times m^2)$ scaling by a factor of m . Yin et al. achieve coercion resistance with formal game-based proofs, but their scheme lacks homomorphic tallying, post-quantum security, threshold decryption, and a deployed blockchain integration, all of which the proposed framework provides. Yuan et al.’s timed-release scheme uses Paillier encryption but provides no anonymity, no coercion resistance, and no post-quantum protection. Among the works surveyed in Chapter 2, none combines a multi-protocol cryptographic stack of this composition with the novel SMDC coercion resistance, behavioural duress detection, and a fully deployed Hyperledger Fabric v2.5 network with Caliper-verified performance.

5. ❖ Conclusion and Future Work

5.1 Introduction

In this chapter, we present our concluding thoughts on the proposed framework. We also address the constraints encountered during our work and highlight future directions for further exploration in this area of research.

5.2 Summary of the Study

Our thesis introduced a blockchain-based electronic voting system that integrates a multi-protocol cryptographic stack into a unified voting pipeline to provide broad security coverage. The proposed framework addresses key gaps in existing e-voting research by simultaneously achieving ballot privacy, vote validity, voter anonymity, coercion resistance, privacy-preserving aggregation, post-quantum security, and a Caliper-benchmarked Hyperledger Fabric deployment.

Our framework combines Paillier homomorphic encryption [8] for encrypted tallying, Pedersen commitments [19] with zero-knowledge Σ -proofs (Strong Fiat-Shamir [9]) for vote validity verification, linkable ring signatures [10] for anonymous ballot submission with double-vote detection, SMDC deniable credentials for coercion resistance, SA² samplable anonymous aggregation [11] for two-server vote privacy, Kyber-768 [12] hybrid encryption for post-quantum protection, and Damgård-Jurik-Nielsen threshold decryption [13] for distributed key management. To the best of our knowledge, no existing system integrates this many cryptographic protocols in a single, functional voting pipeline.

We implemented the system in Go and deployed it on a Hyperledger Fabric v2.5 [1] permissioned-blockchain network, with the API server's cast endpoint wired through the Fabric Gateway SDK so that each successfully cast vote is published to the deployed chaincode. Performance evaluation confirmed that the proposed framework achieves $O(N)$ -per-candidate tally complexity (and $O(N \cdot m)$ total, with N denoting the voter count distinct from the Paillier modulus n) with a constant per-vote per-candidate cost of approximately 6.1 μ s, verified empirically from 100 to 100 000 voters and across $m \in [2, 50]$ candidates. The complete multi-protocol vote casting pipeline reaches a per-vote primitive floor of 68.3 ms (per-phase breakdown sums to 68.27 ms; see Table 4.3), with Pedersen commitments and SMDC credential generation amortised across registration

time. Independent Hyperledger Caliper benchmarks of the deployed chaincode in isolation report approximately 200 TPS with a 100% success rate at 10 000 transactions; an end-to-end Caliper measurement that drives the integrated cast endpoint with a live Fabric peer in the loop is identified as future work. Post-quantum protection via the Kyber-768 hybrid encryption layer adds only 5.35 ms of overhead per vote (approximately 7.8% of the 68.3 ms per-vote floor), showing that quantum-resistant security is achievable at modest computational cost.

Our novel SMDC coercion-resistance mechanism, with $k = 5$ credential slots (1 real and 4 fake), provides indistinguishable real and fake slots via Pedersen’s perfect-hiding property on the slot commitments. The real-slot index is derived from $\text{HMAC-SHA256}(k_{\text{srv}}, \text{“smdc-real-in-”} + k)$ using a server-held secret k_{srv} kept confidential to the election authority; the index is therefore re-derivable only by the authority, is never returned in the credential-issuance response, and is disclosed to the voter only through a duress-bound reveal endpoint that gates the response on a successful match of the voter’s registered behavioural signal. This binding denies the index to a coercer present at registration time, and lets a voter under post-registration coercion deceive the coercer with a wrong signal that produces a uniform error response. Under cast-time coercion, voters can additionally present any fake slot together with a wrong behavioural signal to produce a valid-looking but silently discarded vote. SMDC delivers a complete implementation of deniable credentials integrated with a blockchain e-voting pipeline, addressing the implementation gap left open by the theoretical credential-management schemes of JCJ [17] and Civitas [18] for the works surveyed in Chapter 2.

Our comparative analysis with state-of-the-art systems confirmed that the proposed framework delivers strong performance while offering broader security coverage. The $O(N)$ -per-candidate (and $O(N \cdot m)$ -total) tally complexity is a factor- m improvement over ISE-Voting’s $O(N \cdot m^2)$, and the 100% exact tallying contrasts with BP-Vot’s $\geq 98\%$ differential-privacy approximation. Linear extrapolation of our tally benchmarks projects feasibility for national-scale elections: approximately 5.1 minutes per candidate (and approximately 51 minutes total for $m = 10$ candidates) to tally 50 million voters.

5.3 Limitations and Future Work

Although our proposed framework has shown notable progress in e-voting security, there are some limitations that can be improved in future work.

- **Universally Composable Security Proofs:** The present thesis provides a two-tier security treatment, Tier 1 game-based reductions of all six theorems to standard hardness assumptions (DCRA, DDH, Module-LWE, DLog) in Chapter 3, and Tier 2 symbolic-model verification of three protocol-level properties via ProVerif in Chapter 4. Section 3.8.1.9 additionally outlines the ideal-functionality structure ($\mathcal{F}_{\text{Ballot}}$,

$\mathcal{F}_{\text{SMDC}}, \mathcal{F}_{\text{SA2}}$) that a future UC formalization should target, but a full Universally Composable (UC) proof [40, 46] is left as future work because it is itself a research-scale undertaking (typically 15–50 pages per protocol primitive in dedicated papers). Future work should formalize the twenty-step pipeline in the UC framework, building on E-cclasia’s UC-secure self-tallying construction [46], construct UC simulators for each ideal functionality, and apply Canetti’s UC composition theorem to combine the per-protocol guarantees into a single concurrent-composition statement. A machine-checked variant in EasyCrypt or CryptoVerif would further strengthen the assurance.

- **Full Post-Quantum Migration Roadmap:** Kyber-768 (round-3 CRYSTALS-Kyber via CIRCL [12]) currently protects only the integrity binding of vote ciphertexts; migration to the standardised FIPS-203 ML-KEM-768 implementation is identified as a near-term engineering follow-up. A complete post-quantum migration entails three further replacements. (i) The linkable ring signatures rely on the discrete logarithm problem and should be replaced with a lattice-based linkable ring signature (e.g., one constructed from Module-SIS, the assumption underlying ML-DSA in FIPS 204 [16]). (ii) Paillier homomorphic tallying is vulnerable to Shor’s algorithm and should be migrated to a fully homomorphic encryption scheme such as BFV, BGV, or CKKS. (iii) Threshold decryption should likewise be reformulated over the chosen lattice-based scheme. A staged migration is realistic: signature replacement first (ML-DSA is already standardised), homomorphic-tally replacement second (lattice FHE has acceptable performance for the small plaintext domain $\{0, 1\}$ used here), and threshold-decryption replacement last. Until that migration is complete, the present hybrid construction provides confidentiality against harvest-now-decrypt-later adversaries, which is the most pressing of the post-quantum threats for ballots that must remain secret for decades.
- **Single Orderer Limitation:** Our current deployment uses a single Raft orderer node, which is sufficient for the development and benchmarking workload reported in this thesis but provides no fault tolerance: any orderer crash halts ordering. Production deployment requires at least three orderer nodes to tolerate one crash failure under Raft, and for high-stakes elections where orderer nodes may also be adversarial, migrating to a Byzantine-fault-tolerant consensus mechanism (e.g., SmartBFT in Fabric v3.x) would strengthen the system’s resilience to malicious nodes.
- **Fixed Ring Size:** The ring size is fixed at $|\mathcal{R}| = 100$ in our implementation. While this provides strong anonymity, the signer is statistically indistinguishable among the 100 ring members under DDH, a dynamic ring size mechanism that adapts to the voter pool size and latency requirements would provide more flexible deployment options.

- **Blockchain Scalability at Extreme Load:** Our Hyperledger Caliper benchmarks revealed that the single-Raft, two-peer reference configuration shows commit-time degradation at transaction volumes above 25,000, attributable to a combination of block-batching at the orderer, version-vector validation on shared election-state reads, and Fabric Gateway connection-pool capacity. Production deployments for national-scale elections would require careful infrastructure engineering including multi-orderer consensus, Gateway connection-pool tuning, horizontal peer scaling, and load balancing.
- **Mobile Client Implementation:** Our current implementation provides a server-side API but lacks a dedicated mobile client application. For real-world deployment, a mobile application with on-device biometric authentication, secure enclave key storage, and offline credential management would improve voter accessibility.
- **Threshold Parameter Optimization:** The current threshold Paillier key generation takes approximately 30 seconds due to safe prime generation for the DJN scheme. For elections with frequent key rotation, exploring alternative threshold schemes (e.g., based on class groups) could reduce setup time.
- **Large-Scale User Study:** Our system has been evaluated through automated benchmarks and unit tests but has not undergone a large-scale user study with real voters. A deployment trial with hundreds of participants would provide valuable data on usability, biometric enrollment success rates, and voter confidence.

The limitations we identify above and the future work directions suggest a roadmap for advancing our framework toward production-ready deployment. By addressing formal verification, full post-quantum migration, infrastructure scaling, and user experience design, the system can evolve into a practical solution for secure, coercion-resistant democratic elections.

In summary, the proposed framework achieves the following key quantitative results:

1. **6.1 μ s** per-vote per-candidate tally cost, constant across 100 to 100 000 voters and across $m \in [2, 50]$ candidates ($O(N)$ per candidate, $O(N \cdot m)$ total).
2. **68.3 ms** per-vote primitive floor of the multi-protocol cast pipeline (sum of per-phase Table 4.3; Pedersen/SMDC issuance is amortised at registration time and excluded).
3. **~200 TPS** sustained chaincode throughput with 100% success at 10 000 transactions, measured by Hyperledger Caliper at the chaincode level (an end-to-end Caliper run through the integrated cast endpoint is left as future work).
4. **5.35 ms** Kyber-768 hybrid-encryption overhead per vote ($\approx 7.8\%$ of the per-vote primitive floor) for post-quantum protection.

5. **Multi-protocol cryptographic stack** integrated in a single pipeline, with a broader composition of cryptographic primitives than any system surveyed in Chapter 2.
6. **188 tests** (159 unit/integration + 29 benchmarks) with 100% pass rate and 78–92% coverage on core packages.

In summary, we believe our study presents a promising and rigorous approach to designing next-generation blockchain-based e-voting systems. Among the works surveyed in Chapter 2, no prior framework simultaneously provides ballot privacy, vote validity, voter anonymity, coercion resistance, privacy-preserving aggregation, a post-quantum encryption layer, and a Caliper-benchmarked Hyperledger Fabric deployment with thorough empirical validation. While the encryption layer achieves post-quantum protection via Kyber-768 (CRYSTALS-Kyber round-3 via CIRCL; migration to the FIPS-203 ML-KEM-768 standard is identified as a near-term engineering follow-up), the ring signature and homomorphic tallying components remain classically secure, with lattice-based replacements identified as longer-term future work. We believe that the design principles, cryptographic constructions, and engineering practices presented in this thesis can serve as a foundation for future research in this critical area of democratic technology.

Reference

- [1] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, and J. Yellick, “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in *Proceedings of the Thirteenth EuroSys Conference (EuroSys '18)*, pp. 1–15, ACM, 2018.
- [2] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system.” <https://bitcoin.org/bitcoin.pdf>, 2008. Original blockchain whitepaper.
- [3] M. A. Specter, J. Koppel, and D. Weitzner, “The ballot is busted before the blockchain: A security analysis of Voatz, the first internet voting application used in U.S. federal elections,” in *29th USENIX Security Symposium*, pp. 1535–1553, USENIX Association, 2020.
- [4] National Institute of Standards and Technology, “Advanced Encryption Standard (AES),” Tech. Rep. FIPS PUB 197, NIST, 2001.
- [5] R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [6] G. Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger,” 2014. Ethereum Project Yellow Paper.
- [7] Y. Ucbas, A. Eleyan, M. Hammoudeh, and M. Alohalay, “Performance and scalability analysis of Ethereum and Hyperledger Fabric,” *IEEE Access*, vol. 11, pp. 67156–67167, 2023.
- [8] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *Advances in Cryptology — EUROCRYPT '99*, vol. 1592 of *Lecture Notes in Computer Science*, pp. 223–238, Springer, 1999.
- [9] D. Bernhard, O. Pereira, and B. Warinschi, “How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios,” in *Advances in Cryptology —*

- ASIACRYPT 2012, vol. 7658 of *Lecture Notes in Computer Science*, pp. 626–643, Springer, 2012.
- [10] J. K. Liu, V. K. Wei, and D. S. Wong, “Linkable spontaneous anonymous group signature for ad hoc groups,” in *Information Security and Privacy — ACISP 2004*, vol. 3108 of *Lecture Notes in Computer Science*, pp. 325–335, Springer, 2004.
 - [11] K. Talwar, S. Wang, A. McMillan, V. Jina, V. Feldman, B. Basile, A. Cahill, Y. S. Chan, M. Chatzidakis, O. Chick, M. Chitnis, S. Ganta, Y. Goren, F. Granqvist, K. Guo, F. Jacobs, O. Javidbakht, A. Liu, R. Low, D. Mascenik, S. Myers, D. Park, W. Park, G. Parsa, T. Pauly, C. Priebe, R. Rishi, G. Rothblum, M. Scaria, L. Song, C. Song, K. Tarbe, S. Vogt, L. Winstrom, and S. Zhou, “Samplable anonymous aggregation for private federated data analysis,” in *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS ’24)*, ACM, 2024.
 - [12] National Institute of Standards and Technology, “Module-lattice-based key-encapsulation mechanism standard (FIPS 203),” Tech. Rep. FIPS 203, U.S. Department of Commerce, Aug. 2024. Standardizes ML-KEM (formerly CRYSTALS-Kyber).
 - [13] I. Damgård, M. Jurik, and J. B. Nielsen, “A generalization of Paillier’s public-key system with applications to electronic voting,” *International Journal of Information Security*, vol. 9, no. 6, pp. 371–385, 2010.
 - [14] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
 - [15] P. W. Shor, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
 - [16] National Institute of Standards and Technology, “Module-lattice-based digital signature standard (FIPS 204),” Tech. Rep. FIPS 204, U.S. Department of Commerce, Aug. 2024. Standardizes ML-DSA (formerly CRYSTALS-Dilithium).
 - [17] A. Juels, D. Catalano, and M. Jakobsson, “Coercion-resistant electronic elections,” in *Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society (WPES ’05)*, pp. 61–70, ACM, 2005.
 - [18] M. R. Clarkson, S. Chong, and A. C. Myers, “Civitas: Toward a secure voting system,” in *2008 IEEE Symposium on Security and Privacy*, pp. 354–368, IEEE, 2008.
 - [19] T. P. Pedersen, “Non-interactive and information-theoretic secure verifiable secret sharing,” in *Advances in Cryptology — CRYPTO ’91*, vol. 576 of *Lecture Notes in Computer Science*, pp. 129–140, Springer, 1991.

- [20] J. Zhang, C. Wu, R. S. Sherratt, and J. Wang, "An improved secure and efficient E-Voting scheme based on blockchain systems," *IEEE Internet of Things Journal*, vol. 12, no. 7, pp. 8628–8640, 2025.
- [21] H. Baniata and G. Caluna, "BP-Vot: Blockchain-based e-voting using smart contracts, differential privacy and self-sovereign identities," *IEEE Access*, vol. 13, pp. 46106–46123, 2025.
- [22] M. J. H. Faruk, F. Alam, M. Islam, and A. Rahman, "Transforming online voting: A novel system utilizing blockchain and biometric verification for enhanced security, privacy, and transparency," *Cluster Computing*, vol. 27, no. 4, pp. 4015–4034, 2024.
- [23] K. Yuan, P. Sang, J. Ge, B. Zhou, and C. Jia, "A timed-release E-Voting scheme based on Paillier homomorphic encryption," *IEEE Transactions on Sustainable Computing*, vol. 9, no. 5, pp. 740–753, 2024.
- [24] Z. Yin, B. Zhang, A. Nastenkov, R. Oliynykov, and K. Ren, "A scalable coercion-resistant voting scheme for blockchain decision-making," *IEEE Transactions on Dependable and Secure Computing*, 2024. Also available as IACR ePrint 2023/1578.
- [25] H. O. Ohize, A. J. Onumanyi, B. U. Umar, L. A. Ajao, R. O. Isah, E. M. Dogo, B. K. Nuhu, O. M. Olaniyi, J. G. Ambafi, V. B. Sheidu, and M. M. Ibrahim, "Blockchain for securing electronic voting systems: A survey of architectures, trends, solutions, and challenges," *Cluster Computing*, 2024.
- [26] C.-P. Schnorr, "Efficient signature generation by smart cards," *Journal of Cryptology*, vol. 4, no. 3, pp. 161–174, 1991.
- [27] A. Fiat and A. Shamir, "How to prove yourself: Practical solutions to identification and signature problems," in *Advances in Cryptology — CRYPTO '86*, vol. 263 of *Lecture Notes in Computer Science*, pp. 186–194, Springer, 1986.
- [28] R. L. Rivest, A. Shamir, and Y. Tauman, "How to leak a secret," in *Advances in Cryptology — ASIACRYPT 2001*, vol. 2248 of *Lecture Notes in Computer Science*, pp. 552–565, Springer, 2001.
- [29] National Institute of Standards and Technology, "Secure hash standard (SHS)," Tech. Rep. FIPS PUB 180-4, National Institute of Standards and Technology, 2015.
- [30] National Institute of Standards and Technology, "SHA-3 standard: Permutation-based hash and extendable-output functions," Tech. Rep. FIPS PUB 202, National Institute of Standards and Technology, 2015.
- [31] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication." RFC 2104, Internet Engineering Task Force, 1997.

- [32] National Institute of Standards and Technology, “The keyed-hash message authentication code (HMAC),” Tech. Rep. FIPS PUB 198-1, National Institute of Standards and Technology, 2008.
- [33] M. J. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” Tech. Rep. NIST SP 800-38D, National Institute of Standards and Technology, 2007.
- [34] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*, pp. 1175–1191, ACM, 2017.
- [35] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm (Raft),” in *USENIX Annual Technical Conference (ATC)*, pp. 305–319, 2014.
- [36] B. Adida, “Helios: Web-based Open-Audit Voting,” in *17th USENIX Security Symposium (USENIX Security 08)*, pp. 335–348, USENIX Association, 2008.
- [37] R. Cramer, I. Damgård, and B. Schoenmakers, “Proofs of Partial Knowledge and Simplified Design of Witness Hiding Protocols,” in *Advances in Cryptology – CRYPTO '94*, vol. 839 of LNCS, pp. 174–187, Springer, 1994.
- [38] I. Damgård and M. Jurik, “A Generalisation, a Simplification and Some Applications of Paillier’s Probabilistic Public-Key System,” in *Public Key Cryptography – PKC 2001*, vol. 1992 of LNCS, pp. 119–136, Springer, 2001.
- [39] D. Dolev and A. C. Yao, “On the security of public key protocols,” *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [40] R. Canetti, “Universally Composable Security: A New Paradigm for Cryptographic Protocols,” in *IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 136–145, IEEE, 2001. Updated full version: Cryptology ePrint Archive 2000/067.
- [41] M. Bellare and P. Rogaway, “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols,” in *ACM Conference on Computer and Communications Security (CCS)*, pp. 62–73, ACM, 1993.
- [42] M. Bellare and G. Neven, “Multi-Signatures in the Plain Public-Key Model and a General Forking Lemma,” in *ACM Conference on Computer and Communications Security (CCS)*, pp. 390–399, ACM, 2006.
- [43] D. Pointcheval and J. Stern, “Security Arguments for Digital Signatures and Blind Signatures.” *Journal of Cryptology*, vol. 13, no. 3, pp. 361–396, 2000.

- [44] D. Hofheinz, K. Hövelmanns, and E. Kiltz, “A Modular Analysis of the Fujisaki-Okamoto Transformation,” in *Theory of Cryptography Conference (TCC)*, vol. 10677 of *LNCS*, pp. 341–371, Springer, 2017.
- [45] R. Cramer and V. Shoup, “Design and Analysis of Practical Public-Key Encryption Schemes Secure Against Adaptive Chosen Ciphertext Attack,” in *SIAM Journal on Computing*, vol. 33, pp. 167–226, 2003.
- [46] L. Ackermann, M. Arapinis, P. Georgiou, N. Lamprou, L. Mareková, and T. Zacharias, “E-cclasia: Universally composable self-tallying elections over anonymous broadcast,” *IACR Communications in Cryptology*, vol. 2, Oct. 2025. Earlier version: IACR ePrint 2020/513.
- [47] Hyperledger Foundation, “Hyperledger Caliper: Blockchain performance benchmark framework.” <https://hyperledger.github.io/caliper/>, 2024. v0.6.0.
- [48] Cloudflare, Inc., “CIRCL: Cloudflare interoperable reusable cryptographic library.” <https://github.com/cloudflare/circl>, 2024. Go library implementing post-quantum cryptographic primitives including ML-KEM (Kyber).
- [49] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and ProVerif,” *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.

A. ProVerif Formal Verification Artefacts

This appendix reproduces the ProVerif source models and the corresponding tool output transcripts that underpin the formal verification results summarised in Table 4.10 of Chapter 4. The three models target ballot privacy, individual verifiability, and voter eligibility, respectively; each is verified by ProVerif 2.05 [49] under the symbolic Dolev–Yao adversary model. All three queries return true.

The model files are versioned in the project repository under the directory `proverif/`. The shell script `proverif/run_verification.sh` reproduces all three runs in sequence and prints the `RESULT` lines.

A.1 Reproducing the Verification

Install ProVerif via OPAM (OCaml package manager):

```
opam install proverif
```

Run all three verifications:

```
cd proverif
./run_verification.sh
```

The expected condensed output is:

```
RESULT Observational equivalence is true.
RESULT inj-event(BallotRecorded(b_2)) ==>
      inj-event(VoterCastBallot(pkV,b_2)) is true.
RESULT event(ValidBallotCast(pkV)) ==>
      event(VoterRegistered(pkV)) is true.
```

A.2 Ballot Privacy: `privacy.pv`

Ballot privacy is verified via observational equivalence: an adversary controlling the network cannot distinguish between two honest voters swapping their ballots. The model uses the `diff [v0, v1]` construct to test whether the protocol is observationally equivalent under both ballot assignments.

Source: proverif/privacy.pv

```
(* CovertVote: Ballot Privacy via Observational Equivalence *)
(* NO queries allowed in this file -- diff mode only *)

type skey.
type pkey.
type vote.
type nonce.
type commitment.
type zkprf.
type signature.
type keyimage.

free ch: channel.
free bb: channel.
free sa2_a: channel [private].
free sa2_b: channel [private].

free v0: vote.
free v1: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.
fun commit(vote, nonce): commitment.
fun zkp_make(vote, nonce, commitment): zkprf.
fun ring_sign(bitstring, skey, pkey): signature.
fun key_image(skey): keyimage.
fun sa2_share_a(bitstring, nonce): bitstring.
fun sa2_share_b(bitstring, nonce): bitstring.
fun make_ballot(bitstring, commitment, zkprf, signature, keyimage): bitstring.

let SA2ServerA() = in(sa2_a, x: bitstring); 0.
let SA2ServerB() = in(sa2_b, x: bitstring); 0.

let VoterPrivacy(skV: skey, pkEA: pkey) =
  new r_enc: nonce;
  new r_com: nonce;
  new r_sa2: nonce;
  let v = diff[v0, v1] in
  let encVote = enc(v, pkEA, r_enc) in
  let com = commit(v, r_com) in
  let zk = zkp_make(v, r_com, com) in
  let sig = ring_sign(encVote, skV, pk(skV)) in
  let ki = key_image(skV) in
  let b = make_ballot(encVote, com, zk, sig, ki) in
  out(sa2_a, sa2_share_a(encVote, r_sa2));
  out(sa2_b, sa2_share_b(encVote, r_sa2));
  out(bb, b);
  0.

process
  new skEA: skey;
  let pkEA = pk(skEA) in
  out(ch, pkEA);
  (
    new skAlice: skey;
    new skBob: skey;
    out(ch, pk(skAlice));
```

```

        out(ch, pk(skBob));
        VoterPrivacy(skAlice, pkEA) |
        VoterPrivacy(skBob, pkEA) |
        SA2ServerA() |
        SA2ServerB()
    )

```

ProVerif output (excerpt)

```

$ proverif privacy.pv
...
RESULT Observational equivalence is true.
-----

```

Verification summary:

```

Observational equivalence is true.
-----

```

Interpretation. The “Observational equivalence is true” result means that for every possible adversary strategy, the trace of network observations is identical regardless of whether each voter cast v0 or v1. Therefore an adversary cannot infer how any individual voter voted from network-level observations alone.

A.3 Individual Verifiability: `verifiability.pv`

Individual verifiability is verified via injective correspondence: every recorded ballot on the bulletin board corresponds to exactly one authentic cast event by a registered voter. The query asserts $\text{BallotRecorded}(b) \implies \text{inj-event}(\text{VoterCastBallot}(\text{pkV}, b))$.

Source: `proverif/verifiability.pv`

```
(* CovertVote: Individual Verifiability *)
```

```

type skey.
type pkey.
type vote.
type nonce.
type commitment.
type zkprf.
type signature.
type keyimage.

free ch: channel.
free bb: channel.
free auth_bb: channel [private].
free sa2_a: channel [private].
free sa2_b: channel [private].

free v0: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.

```

```

fun commit(vote, nonce): commitment.
fun zkp_make(vote, nonce, commitment): zkprf.
fun ring_sign(bitstring, skey, pkey): signature.
fun key_image(skey): keyimage.
fun sa2_share_a(bitstring, nonce): bitstring.
fun sa2_share_b(bitstring, nonce): bitstring.
fun make_ballot(bitstring, commitment, zkprf, signature, keyimage): bitstring.

event VoterCastBallot(pkey, bitstring).
event BallotRecorded(bitstring).

query b: bitstring, pkV: pkey;
  event(BallotRecorded(b)) ==> inj-event(VoterCastBallot(pkV, b)).

let SA2ServerA() = in(sa2_a, x: bitstring); 0.
let SA2ServerB() = in(sa2_b, x: bitstring); 0.

let VoterVerifiable(skV: skey, v: vote, pkEA: pkey) =
  new r_enc: nonce;
  new r_com: nonce;
  new r_sa2: nonce;
  let encVote = enc(v, pkEA, r_enc) in
  let com = commit(v, r_com) in
  let zk = zkp_make(v, r_com, com) in
  let sig = ring_sign(encVote, skV, pk(skV)) in
  let ki = key_image(skV) in
  let b = make_ballot(encVote, com, zk, sig, ki) in
  event VoterCastBallot(pk(skV), b);
  out(sa2_a, sa2_share_a(encVote, r_sa2));
  out(sa2_b, sa2_share_b(encVote, r_sa2));
  out(auth_bb, b);
  0.

let BulletinBoard() =
  in(auth_bb, b: bitstring);
  event BallotRecorded(b);
  out(bb, b);
  0.

process
  new skEA: skey;
  let pkEA = pk(skEA) in
  out(ch, pkEA);
  (
    new skV: skey;
    VoterVerifiable(skV, v0, pkEA) |
    BulletinBoard() |
    SA2ServerA() |
    SA2ServerB()
  )

```

ProVerif output (excerpt)

```

$ proverif verifiability.pv
...
Starting query inj-event(BallotRecorded(b_2))
      ==> inj-event(VoterCastBallot(pkV,b_2))
goal reachable: b-inj-event(VoterCastBallot(...))
      -> inj-event(BallotRecorded(...))

```

The hypothesis occurs strictly before the conclusion.

```
RESULT inj-event(BallotRecorded(b_2)) ==>
      inj-event(VoterCastBallot(pkV,b_2)) is true.
```

Verification summary:

```
Query inj-event(BallotRecorded(b_2)) ==>
      inj-event(VoterCastBallot(pkV,b_2)) is true.
```

Interpretation. The injective correspondence guarantees that no recorded ballot is forged or duplicated: each `BallotRecorded` event on the bulletin board is matched to a unique prior `VoterCastBallot` event by an authenticated voter, providing individual verifiability.

A.4 Voter Eligibility: `eligibility.pv`

Voter eligibility is verified via causal correspondence: every valid ballot cast was preceded by a legitimate registration event with the registration authority. The query asserts $\text{ValidBallotCast}(\text{pkV}) \implies \text{VoterRegistered}(\text{pkV})$.

Source: `proverif/eligibility.pv`

```
(* CovertVote: Voter Eligibility *)
(* NO diff allowed -- query mode only *)

type skey.
type pkey.
type vote.
type credential.
type nonce.

free ch: channel.
free bb: channel.
free reg: channel [private].

free v0: vote.

fun pk(skey): pkey.
fun enc(vote, pkey, nonce): bitstring.
fun real_cred(skey): credential.
fun fake_cred(skey): credential.

event VoterRegistered(pkey).
event ValidBallotCast(pkey).

query pkV: pkey;
      event(ValidBallotCast(pkV)) ==> event(VoterRegistered(pkV)).

let RegistrationAuthority() =
  in(reg, voterPK: pkey);
  new skCred: skey;
```

```

    out(reg, (real_cred(skCred), fake_cred(skCred)));
    0.

let VoterEligible(skV: skey, v: vote, pkEA: pkey) =
  out(reg, pk(skV));
  in(reg, (realC: credential, fakeC: credential));
  event VoterRegistered(pk(skV));
  new r_enc: nonce;
  let encVote = enc(v, pkEA, r_enc) in
  event ValidBallotCast(pk(skV));
  out(bb, encVote);
  0.

process
  new skEA: skey;
  let pkEA = pk(skEA) in
  out(ch, pkEA);
  (
    new skV: skey;
    RegistrationAuthority() |
    VoterEligible(skV, v0, pkEA)
  )

```

ProVerif output (excerpt)

```

$ proverif eligibility.pv
...
Starting query event(ValidBallotCast(pkV))
==> event(VoterRegistered(pkV))
goal reachable: b-event(VoterRegistered(pk(skV[])))
&& mess(reg[], voterPK_1)
-> event(ValidBallotCast(pk(skV[])))

RESULT event(ValidBallotCast(pkV)) ==>
event(VoterRegistered(pkV)) is true.
-----
Verification summary:

Query event(ValidBallotCast(pkV)) ==>
event(VoterRegistered(pkV)) is true.
-----

```

Interpretation. The causal correspondence guarantees that any ballot accepted as “valid” is necessarily preceded by a successful registration of the same voter public key with the registration authority. This rules out unauthorised vote injection by unregistered parties.

A.5 Summary

Table 4.10 of Chapter 4 summarises the three results in tabular form: all three properties hold under the Dolev–Yao adversary model. Together they establish symbolic-model security against eavesdropping, replay, and man-in-the-middle attacks at the protocol level.

Stronger guarantees in the computational model (game-based or universally composable proofs) are identified as future work in Chapter 5.

Please use the following page format for the cover of your report.

A Coercion-Resistant Blockchain E-Voting Framework with Multi-Protocol Cryptography and Post-Quantum Security

by

Monowar Hossen Sourav
221002578

Md. Sajeeb Mia	Md. Naim Ahammed
221002251	221002599

Thesis for the degree of

Bachelor of Science in Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
GREEN UNIVERSITY OF BANGLADESH

SPRING 2026